Safety-Critical Time-Series Forecasting with spotforecast2-safe

Lecture notes for students with no prior knowledge of AI regulation

Thomas Bartz-Beielstein

2026-05-31

Table of contents

Preface **1**
A story to start with . . . 1
Who is bound by the AI Act? . . . 2
The four code rules (CR-1 to CR-4) . . . 2
Roadmap through the chapters . . . 5
How to read these notes with Claude . . . 5
Preparing your environment . . . 6

1 Switch on the audit log **9**
1.1 Why log at all? . . . 9
1.2 Code in action . . . 10
1.3 What is inside the log file? . . . 10
1.4 Visualisation: How an audit run unfolds . . . 11

2 Loading live data from ENTSO-E **15**
2.1 What the running example in the report was about . . . 15
2.2 AI Act link . . . 15
2.3 Code in action . . . 16
2.4 Reading and checking the data . . . 18
2.5 Visualisation: one year of German load . . . 19
2.6 Persisting state . . . 21

3 Annotate data gaps instead of inventing them **23**
3.1 Why not simply interpolate? . . . 23
3.2 AI Act link . . . 24
3.3 Loading data . . . 24

3.4 Inject an artificial anomaly . . . 25
3.5 Step 1 — Mark outliers . . . 26
3.6 Step 2 — Fill gaps *and* weight them . . . 27
3.7 Step 3 — `WeightFunction` as a picklable contract . . . 29
3.7.1 Edge case: a window entirely in the danger zone . . . 29
3.7.2 Picklability (CR-2) . . . 30
3.8 Visualisation: the danger zone made visible . . . 31
3.9 Persisting for follow-up chapters . . . 34

4 Handle data gaps explicitly 35
4.1 Why are data gaps an AI Act topic? . . . 35
4.2 The three modes of `LinearlyInterpolateTS` . . . 36
4.3 Code in action: the default is `raise` . . . 37
4.4 Endpoint gaps: here CR-3 shows its effect . . . 39
4.5 `ffill_bfill` as a deliberate opt-in . . . 39
4.6 `passthrough`: responsibility passed to the caller . . . 40
4.7 Visualisation: before/after picture of gap handling . . . 40
4.8 Which strategy when? — a comparison at a glance . . . 42
4.9 Persisting for follow-up chapters . . . 44

5 Separating training and evaluation range 47
5.1 Why not split randomly? . . . 47
5.2 Code in action . . . 48
5.3 Visualisation: the split on the time axis . . . 49

6 Calendar features and real weather data 53
6.1 Why does the model need more than the load itself? . . . 53
6.2 AI Act link . . . 54
6.3 Code in action: cyclic calendar features . . . 54
6.4 Code in action: weather . . . 56
6.5 Looking at weather and load together . . . 57
6.6 Concatenating calendar and weather into a single matrix . . . 59

7 Instantiate and train the forecaster **61**
7.1 What happens during training? . . . 61
7.2 AI Act link . . . 61
7.3 Code in action . . . 62
7.4 Run the training . . . 64
7.5 Which features shaped the model most strongly? . . . 65
7.6 Persist model for follow-up chapters . . . 66

8 Produce a 24-hour forecast **69**
8.1 Recursive multi-step forecasting . . . 69
8.2 AI Act link . . . 70
8.3 Code in action . . . 70
8.4 Visualisation: forecast, reality, baseline . . . 71
8.5 Persisting for follow-up chapters . . . 73

9 Measuring accuracy: MAE, MSE, RMSE, MAPE **75**
9.1 Why four measures instead of one? . . . 75
9.2 AI Act link . . . 76
9.3 Code in action . . . 76
9.4 Visualisation: geometry of the metrics . . . 77
9.5 When is which measure appropriate? . . . 78
9.6 Comparison with the baseline . . . 79

10 Rolling-origin backtest **81**
10.1 Why backtest at all? . . . 81
10.2 AI Act link . . . 81
10.3 Code in action . . . 82
10.4 Visualisation: the folds on the time axis . . . 84
10.5 Distribution of errors across folds . . . 86

11 Provenance and reproducibility **89**
11.1 What is this about? . . . 89
11.2 Which fields does `spotforecast2-safe` record automatically? 90
11.3 Data provenance from the audit log . . . 91

11.4 Visualisation: the provenance diagram . . . 92
11.5 How all four code rules come together here . . . 94
11.6 AI Act link, compact . . . 94
11.7 Persisting the provenance note . . . 95
11.8 Closing remark . . . 96

12 Challenge: live load forecasting **97**
12.1 Why a live challenge? . . . 97
12.2 AI Act link . . . 98
12.3 Rules at a glance . . . 98
12.4 Participation . . . 99
12.5 GitHub onboarding . . . 100
12.5.1 What is GitHub, what is your handle? . . . 100
12.5.2 Create an account . . . 101
12.5.3 Find your handle (for the registration email) . . . 101
12.5.4 Set up authentication locally . . . 102
12.5.5 Install the `gh` CLI (optional but practical) . . . 102
12.5.6 Fork and local clone . . . 103
12.6 Submission format . . . 104
12.7 Submission workflow . . . 105
12.7.1 One-time setup — preconditions checklist . . . 106
12.7.2 Submission cycle per target day . . . 106
12.7.3 When the validator is red . . . 109
12.8 Baseline implementation . . . 110
12.9 Scoring methodology in detail . . . 114
12.10 Evaluation architecture (GitHub Actions + Pages) . . . 115
12.11 Leaderboard . . . 116
12.12 Calendar . . . 117
Exercises (with Claude) . . . 117
Appendix: maintaining `teams.yml` (for lecturers) . . . 118
12.12.1 Fields . . . 118
12.12.2 Concrete example: handle two registrations . . . 119
12.12.3 Further registration waves . . . 121
12.12.4 Common pitfalls . . . 121

13 Live submission to the ddmo-sose-26 challenge 123
13.1 Overview . . . 123
13.2 Submission contract . . . 124
13.3 Registered teams . . . 125
13.4 Step-by-step student walkthrough . . . 126
13.4.1 Prerequisites . . . 126
13.4.2 Step 1: Clone the lecture and fork the leaderboard . . . 126
13.4.3 Step 2: Install the pipeline . . . 127
13.4.4 Step 3: Export your ENTSO-E API key . . . 127
13.4.5 Step 4: Set your team id . . . 127
13.4.6 Step 5: Generate the forecast and write the CSV . . . 127
13.4.7 Step 6: Validate locally before pushing . . . 128
13.4.8 Step 7: Open the submission PR . . . 128
13.5 Live example — generate $\hat{y}$ end-to-end . . . 129
13.5.1 Dynamic date constants . . . 130
13.5.2 Download today's ENTSO-E actuals . . . 131
13.5.3 Determine the live training cutoff and forecast horizon . 132
13.5.4 Build the live `ConfigEntsoe` . . . 135
13.5.5 Train with Optuna and predict forward through tomorrow 137
13.5.6 Visual Inspection . . . 139
13.5.7 Validate the schema and write the submission CSV . . . 140
13.5.8 Local validation with the leaderboard's own script . . . 141
13.6 What the render did NOT do . . . 142
13.7 Tomorrow's scoring . . . 142

14 Team 4 live submission — advanced setup 145
14.1 Overview . . . 145
14.2 Submission contract . . . 146
14.3 Team identity . . . 146
14.4 Prerequisites and walkthrough . . . 147
14.5 Live pipeline . . . 147
14.5.1 Dynamic date constants . . . 148
14.5.2 Download today's ENTSO-E actuals and ENTSO-E's own day-ahead `Forecasted Load` . . . 149

14.5.3 The interim energy-load frame . . . 150
14.5.4 Extract ENTSO-E's day-ahead `Forecasted Load` . . . 152
14.5.5 Native resolution: 15-minute actuals aggregated to hourly 153
14.5.6 Training cutoff and forecast horizon . . . 154
14.5.7 Data availability vs. the requested interval . . . 155
14.5.8 Visualization of the training data and ENTSO-E's own day-ahead `Forecasted Load` . . . 158
14.6 Lag structure: select `key_lags` from the data . . . 161
14.6.1 Advanced `ConfigEntsoe` . . . 164
14.6.2 Cross-validation geometry recap . . . 172
14.6.3 Step 1 — Endogenous data: prepare and profile . . . 181
14.6.4 Step 2 — Outlier detection and curation . . . 183
14.6.5 Step 3 — Missing-data imputation . . . 187
14.6.6 Step 4 — Exogenous features: weather, holidays, calendar, polynomial . . . 189
14.6.7 Processing of the Weather Data . . . 195
14.6.8 Step 5 — Train with SpotOptim and predict through tomorrow . . . 204
14.6.9 Step 6 — Feature importance and SHAP . . . 210
14.6.10 Visual inspection . . . 216
14.6.11 ENTSO-E's own day-ahead forecast vs. ours . . . 217
14.6.12 Validate the schema and write the submission CSV . . . 219
14.6.13 Local validation with the leaderboard's own script . . . 221
14.7 Commit and push the submission PR . . . 221
14.8 What changed vs. chapter 13 — summary . . . 222

15 ENTSO-E day-ahead baseline submission 225
15.1 Overview . . . 225
15.2 Submission contract . . . 226
15.3 Team identity . . . 226
15.4 Prerequisites and walkthrough . . . 227
15.5 Live pipeline . . . 227
15.5.1 Dynamic date constants . . . 228
15.5.2 Download ENTSO-E's day-ahead `Forecasted Load` . 229

15.5.3 Assemble the 24-hour target slice 231
15.5.4 Visual inspection 233
15.5.5 Validate the schema and write the submission CSV . . . 235
15.5.6 Local validation with the leaderboard's own script . . . 236
15.6 Commit and push the submission PR 237
15.7 Why a baseline team 238

References **241**

Preface

These lecture notes accompany a course on safety-critical time-series forecasting with the open-source package `spotforecast2-safe`. They are aimed at students with no prior knowledge of AI regulation and no deep experience with time-series analysis. Anyone who can say at the end of the course *why* the EU AI Regulation (European Parliament and Council 2024) exists, *which* four code rules a safety-critical library must obey, and *what* a complete load-forecasting pipeline built on real ENTSO-E and Open-Meteo data looks like, has reached the learning goal.

A story to start with

Imagine a transmission system operator scheduling a delivery of 65 gigawatts of electric power into the German grid for 08:00 tomorrow. The number 65 comes out of a software system that computed a machine-learning load forecast. The real load at 08:00 is 72 gigawatts. Seven gigawatts short — and in a fraction of a second it is decided whether the lights go out in a major Central-German city.

The story is invented, the risk is not. Software that prepares decisions inside critical infrastructure can have physical consequences from a silent failure that no single human can stop once the dispatch has been issued. It is precisely this concern that drove the European legislator to adopt a regulation on artificial intelligence (Regulation (EU) 2024/1689, abbreviated *AI Act* or *EU AI Act*) which sets binding requirements for high-risk AI systems: data governance, technical documentation, record-keeping, transparency, human oversight, as well as accuracy and robustness (Art. 9 to 15 AI Act).

The story also explains why this course is not pure statistics: a safety-critical forecasting library can no longer afford *silent* shortcuts. If data are missing, the system must halt and ask, not simply interpolate. If a training run is repeated, the result must remain bit-identical. If something can be logged, it is logged, because a supervisory authority may need to reconstruct years later who made which decision when.

Who is bound by the AI Act?

The AI Act classifies AI systems by risk (Art. 5 to 7). The pyramid below summarises the four levels: a narrow top of explicitly forbidden applications and a wide base of many systems with minimal risk:

The use case of this course — electric load forecasting for a grid operator — clearly lies in the second tier (high risk), because Annex III No. 2 of the AI Act explicitly lists systems for the management of critical infrastructure. The following chapters show how the obligations from Art. 9 to 15 of the AI Act can be fulfilled *technically* — not via additional prose documents, but via code that guarantees specific properties.

The four code rules (CR-1 to CR-4)

`spotforecast2-safe` operationalises the regulatory requirements through four non-negotiable code rules. They form the backbone of the entire course; every one of the ten chapters refers to at least one of them.

Rule	Short form	What it is about	AI Act link
CR-1	No dead code	Every function, class, branch is reached by a test or docstring example.	Art. 9, 13

Rule	Short form	What it is about	AI Act link
CR-2	Determinism	Same input, bit-identical output. Fixed seeds, fixed iteration order.	Art. 12, 15
CR-3	Fail-safe processing	Invalid inputs (NaN, wrong dtype) raise an *explicit* exception instead of being silently imputed.	Art. 10, 15
CR-4	Minimal CVE attack surface	A short, versioned deny-list of forbidden dependencies, enforced by a test against the lockfile.	Art. 15

The detailed wording of the rules and their connection to the IEC 62443-4-1 norm can be found in the package's companion report under *Design principles*. For this course the rules of thumb suffice:

- *CR-1* ensures that every line of code that ever runs has also been checked at some point.
- *CR-2* ensures that you can reproduce a result, even if you repeat it a year later.
- *CR-3* ensures that the system never silently guesses but always asks back when something is missing.
- *CR-4* ensures that the list of libraries your system depends on stays small — because each dependency is a potential entry point for security vulnerabilities.

In the ten chapters that follow, you will see how these rules translate into concrete safeguards — at the very moment when `spotforecast2-safe` does something that another library would just wave through.

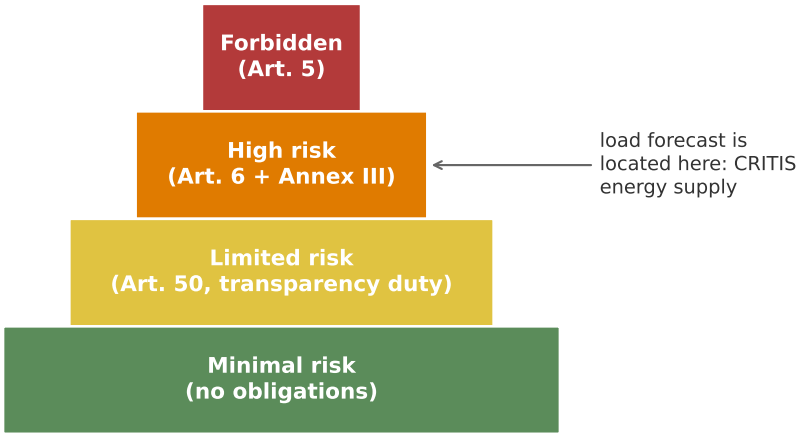


Figure 1: Risk pyramid of the AI Act. The top (forbidden) is very narrow — only a handful of applications are inadmissible under Art. 5. The base (minimal risk) is wide — the vast majority of AI systems fall into the lowest tier and have no obligations. The load forecast covered by this course falls into the second tier (high-risk) under Annex III No. 2 of the AI Act (management and operation of critical infrastructure).

Roadmap through the chapters

The lecture notes follow the order of the running example "electric load forecasting": a complete load-forecasting pipeline for an ENTSO-E market zone, from data acquisition to reproducibility audit.

Chapter	Content	Code rule	AI Act
01	Switch on the audit log	CR-1	Art. 12
02	Load live data from ENTSO-E	CR-2	Art. 10
03	Annotate data anomalies explicitly	**CR-3**	Art. 10
04	Handle data gaps explicitly	**CR-3**	Art. 10
05	Separate training and evaluation range	CR-2	Art. 10
06	Calendar features + real weather data	CR-3	Art. 10
07	Instantiate and train the forecaster	CR-2, **CR-4**	Art. 9, 15
08	Produce a 24-hour forecast	CR-1	Art. 13
09	Measure accuracy (MAE, RMSE, MAPE)	CR-2, CR-3	**Art. 15**
10	Rolling-origin backtest	CR-2	Art. 9
11	Provenance and reproducibility	CR-1 to CR-4	Art. 11, 12
12	Live challenge: forecasting energy consumption	CR-1 to CR-4	Art. 10, 12, 13, 15

How to read these notes with Claude

Every chapter ends with an *Exercises* box. The exercises are phrased so that you can paste them directly into the AI assistant Claude (e.g. https://claude.ai or *Claude Code* in the terminal) to explore the source code of `spotforecast2-safe` on your own.

A typical exercise looks like this:

> *Ask Claude:* "Show me the source code of `spotforecast2_safe.preprocessing.Linearly`. Explain line by line how the `on_missing` parameter changes the behaviour. Write a Python example that compares all three permitted values ('raise', 'ffill_bfill', 'passthrough') on the same data snippet."

We recommend really putting each exercise to Claude rather than googling the answer yourself. The learning effect comes from the dialogue in which you question Claude's explanation, push back on an insufficient answer, ask for a concrete example, or probe for the connection to the AI Act or to one of the four code rules.

Preparing your environment

For the live-data chapters (02, 06) you need two things:

- A free *ENTSO-E Restful API* key. You can request it at https://transparency.entsoe.eu/ after registering with a professional email address. Export it in your shell before rendering:

  ```
  export ENTSOE_API_KEY="your-token-here"
  ```

- An active internet connection for Open-Meteo. Open-Meteo requires no key for the archive or forecast API.

The render command is:

```
cd lecture
QUARTO_PYTHON=$HOME/workspace/Lehre.d/ddmo-sose-26/.venv/bin/python3 quarto r
```

When you run this command a second time, the once-fetched ENTSO-E and weather data are served from `_freeze/` and `_cache/`; a re-render then takes

under thirty seconds. Any change to a chapter's source invalidates only that chapter's cache.

```
import os

if "ENTSOE_API_KEY" not in os.environ:
    raise RuntimeError(
        "Set ENTSOE_API_KEY in the shell before rendering chapter 02. "
        "Request a key at: https://transparency.entsoe.eu/"
    )
print("Environment looks good. You can now run `quarto render`.")
```

Determinism despite live data

Live APIs return time-dependent values. So that our pipeline can still satisfy the CR-2 condition of bit-identical reproducibility, we pin the endpoints of the time window under consideration in a single constant that is passed to all chapters:

```
DATA_START = "2024-01-01 00:00+00:00"
DATA_END   = "2024-12-31 23:00+00:00"
COUNTRY    = "DE"
```

That way all students get identical data for an identical time window.

1 Switch on the audit log

> **Learning objective**
>
> You can explain *why* a safety-critical library has to keep a record of what it does from the very first line of code on, and you can switch on `spotforecast2-safe`'s audit log with two lines of code.

1.1 Why log at all?

Imagine that the load-forecasting system from the introduction really did predict 65 instead of 72 gigawatts at 08:00, and the grid is therefore severely under-supplied. The supervisory authority asks: *Which data were available to the model at 06:00, with which hyperparameters was it trained, which version of the library was running in production?* If the answers are not in a verifiable log, the incident is considered unreconstructable — and the operator has breached their duty under Art. 12 of the AI Act (*record-keeping*).

`spotforecast2-safe` therefore writes every action — initialisation of the forecaster, data download, training run, prediction — into a structured log. We switch this log on *first*, before doing anything else, because the initialisation itself is already a log-worthy action. This practice instantiates both Art. 12 of the AI Act and the discipline of a structured audit log anchored in process rule PR-4. It connects directly to code rule **CR-1** (no dead code): every function that is logged *has*, by definition, been executed, hence is not dead.

1.2 Code in action

```
import logging
from pathlib import Path
from spotforecast2_safe.manager.logger import setup_logging

# We write into a project-local log directory:
log_dir = Path("_cache/logs")
log_dir.mkdir(parents=True, exist_ok=True)

logger, log_path = setup_logging(level=logging.INFO, log_dir=log_dir)
logger.info("Course run starts: load-forecasting pipeline (chapter 01).")
print(f"Log file: {log_path.name if log_path else '<stdout only>'}")
```

```
2026-05-31 21:50:57,585 - task_safe_n_to_1 - INFO - Persistent logging initia
2026-05-31 21:50:57,586 - task_safe_n_to_1 - INFO - Course run starts: load-
forecasting pipeline (chapter 01).
Log file: task_safe_n_to_1_20260531_215057.log
```

The call returns two things: a `logger`, with which we record every relevant action from now on, and a `log_path` pointing to the concrete file where the log lives. The default logger name is `task_safe_n_to_1`. From now on every function in `spotforecast2-safe` that logs its own steps writes into the same file — without us having to forward where logging happens.

1.3 What is inside the log file?

```
if log_path is not None and log_path.exists():
    print("--- first line of the log ---")
    print(log_path.read_text().splitlines()[0])
else:
    print("No file handler active; the log only goes to stdout.")
```

```
--- first line of the log ---
{"schema_version":"1.0.0","timestamp_utc":"2026-05-31T19:50:57.585840Z","logger":"task_safe_
```

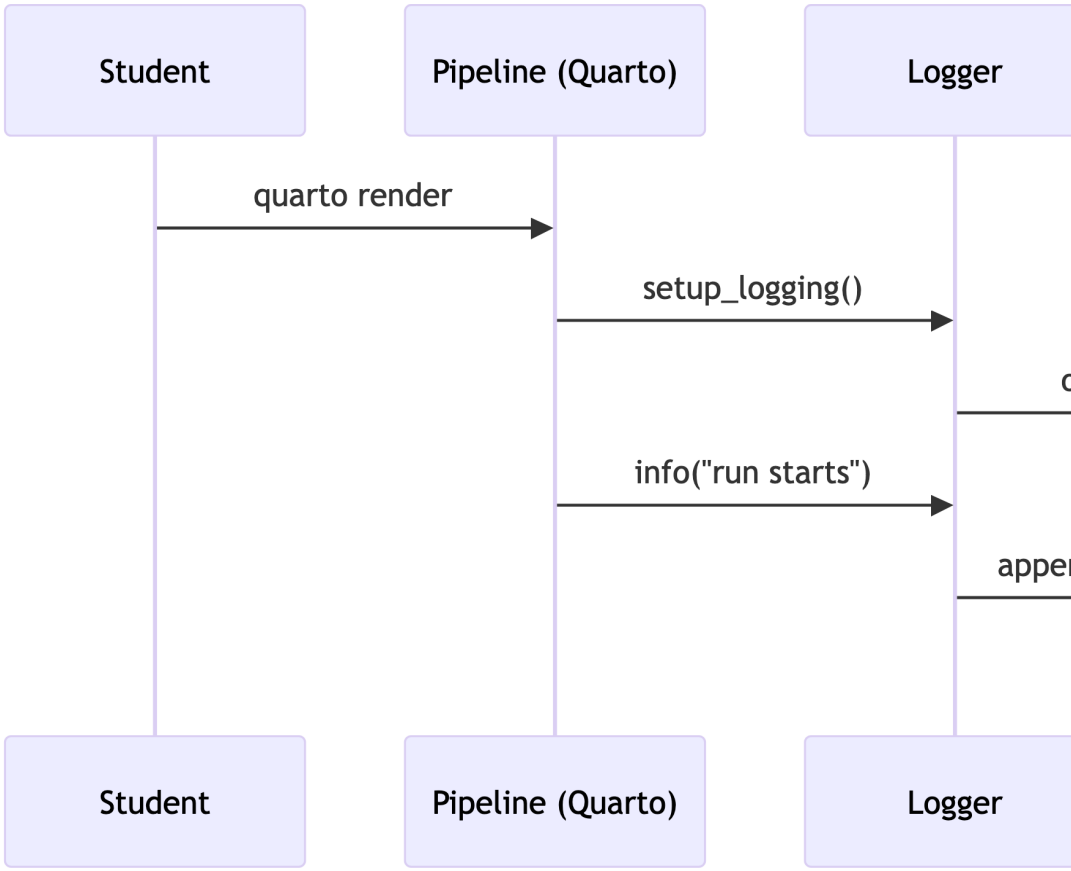
Every entry contains timestamp, logger name, level (INFO/WARNING/ERROR/DEBUG) and text. These four fields are enough for a later auditor to reconstruct the exact order and content of every action.

1.4 Visualisation: How an audit run unfolds

The following sequence flow shows which actors talk to each other while the pipeline renders. You will fill it out step by step in the coming chapters — each chapter adds a new arrow.

```
sequenceDiagram
    participant Student
    participant Pipeline as Pipeline (Quarto)
    participant Logger
    participant File as Log file

    Student->>Pipeline: quarto render
    Pipeline->>Logger: setup_logging()
    Logger->>File: open, write header
    Pipeline->>Logger: info("run starts")
    Logger->>File: append line with timestamp
    Note over File: later auditor reads here
```



Exercises (with Claude)

1. *Source-code exploration.* Ask Claude to display the source of `spotforecast2_safe.manager.logger.setup_logging` and to explain how the function makes sure that file creation does *not* abort the entire run when the directory is read-only. Which code rule addresses this behaviour?

2. *Comparison with `print`.* Discuss with Claude why `print()` would be *unsuitable* for an audit log. At least three traceable reasons should be given, each tied to Art. 12 of the AI Act or to one of the four code rules.
3. *Advanced exercise.* Have Claude write a mini pipeline that performs two training runs with different seeds and, from the audit log alone, figures out in which order the runs took place. Explain why exactly this information is decisive for a later reproduction.

2 Loading live data from ENTSO-E

> **Learning objective**
>
> You can use the function `download_new_data` from `spotforecast2_safe.downloader` to drop a real electric-load time series for an ENTSO-E market zone into a local CSV, and you can explain how this function guarantees CR-2 (determinism) despite the live API call.

2.1 What the running example in the report was about

The running example in the companion report generates a *synthetic* load series, because the document must render deterministically and offline. For a course that wants to teach real engineering practice, we replace the synthetic series with real data from the *ENTSO-E Transparency Platform* — the central database of the European Network of Transmission System Operators for Electricity. ENTSO-E publishes hourly the actual and forecast load for every market zone in Europe.

2.2 AI Act link

Data origin is a central concern of the AI Act: Art. 10(2) obliges providers and deployers to maintain *data management and data governance procedures* which,

among other things, document origin, collection method and assumptions about data processing. By pulling the data directly from the ENTSO-E API rather than from a spreadsheet of unknown origin, we keep the provenance explicit and documentable.

Code rule **CR-2** (determinism) would be jeopardised by a live API, because the API returns different values tomorrow than today. We solve this by pinning the date *window*: all students load exactly the same period (`DATA_START` to `DATA_END`), therefore receive byte-identical values, and can compare their results with each other.

2.3 Code in action

```
import os
from pathlib import Path

# Keep cache directory local to the book rather than in the student's home:
DATA_HOME = Path(__file__).parent.resolve() / "_cache" / "spotforecast2_data"
    if "__file__" in dir() else Path.cwd() / "_cache" / "spotforecast2_data"
DATA_HOME.mkdir(parents=True, exist_ok=True)
os.environ["SPOTFORECAST2_DATA"] = str(DATA_HOME)

# Fixed constants: the same window for every student
DATA_START = "202401010000"   # 2024-01-01 00:00 UTC
DATA_END   = "202412312300"   # 2024-12-31 23:00 UTC
COUNTRY    = "DE"             # ENTSO-E country code: Germany-Luxembourg mar

print(f"Data home: {DATA_HOME}")
print(f"Window:    {DATA_START} to {DATA_END}")
print(f"Country:   {COUNTRY}")
```

```
Data home: /Users/bartz/workspace/Lehre.d/ddmo-sose-26/lecture/_cache/spotfo
```

```
Window:    202401010000 to 202412312300
Country:   DE
```

```python
import logging
from spotforecast2_safe.downloader.entsoe import download_new_data
from spotforecast2_safe.data.fetch_data import fetch_data, get_data_home

logging.getLogger("spotforecast2_safe.downloader.entsoe").setLevel(logging.INFO)

interim = get_data_home() / "interim" / "energy_load.csv"

if interim.exists():
    print(f"Cache hit, no download needed: {interim.relative_to(DATA_HOME)}")
else:
    api_key = os.environ.get("ENTSOE_API_KEY")
    if not api_key:
        raise RuntimeError(
            "ENTSOE_API_KEY is not set. Register for free at "
            "https://transparency.entsoe.eu/, obtain a token and export "
            "it before rendering: export ENTSOE_API_KEY=...")
    download_new_data(
        api_key=api_key,
        country_code=COUNTRY,
        start=DATA_START,
        end=DATA_END,
        force=True,
    )
    print(f"Download complete, stored at {interim.relative_to(DATA_HOME)}")
```

```
Cache hit, no download needed: interim/energy_load.csv
```

After the download the merged CSV file lives at `_cache/spotforecast2_data/interim/energy_load.csv`.

On the next render of this chapter the block skips the download because the file already exists — and Quarto serves the pre-computed result from `_freeze/`.

2.4 Reading and checking the data

```
import pandas as pd

df = fetch_data(filename=str(interim))
df.index = pd.to_datetime(df.index, utc=True)

# We focus on the "Actual Load" column:
load_col = next(c for c in df.columns if "Actual" in c and "Load" in c)
y = df[load_col].astype(float).rename("load")

# ENTSO-E delivers 15-minute values for Germany; we aggregate to hourly
# means so that the index matches our hourly weather series and the
# hourly-lag logic of the forecaster.
if y.index.inferred_freq != "h":
    y = y.resample("h").mean()

print(f"Columns in the file: {list(df.columns)}")
print(f"Used column:         '{load_col}'")
print(f"Number of timestamps: {len(y):,}")
print(f"Time window:          {y.index.min()}  ...  {y.index.max()}")
print(f"Missing values:       {int(y.isna().sum())}")
```

```
Columns in the file: ['Forecasted Load', 'Actual Load']
Used column:         'Actual Load'
Number of timestamps: 8,783
Time window:          2024-01-01 00:00:00+00:00  ...  2024-
12-31 22:00:00+00:00
```

```
Missing values:       0
```

2.5 Visualisation: one year of German load

```python
import matplotlib.pyplot as plt
import matplotlib.dates as mdates

fig, ax = plt.subplots(figsize=(8.5, 3.5))
ax.plot(y.index, y.values / 1000, color="#1F4E79", linewidth=0.5)
ax.set_ylabel("Load [GW]")
ax.set_xlabel("Time")
ax.set_title(f"ENTSO-E market zone {COUNTRY}: 2024")
ax.grid(True, color="#E5E5E5", linewidth=0.5)
ax.xaxis.set_major_locator(mdates.MonthLocator())
ax.xaxis.set_major_formatter(mdates.DateFormatter("%b"))
for s in ("top", "right"):
    ax.spines[s].set_visible(False)
fig.tight_layout()
plt.show()
```

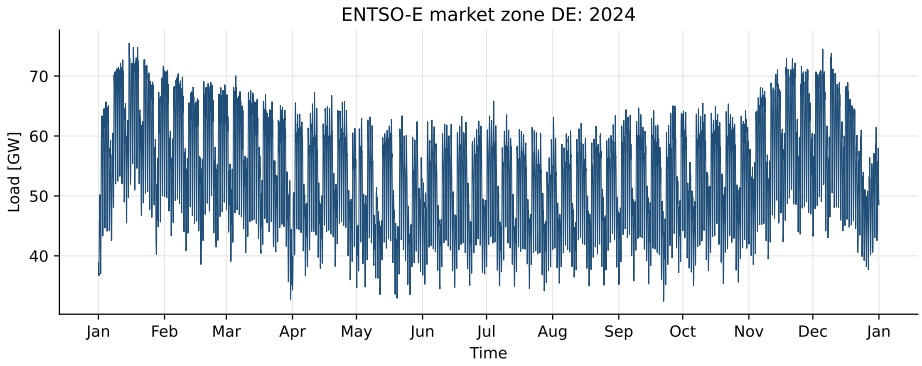


Figure 2.1: Actual electric load of the Germany-Luxembourg market zone over the year 2024, quarter-hourly resolution (aggregated by ENTSO-E to hourly). The daily and weekly rhythm is clearly visible, as well as the seasonal reduction during summer weeks and holidays.

```
week = y.loc["2024-02-05":"2024-02-12"]

fig, ax = plt.subplots(figsize=(8.5, 3.0))
ax.plot(week.index, week.values / 1000, color="#1F4E79", linewidth=1.0)
ax.set_ylabel("Load [GW]")
ax.grid(True, color="#E5E5E5", linewidth=0.5)
ax.xaxis.set_major_locator(mdates.DayLocator())
ax.xaxis.set_major_formatter(mdates.DateFormatter("%a %-d.%-m"))
for s in ("top", "right"):
    ax.spines[s].set_visible(False)
fig.tight_layout()
plt.show()
```

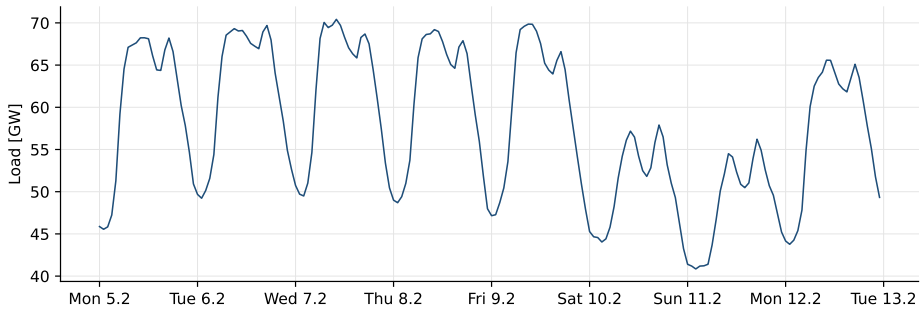


Figure 2.2: A sample week in February 2024. The weekly structure (weekday peak, weekend trough) and the typical morning/evening double peak are immediately visible.

2.6 Persisting state

We make the series `y` available to the following chapters. In a book the variables from one chapter are not automatically visible in the next; we therefore save the loaded state in a small Parquet file which the next chapters read back.

```
shared = Path("_cache/shared")
shared.mkdir(parents=True, exist_ok=True)
y.to_frame().to_parquet(shared / "load_de_2024.parquet")
print(f"Load series persisted: {shared / 'load_de_2024.parquet'}")
```

```
Load series persisted: _cache/shared/load_de_2024.parquet
```

Exercises (with Claude)

1. *Understanding the `force` parameter.* Have Claude explain the source of `download_new_data` line by line, with particular focus on the

block that checks `force=False` and `hours_diff < 24`. What happens when you call the function twice in quick succession with `force=False` after data are already on disk? Which code rule addresses this behaviour?

2. *Switching the market zone.* Ask Claude to show an alternative variant of the pipeline that loads market zone `FR` (France) instead of `DE` and plots both series side by side over the same February window. Which structural differences in the load stand out?
3. *Documenting data provenance.* Ask Claude which five fields a minimal data-provenance note under Art. 10 of the AI Act should contain. Have Claude generate a concrete example note for our run (source URL, fetch timestamp, market zone, time window, hash of the CSV).

3 Annotate data gaps instead of inventing them

> **Learning objective**
>
> You understand why a safety-critical library **annotates** broken rows instead of interpolating them, and you can assemble the three building blocks `mark_outliers`, `get_missing_weights` and `WeightFunction` into a preprocessing pipeline that complies with the AI Act.

3.1 Why not simply interpolate?

Linear interpolation (see Chapter 4) fills gaps with *invented* values. For yesterday's weather forecast that is uncritical; in a high-risk application under Annex III of the AI Act — such as load forecasting for a transmission system operator — it leads to two structural problems:

1. **Auditability disappears.** In retrospect an interpolated hour looks to the auditor exactly like a real measurement. But Art. 12 of the AI Act demands that *every* data-related action remains discoverable in the audit log.
2. **The model learns phantom data.** A forecaster that takes the interpolated value as a training target optimises its loss against invented values as well. The expected value of the learned function is biased in favour of the imputer — rather than in favour of reality.

The `spotforecast2-safe` library therefore follows the *"annotate, don't invent"* pattern: broken rows are set to `NaN`, filled in via forward/backward fill only far enough that the lag operator does not crash, and assigned weight **0** in the training loss. The model *sees* the compromised rows, but they do not pull the gradient.

3.2 AI Act link

Requirement	Article	How implemented here
Completeness, freedom from errors, representativeness	Art. 10(3)	`mark_outliers` makes outliers visible instead of smoothing them.
Record-keeping of every data action	Art. 12	`verbose=True` and the logger output record both annotation and imputation.
Accuracy & robustness	Art. 15	The training weight 0 keeps compromised rows out of the loss.
Code rule CR-2 (determinism)	—	`random_state=1234` guarantees reproducible outlier sets.
Code rule CR-3 (fail-safe)	—	A gap never leaves the pipeline silently.

3.3 Loading data

We work with the same ENTSO-E load series as every other chapter. Important: `get_missing_weights` expects a **DataFrame** (not a Series), because the function is multi-column-aware and checks NaN values across all columns simultaneously.

```
import numpy as np
import pandas as pd
```

```
from pathlib import Path

shared = Path("_cache/shared")
df = pd.read_parquet(shared / "load_de_2024.parquet")[["load"]].copy()
print(f"{len(df):,} hours loaded, {int(df.isna().sum().sum())} NaN present.")
print(df.head(3))
```

```
8,783 hours loaded, 0 NaN present.
                                load
Time (UTC)
2024-01-01 00:00:00+00:00  38818.1500
2024-01-01 01:00:00+00:00  37847.5650
2024-01-01 02:00:00+00:00  37123.2325
```

3.4 Inject an artificial anomaly

The real series contains no obvious spikes. To keep the example tangible, we insert one strikingly large outlier — a single hour that reaches three times the largest real load. *Such* values are typical sensor defects in practice (transmission errors, wrong scaling, back-calculation bugs).

```
df_demo = df.copy()
spike_pos = 200
spike_ts = df_demo.index[spike_pos]
df_demo.iloc[spike_pos, 0] = float(df["load"].max() * 3)

print(f"Spike inserted at {spike_ts}:  {df_demo.iloc[spike_pos, 0]:,.0f} MW")
print(df_demo.iloc[spike_pos-1:spike_pos+2])
```

```
Spike inserted at 2024-01-09 08:00:00+00:00:  226,524 MW
                                load
```

```
Time (UTC)
2024-01-09 07:00:00+00:00   70871.4925
2024-01-09 08:00:00+00:00  226524.3075
2024-01-09 09:00:00+00:00   70830.1750
```

3.5 Step 1 — Mark outliers

`mark_outliers` runs an `IsolationForest` on each column and replaces detected anomalies with `NaN`. The parameter `contamination` is the *expected* outlier ratio — keep it small, otherwise the algorithm treats regular load peaks as defects too.

```python
from spotforecast2_safe.preprocessing import mark_outliers

df_marked, labels = mark_outliers(
    df_demo.copy(),
    contamination=0.005,
    random_state=1234,
    verbose=True,
)

n_marked = int(df_marked["load"].isna().sum())
print(f"\nMarked rows: {n_marked} ({n_marked/len(df_marked):.2%})")
print(f"The injected spike at {spike_ts}:  {df_marked.loc[spike_ts, 'load']!r
```

```
Column 'load': Marked 0.5010% of data points as outliers.

Marked rows: 44 (0.50%)
The injected spike at 2024-01-09 08:00:00+00:00:  np.float64(nan)
```

The constructed spike was detected — the value is now `NaN`. The `IsolationForest` additionally finds about four dozen other inconspicuous-looking hours whose

hourly profile deviates from the 2024 pattern; in a production pipeline you would reconcile this suggestion list with domain expertise (dispatcher, grid operator). The list is part of the audit log.

3.6 Step 2 — Fill gaps *and* weight them

`get_missing_weights` does two things in a single call:

1. **Imputation**: forward-fill, then backward-fill — so that no `NaN` remains in the series (a technical prerequisite for the forecaster's lag operator).
2. **Weighting**: for every originally compromised row *and* for the following `window_size` hours, the weight is set to `0.0`. All other rows receive weight `1.0`.

The second point is the decisive step: because the recursive forecaster reads a **lag window** of size `window_size` at training time, every row *inside* that window after a gap is *also* contaminated — it would be trained against the imputed value as a lag feature. Weight 0 excludes it consistently.

```
from spotforecast2_safe.preprocessing import get_missing_weights

df_filled, weights = get_missing_weights(
    df_marked,
    window_size=72,
    verbose=True,
)

print(f"\nNaN after imputation:                {int(df_filled.isna().sum().sum())}")
print(f"Rows with training weight 0:        {int((weights == 0).sum()):,}")
print(f"Share of excluded rows:             {(weights == 0).mean():.2%}")
print(f"Row identity preserved:             {len(df_filled) == len(df)}")
```

```
Number of rows with missing values: 44
Percentage of rows with missing values: 0.50%
missing_indices: DatetimeIndex(['2024-01-09 08:00:00+00:00', '2024-
01-15 07:00:00+00:00',
               '2024-01-15 08:00:00+00:00', '2024-01-15 09:00:00+00:00',
               '2024-01-15 10:00:00+00:00', '2024-01-15 11:00:00+00:00',
               '2024-01-15 12:00:00+00:00', '2024-01-15 16:00:00+00:00',
               '2024-01-17 07:00:00+00:00', '2024-01-17 08:00:00+00:00',
               '2024-01-17 09:00:00+00:00', '2024-01-17 10:00:00+00:00',
               '2024-01-17 11:00:00+00:00', '2024-01-17 12:00:00+00:00',
               '2024-01-18 17:00:00+00:00', '2024-01-19 07:00:00+00:00',
               '2024-01-19 08:00:00+00:00', '2024-01-19 09:00:00+00:00',
               '2024-01-19 10:00:00+00:00', '2024-01-19 11:00:00+00:00',
               '2024-03-31 02:00:00+00:00', '2024-03-31 03:00:00+00:00',
               '2024-05-12 01:00:00+00:00', '2024-05-19 02:00:00+00:00',
               '2024-05-19 03:00:00+00:00', '2024-05-20 00:00:00+00:00',
               '2024-05-20 01:00:00+00:00', '2024-05-20 02:00:00+00:00',
               '2024-05-20 03:00:00+00:00', '2024-05-26 01:00:00+00:00',
               '2024-05-26 03:00:00+00:00', '2024-09-22 00:00:00+00:00',
               '2024-09-22 01:00:00+00:00', '2024-09-22 02:00:00+00:00',
               '2024-09-22 03:00:00+00:00', '2024-11-18 16:00:00+00:00',
               '2024-11-21 10:00:00+00:00', '2024-11-21 11:00:00+00:00',
               '2024-12-05 15:00:00+00:00', '2024-12-05 16:00:00+00:00',
               '2024-12-05 17:00:00+00:00', '2024-12-09 12:00:00+00:00',
               '2024-12-09 16:00:00+00:00', '2024-12-09 17:00:00+00:00'],
              dtype='datetime64[us, UTC]', name='Time (UTC)', freq=None)
Number of rows with missing weights after processing: 935
Percentage of rows with missing weights after processing: 10.65%

NaN after imputation:                0
Rows with training weight 0:       935
Share of excluded rows:            10.65%
Row identity preserved:            True
```

The series length is unchanged (`len(df_filled) == len(df)`). Every original hour still exists and is findable in the index. That is the central difference from interpolation: the gap is *declared*, not overwritten.

3.7 Step 3 — `WeightFunction` as a picklable contract

The forecaster accepts a **callable** for its `weight_func` parameter, not an array. Background: the lag matrix is `max(lags)` rows shorter than the original series; a positional handover would create off-by-N errors. `WeightFunction` resolves this index-based:

```
from spotforecast2_safe.preprocessing import WeightFunction

weight_func = WeightFunction(weights)
print(weight_func)

# Call with a timestamp index -> numpy array
sample = df_filled.index[:5]
print(f"\nWeights for {sample[0]} ... {sample[-1]}:")
print(weight_func(sample))
```

```
WeightFunction(weights_series with 8783 entries)

Weights for 2024-01-01 00:00:00+00:00 ... 2024-01-01 04:00:00+00:00:
[1. 1. 1. 1. 1.]
```

3.7.1 Edge case: a window entirely in the danger zone

If every row of a lag window carries weight 0, the weighted loss would collapse to 0/0. `WeightFunction` detects this and returns `None`; the forecaster interprets `None` as "uniform weighting", so the location remains operational.

```
zero_idx = weights.index[weights == 0]
window_in_gap = zero_idx[:3]
print("Index in danger zone:", window_in_gap.tolist())
print("Return value:", weight_func(window_in_gap))
```

```
WeightFunction: all sample weights for the requested index are zero (the winc
penalty zones). Returning None so ForecasterRecursive uses uniform weighting

Index in danger zone: [Timestamp('2024-01-09 08:00:00+0000', tz='UTC'), Times
01-09 09:00:00+0000', tz='UTC'), Timestamp('2024-01-09 10:00:00+0000', tz='U
Return value: None
```

3.7.2 Picklability (CR-2)

So that the entire forecaster — including the weight function — can later be reloaded deterministically, `weight_func` must be picklable. An ordinary closure would not be; a `WeightFunction` instance is:

```
import pickle

raw = pickle.dumps(weight_func)
weight_func_restored = pickle.loads(raw)

original  = weight_func(df_filled.index[:5])
restored  = weight_func_restored(df_filled.index[:5])
print("Original :", original)
print("Restored :", restored)
print("Identical:", bool(np.array_equal(original, restored)))
```

```
Original : [1. 1. 1. 1. 1.]
Restored : [1. 1. 1. 1. 1.]
Identical: True
```

3.8 Visualisation: the danger zone made visible

```
import matplotlib.pyplot as plt
import matplotlib.dates as mdates

view = slice(spike_pos - 5, spike_pos + 90)
idx_view = df.index[view]

fig, (ax_top, ax_mid, ax_bot) = plt.subplots(
    3, 1, figsize=(8.0, 6.5), sharex=True
)

# (1) after mark_outliers: spike is NaN
ax_top.plot(idx_view, df_marked["load"].iloc[view].values / 1000,
            "o-", color="#2E5C8A", linewidth=1.0, markersize=3)
ax_top.axvline(spike_ts, color="#B33A3A", linewidth=1.2,
               linestyle="--", label="annotated anomaly")
ax_top.set_ylabel("Load [GW]")
ax_top.set_title("after `mark_outliers`: spike -> NaN")
ax_top.legend(loc="upper right", fontsize=8)
ax_top.grid(True, color="#E5E5E5", linewidth=0.5)

# (2) after get_missing_weights: ffill+bfill closes the gap
ax_mid.plot(idx_view, df_filled["load"].iloc[view].values / 1000,
            "o-", color="#2E7D55", linewidth=1.0, markersize=3)
ax_mid.set_ylabel("Load [GW]")
ax_mid.set_title("after `get_missing_weights`: ffill+bfill (row identity preserved)")
ax_mid.grid(True, color="#E5E5E5", linewidth=0.5)

# (3) training weights
w_view = weights.iloc[view].values
ax_bot.fill_between(idx_view, 0, 1, where=(w_view == 0),
```

```
                    step="post", alpha=0.35, color="#888888",
                    label="training weight 0 (danger zone)")
ax_bot.plot(idx_view, w_view, drawstyle="steps-post",
            color="#1A1A1A", linewidth=1.2)
ax_bot.set_ylabel("Weight")
ax_bot.set_ylim(-0.1, 1.2)
ax_bot.set_title(f"Training weighting (window_size=72)")
ax_bot.legend(loc="lower right", fontsize=8)
ax_bot.grid(True, color="#E5E5E5", linewidth=0.5)

ax_bot.xaxis.set_major_locator(mdates.DayLocator(interval=1))
ax_bot.xaxis.set_major_formatter(mdates.DateFormatter("%-d.%-m"))

for ax in (ax_top, ax_mid, ax_bot):
    for s in ("top", "right"):
        ax.spines[s].set_visible(False)

fig.tight_layout()
plt.show()
```

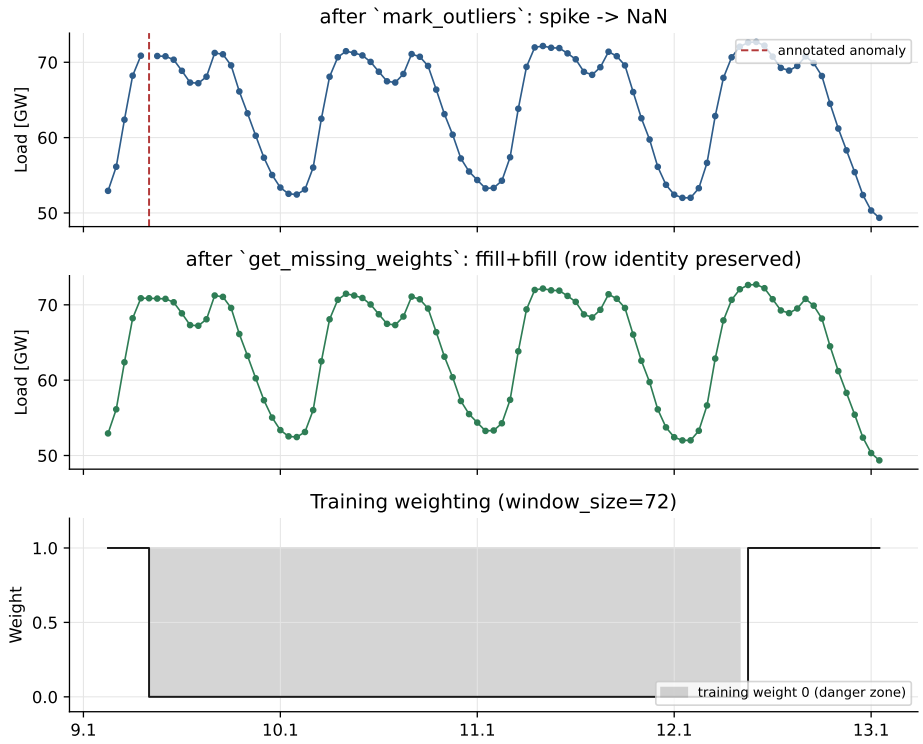


Figure 3.1: Annotate instead of impute, illustrated on hours 195-290 (around the injected spike). Top: the series after `mark_outliers` — the spike is visible as a NaN gap (red marker). Middle: the same series after `get_missing_weights` (ffill+bfill). Bottom: the binary training weight — the entire lag-window range after the gap is excluded from the loss with weight 0 (grey area).

The grey area illustrates what `window_size=72` means in practice: after each annotated anomaly, another 72 hours are excluded from training because the lag operator would read them as a feature vector. Larger gaps widen the zone accordingly.

3.9 Persisting for follow-up chapters

The annotation pipeline produces *two* artefacts, both deterministically reproducible: the imputed series and the corresponding weight series. Both are written into the cache so that chapter 07 (forecaster training) can read them without re-running the annotation.

```
df_filled.to_parquet(shared / "load_de_2024_marked.parquet")
weights.to_frame("weight").to_parquet(shared / "load_de_2024_weights.parquet"
print("Annotated series + weight series persisted.")
```

```
Annotated series + weight series persisted.
```

Exercises (with Claude)

1. *Annotate vs. interpolate on the same spike.* Ask Claude to treat the spike injected here with `LinearlyInterpolateTS(on_missing="ffill_bfill")` instead and compare the resulting histogram of the `load` column with the annotated result. Which strategy shifts the mean less? Which one stays structurally closer to Art. 10 of the AI Act?
2. *Sensitivity of `window_size`.* Have Claude produce a table for `window_size in {24, 48, 72, 168}` showing the share of rows with weight 0. At which window size does the danger zone become usefully large in practice, and when is it too large?
3. *Test the invariant.* Ask Claude for a `pytest` test that, after `mark_outliers` + `get_missing_weights`, checks *two* properties simultaneously: (a) the series contains no more `NaN` and (b) every originally annotated row has weight 0 and at least one of its `window_size` follow-up rows does as well. Why is this a *contractual property* and not an implementation detail?

4 Handle data gaps explicitly

i Learning objective

You understand why a safety-critical library *by default* raises an exception on missing values instead of silently interpolating, and you can correctly apply the three modes `raise`, `ffill_bfill`, `passthrough` of the class `LinearlyInterpolateTS`.

! Preferred strategy: annotation

In safety-critical pipelines the **annotation** strategy from Chapter 3 (`mark_outliers` + `get_missing_weights` + `WeightFunction`) is the preferred one, because it does not replace gaps with invented values — broken rows are consistently excluded from the loss with weight 0. The linear interpolation shown here remains relevant for three situations: (1) migration from legacy notebooks, (2) as a follow-up step after a deliberately *non*-imputing imputer, and (3) when the downstream processing demands a NaN-free series without an explicit weight series.

4.1 Why are data gaps an AI Act topic?

Real sensor data contain gaps. A weather station fails for an hour, an electricity meter is rebooted, a data export drops an empty field. In a pure research notebook this is harmless. In a high-risk application under Annex III of the AI Act — such

as the electric load forecast that a transmission system operator uses for dispatch — an *unnoticed* gap is dangerous. It produces a forecast that is returned with the same apparent confidence as a forecast without a gap; the auditor will never notice afterwards.

Code rule **CR-3** (fail-safe processing) therefore requires: invalid inputs *explicitly* raise an exception instead of being silently imputed. The AI Act underpins this discipline in Art. 10(3) (representativeness, error-freedom, completeness) and Art. 15 (accuracy, robustness, cybersecurity).

`pandas`, `scikit-learn`, `skforecast`, `Darts`, `statsmodels` — most tools in the Python ecosystem follow the *"silently keep computing"* pattern. `spotforecast2-safe` inverts this default.

4.2 The three modes of `LinearlyInterpolateTS`

Since the major version with the contract switch `on_missing`, `LinearlyInterpolateTS` makes the treatment of remaining NaN values explicit. The class interpolates *inside* the series linearly (`limit_area="inside"`), so middle gaps are cleanly closed, while endpoint gaps remain and are handled further according to the mode:

Mode	Behaviour	Who uses this?
`"raise"` (default)	Raises `ValueError` with a preview of the first gaps as soon as any NaN values remain after interpolation.	Safety-critical pipeline; declares CR-3 actively.
`"ffill_bfill"`	Fills the remaining endpoint gaps by forward/backward fill.	Migration from legacy notebooks; internally also part of the forecaster's training contract.

Mode	Behaviour	Who uses this?
`"passthrough"`	Passes NaN through unchanged; the next step takes responsibility.	When a downstream imputer (e.g. `WeightedAverageImputation`) handles the gaps deliberately.

4.3 Code in action: the default is `raise`

```
import numpy as np
import pandas as pd
from pathlib import Path

shared = Path("_cache/shared")
y = pd.read_parquet(shared / "load_de_2024.parquet")["load"]
print(f"{len(y):,} hours loaded, {int(y.isna().sum())} NaN.")
```

```
8,783 hours loaded, 0 NaN.
```

We construct an artificial gap in the middle of the series to see the mechanism at work:

```
y_with_gap = y.copy()
y_with_gap.iloc[120:123] = np.nan        # three NaNs in a row

print(f"Artificial NaNs: {int(y_with_gap.isna().sum())}")
print("Example:")
print(y_with_gap.iloc[118:126].to_string())
```

```
Artificial NaNs: 3
Example:
Time (UTC)
2024-01-05 22:00:00+00:00    50415.8475
2024-01-05 23:00:00+00:00    47558.1450
2024-01-06 00:00:00+00:00           NaN
2024-01-06 01:00:00+00:00           NaN
2024-01-06 02:00:00+00:00           NaN
2024-01-06 03:00:00+00:00    44314.0025
2024-01-06 04:00:00+00:00    44637.6075
2024-01-06 05:00:00+00:00    45244.7525
```

```python
from spotforecast2_safe import LinearlyInterpolateTS

# `on_missing` is not specified -> default 'raise'
y_clean = LinearlyInterpolateTS().fit_transform(y_with_gap)

print(f"NaN after interpolation:  {int(y_clean.isna().sum())}")
print(y_clean.iloc[118:126].to_string())
```

```
NaN after interpolation:  0
Time (UTC)
2024-01-05 22:00:00+00:00    50415.847500
2024-01-05 23:00:00+00:00    47558.145000
2024-01-06 00:00:00+00:00    46747.109375
2024-01-06 01:00:00+00:00    45936.073750
2024-01-06 02:00:00+00:00    45125.038125
2024-01-06 03:00:00+00:00    44314.002500
2024-01-06 04:00:00+00:00    44637.607500
2024-01-06 05:00:00+00:00    45244.752500
```

The middle gap was cleanly closed because values on both sides were available.

The default mode `raise` only stops *if* NaN values remain after the interpolation — typically at the start or end of the series.

4.4 Endpoint gaps: here CR-3 shows its effect

```
y_edge = y.copy()
y_edge.iloc[0:2] = np.nan        # the first two hours are missing

try:
    LinearlyInterpolateTS(on_missing="raise").fit_transform(y_edge)
except ValueError as exc:
    print("Expected fail-safe abort:")
    print(str(exc))
```

```
Expected fail-safe abort:
2 missing value(s) remain after linear interpolation. First gaps: [2024-
01-01 00:00:00+00:00, 2024-01-01 01:00:00+00:00]. Pass on_missing='ffill_bfill' to opt into
```

The error message contains a preview of the first gaps — that is all a dispatcher or auditor needs to locate the gap precisely. Exactly this visibility is what Art. 12 of the AI Act (record-keeping) demands: nothing must fall under the table.

4.5 `ffill_bfill` as a deliberate opt-in

If you want to tolerate endpoint gaps deliberately (e.g. at the start of a migration), you must declare this explicitly:

```
y_filled = LinearlyInterpolateTS(on_missing="ffill_bfill").fit_transform(y_ed
print(f"NaN after 'ffill_bfill':   {int(y_filled.isna().sum())}")
print("First five hours:", y_filled.iloc[:5].round(2).tolist())
```

```
NaN after 'ffill_bfill':   0
First five hours: [37123.23, 37123.23, 37123.23, 36753.52, 37051.4]
```

Internally `ForecasterRecursiveModel` uses exactly this configuration in eight places — not out of carelessness, but to satisfy the training invariant "no NaN in the lag window at training time" even when the caller passes the series without pre-processing. The contract is made visible at the call site; that is the difference from silent imputation.

4.6 `passthrough`: responsibility passed to the caller

```
y_pt = LinearlyInterpolateTS(on_missing="passthrough").fit_transform(y_edge)
print(f"NaN after 'passthrough':  {int(y_pt.isna().sum())}")
```

```
NaN after 'passthrough':  2
```

Useful, for example, when a downstream `WeightedAverageImputation` step fills the gaps deliberately. The `WARNING` emitted by that imputer is shown so that the caller sees in the audit log that imputation has happened.

4.7 Visualisation: before/after picture of gap handling

```
import matplotlib.pyplot as plt
import matplotlib.dates as mdates

window = slice(115, 130)
fig, (ax_top, ax_bot) = plt.subplots(2, 1, figsize=(8.0, 4.5), sharex=True)

ax_top.plot(y_with_gap.iloc[window].index, y_with_gap.iloc[window].values / 1000,
            "o-", color="#B33A3A", linewidth=1.0, markersize=4)
ax_top.set_ylabel("Load [GW]")
ax_top.set_title("before interpolation: three NaN")
ax_top.grid(True, color="#E5E5E5", linewidth=0.5)

ax_bot.plot(y_clean.iloc[window].index, y_clean.iloc[window].values / 1000,
            "o-", color="#2E7D55", linewidth=1.0, markersize=4)
ax_bot.set_ylabel("Load [GW]")
ax_bot.set_title("after `LinearlyInterpolateTS()` (default `on_missing='raise'`)")
ax_bot.grid(True, color="#E5E5E5", linewidth=0.5)
ax_bot.xaxis.set_major_locator(mdates.HourLocator(byhour=[0, 12]))
ax_bot.xaxis.set_major_formatter(mdates.DateFormatter("%-d.%-m %H:%M"))

for ax in (ax_top, ax_bot):
    for s in ("top", "right"):
        ax.spines[s].set_visible(False)

fig.tight_layout()
plt.show()
```

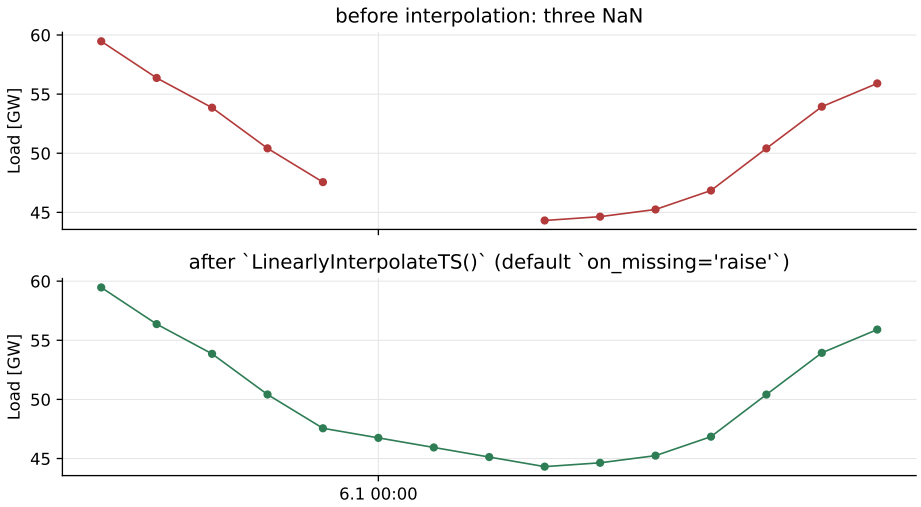


Figure 4.1: Demonstration of the `LinearlyInterpolateTS` treatment. Top: an artificially injected gap (three consecutive NaN values) in January 2024. Bottom: the same series after the default call with `on_missing='raise'` — the three NaNs are filled in by linear interpolation of the neighbouring values; because no endpoint gaps remain, no error is raised.

4.8 Which strategy when? — a comparison at a glance

The three `LinearlyInterpolateTS` modes and the annotation pipeline from Chapter 3 are not mutually exclusive — they target different points in the lifecycle of the data. Before mapping the application situations, a direct comparison of the two strategies' *structural properties* is worthwhile:

Property	`LinearlyInterpolateTS`	`mark_outliers` + `get_missing_weights` (Chapter 3)
Series length after pre-processing	unchanged	unchanged
NaN guarantee after the step	only in `ffill_bfill` mode / after interpolation	yes, guaranteed after `get_missing_weights`
Training target contains invented values	yes (line between neighbours)	no (real values; broken values are ffill placeholders with weight 0)
Auditability of the action	indirect (`raise` error message)	direct (`verbose=True` lists indices; `weights` is serialisable)
Model bias due to imputation	possible	structurally excluded (weight 0)
Recommendation for high-risk applications	only as a migration path or with a downstream imputer	**standard strategy**

The following table translates these structural differences into concrete application situations:

Situation	Recommended strategy
Production high-risk pipeline (Annex III)	**Annotation** (Chapter 3); interpolation only as a downstream special step
Middle gaps of individual hours, no systematic outlier search needed	`LinearlyInterpolateTS(on_missing="raise")` as a CR-3 switch
Migration from legacy notebooks without a `weight_func` path	`LinearlyInterpolateTS(on_missing="ffill_bfill")` as a bridge
Own downstream imputer	`LinearlyInterpolateTS(on_missing="passthrough")`

Situation	Recommended strategy
Audit requires an explicit, picklable weight series	**Annotation** (Chapter 3); weights are part of the model artefact

If in doubt, start with the annotation from Chapter 3 and only fall back to `LinearlyInterpolateTS` when there is a concrete argument against the weight series.

4.9 Persisting for follow-up chapters

We take the clean series as the starting point for all the following steps:

```
y_clean.to_frame("load").to_parquet(shared / "load_de_2024_clean.parquet")
print("Cleaned series persisted.")
```

```
Cleaned series persisted.
```

Exercises (with Claude)

1. *Three modes on the same example.* Ask Claude to write a small comparison table showing, for the three modes `raise`, `ffill_bfill`, `passthrough` on a series with *both* a middle and an endpoint gap, what the result looks like in each case. Which mode is directly compatible with Art. 10 of the AI Act?
2. *Comparison with pandas.* Have Claude compare `pandas.Series.interpolate()` directly with `LinearlyInterpolateTS(on_missing='raise')`. On an example with gaps at the edge: what happens in each variant? Which default is more suitable for a safety-critical pipeline?
3. *Test the contract.* Ask Claude how you would write a *test* that, in

a CI pipeline, ensures that `LinearlyInterpolateTS()` (without an explicit argument) preserves the default `on_missing='raise'`. Argue why this property must be tested as a *contract* — and not just stated in the documentation.

5 Separating training and evaluation range

> **Learning objective**
>
> You understand why a random split is wrong for time series, and you can perform a clean chronological train/test split.

5.1 Why not split randomly?

In classical supervised learning (e.g. image classification) the dataset is split randomly into training and test ranges. For *time series* this is a mistake: every value knows its past, and a randomly drawn training set would contain *future* hours from which the model implicitly learns to predict *past* hours. The accuracy measurement would be self-deception; it would collapse dramatically in live operation.

We therefore split *chronologically*: a contiguous initial segment is used for training, a contiguous final segment for evaluation. This split is the simplest variant of a representativeness check demanded by Art. 10(2)(f) and (g) of the AI Act. Code rule **CR-2** (determinism) additionally requires the cut point to be fixed as a constant — not random, but reproducible.

5.2 Code in action

We take the series cleaned in Chapter 4 and define January to October 2024 as the training range and November as the evaluation range.

```
import pandas as pd
from pathlib import Path

shared = Path("_cache/shared")
y_clean = pd.read_parquet(shared / "load_de_2024_clean.parquet")["load"]

TRAIN_END = pd.Timestamp("2024-10-31 23:00", tz="UTC")
EVAL_END  = pd.Timestamp("2024-11-30 23:00", tz="UTC")

y_train = y_clean.loc[:TRAIN_END]
y_eval  = y_clean.loc[TRAIN_END + pd.Timedelta(hours=1) : EVAL_END]

print(f"Training:    {y_train.index[0]}  to  {y_train.index[-1]}  "
      f"({len(y_train):,} h)")
print(f"Evaluation:  {y_eval.index[0]}  to  {y_eval.index[-1]}  "
      f"({len(y_eval):,} h)")
```

```
Training:    2024-01-01 00:00:00+00:00  to  2024-10-31 23:00:00+00:00  (7,320
Evaluation:  2024-11-01 00:00:00+00:00  to  2024-11-30 23:00:00+00:00  (720 h
```

We persist the cut points so that all following chapters use exactly the same split:

```
y_train.to_frame("load").to_parquet(shared / "y_train.parquet")
y_eval.to_frame("load").to_parquet(shared / "y_eval.parquet")
(shared / "split_meta.json").write_text(
    f'{{"train_end":"{TRAIN_END.isoformat()}",'
    f'"eval_end":"{EVAL_END.isoformat()}",'
```

```
    f'"freq":"h","tz":"UTC"}}'
)
```

102

5.3 Visualisation: the split on the time axis

```
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import matplotlib.patches as mpatches

fig, ax = plt.subplots(figsize=(8.5, 3.0))
ax.plot(y_train.index, y_train.values / 1000,
        color="#1F4E79", linewidth=0.4, label="Training")
ax.plot(y_eval.index, y_eval.values / 1000,
        color="#E07B00", linewidth=0.6, label="Evaluation")

reserve_idx = y_clean.loc[EVAL_END + pd.Timedelta(hours=1):].index
ax.plot(reserve_idx, y_clean.loc[reserve_idx].values / 1000,
        color="#999999", linewidth=0.4, label="Reserve")

ax.axvline(TRAIN_END, color="#555555", linestyle="--", linewidth=0.7)
ax.axvline(EVAL_END,  color="#555555", linestyle="--", linewidth=0.7)
ax.set_ylabel("Load [GW]")
ax.set_xlabel("Time")
ax.grid(True, color="#E5E5E5", linewidth=0.5)
ax.xaxis.set_major_locator(mdates.MonthLocator())
ax.xaxis.set_major_formatter(mdates.DateFormatter("%b %Y"))
ax.legend(loc="upper left", frameon=False)
for s in ("top", "right"):
```

```
    ax.spines[s].set_visible(False)
fig.tight_layout()
plt.show()
```

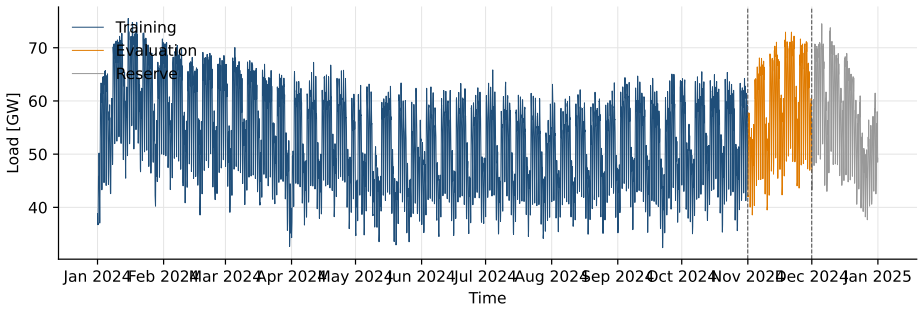


Figure 5.1: Chronological train/test split of the cleaned 2024 load series. Training range (blue): January to October. Evaluation range (orange): November. December (grey) is held back as a reserve for future follow-up experiments. The dashed line marks the cut point.

Exercises (with Claude)

1. *Why chronologically?* Have Claude show on a small concrete example (e.g. a 10-value series) what goes wrong if you split randomly instead of chronologically. Which metrics from Art. 15 of the AI Act are thereby falsified?
2. *Cut point as contract.* Discuss with Claude why `TRAIN_END` should live as a *constant* in a separate file (or at the very top of the notebook) rather than being scattered through the code. Which code rule does this practice follow?
3. *Robustness.* Ask Claude to show a variant in which the split is not performed once but multiple times at different cut-off dates (e.g. monthly rolling). How does the informativeness of the accuracy measurement

change?

6 Calendar features and real weather data

> **Learning objective**
>
> You can set up an `ExogBuilder` with cyclic RBF features, read real weather data from Open-Meteo via the `WeatherService` class, and concatenate both into a single feature matrix that is later passed to the forecaster as *exogenous input*.

6.1 Why does the model need more than the load itself?

The load depends not only on its own immediate history but also on *context information*: time of day, day of week, holiday, weather. If the model only sees "Wednesday 18:00" and "Wednesday 02:00" as hours 18 and 2, it has no way to recognise that these hours stand in a cyclic relationship to each other and repeat periodically. We encode such cyclic features with *radial basis functions* (RBF) so that an hour is always similar to its neighbouring hours — including the midnight transition.

Weather is the second pillar. In cold weather the load rises (heating, lighting); on sunny summer days it falls. The model therefore sees per hour a tuple of hourly temperature, cloud cover, precipitation and wind speed. The Open-Meteo API

provides these values both retrospectively (archive) and prospectively (forecast) without an API key.

6.2 AI Act link

Art. 10(1) of the AI Act requires that "training, validation and testing data sets […] meet the quality criteria laid down in paragraphs 2 to 4". One of the quality requirements is *representativeness*: the model must have seen all relevant seasonal, calendar-related and weather-dependent variations. By training the load jointly with calendar and weather features, we practically deliver the "examination of statistically probable characteristics" demanded by Art. 10(2)(f).

`WeatherService` enforces code rule **CR-3**: any gap in the weather data that remains after resampling is reported as `ValueError` — not silently filled.

6.3 Code in action: cyclic calendar features

```
import pandas as pd
from pathlib import Path
from spotforecast2_safe import Period, ExogBuilder

shared = Path("_cache/shared")
y_train = pd.read_parquet(shared / "y_train.parquet")["load"]
y_eval  = pd.read_parquet(shared / "y_eval.parquet")["load"]

periods = [
    Period(name="hour",      n_periods=12, column="hour",      input_range=(0
    Period(name="dayofweek", n_periods=4,  column="dayofweek", input_range=(0
    Period(name="month",     n_periods=12, column="month",     input_range=(1
]
```

```
builder = ExogBuilder(periods=periods, country_code="DE")
exog_cal = builder.build(y_train.index.min(), y_eval.index.max())

print(f"Calendar features: {exog_cal.shape[0]:,} hours, {exog_cal.shape[1]} columns")
print(f"Columns:           {list(exog_cal.columns)[:6]} ...")
```

```
Calendar features: 8,040 hours, 30 columns
Columns:           ['hour_0', 'hour_1', 'hour_2', 'hour_3', 'hour_4', 'hour_5'] ...
```

```
import matplotlib.pyplot as plt

day = exog_cal.loc["2024-03-04 00:00":"2024-03-04 23:00"]
fig, ax = plt.subplots(figsize=(8.5, 3.0))
hour_cols = [c for c in day.columns if c.startswith("hour_")]
for c in hour_cols:
    ax.plot(range(24), day[c].values, linewidth=1.0, alpha=0.7)
ax.set_xticks(range(0, 24, 2))
ax.set_xlabel("Hour of day")
ax.set_ylabel("RBF activation")
ax.set_title("12 cyclic basis functions for the hour of day")
ax.grid(True, color="#E5E5E5", linewidth=0.5)
for s in ("top", "right"):
    ax.spines[s].set_visible(False)
fig.tight_layout()
plt.show()
```

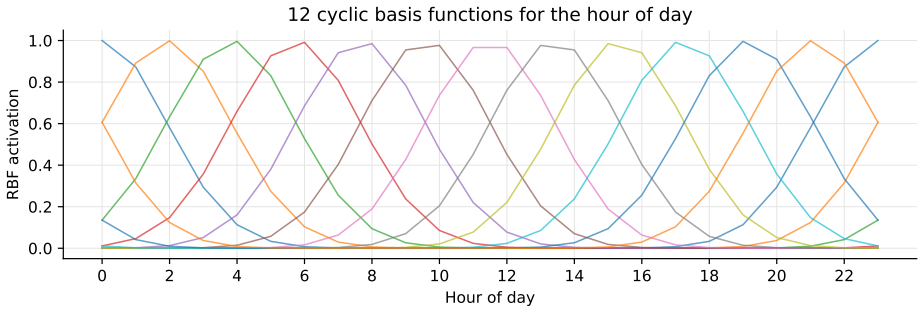


Figure 6.1: RBF encoding of the hour of day. Each of the 12 basis functions is centred on a different hour. At every position of the day the activations of all functions sum to one. Midnight is thus 'near' to 23:00 and to 01:00, and the model does not have to learn the jump between hour 23 and hour 0 artificially.

6.4 Code in action: weather

For a central German location — Frankfurt am Main, a typical load centre — we fetch the hourly values. The `WeatherService` class handles the cache transparently: the first request goes to the API, subsequent identical requests are served from the parquet file.

```
from spotforecast2_safe.weather import WeatherService

cache = Path("_cache") / "weather_de_2024.parquet"
ws = WeatherService(
    latitude=50.110924,
    longitude=8.682127,
    cache_path=cache,
    use_forecast=False,         # historical data only - we stay in 2024
)
```

```
weather = ws.get_dataframe(
    start=y_train.index.min(),
    end=y_eval.index.max(),
    freq="h",
    fill_missing=False,         # CR-3: fail-safe, no silent imputation
)
print(f"Weather frame: {weather.shape[0]:,} hours, {weather.shape[1]} columns")
print(f"First columns: {list(weather.columns)[:5]} ...")
print(f"NaN values:    {int(weather.isna().sum().sum())} (should be 0; otherwise CR-3 would
```

```
Weather frame: 8,040 hours, 15 columns
First columns: ['temperature_2m', 'relative_humidity_2m', 'precipitation', 'rain', 'snowfall
NaN values:    0 (should be 0; otherwise CR-3 would have aborted)
```

6.5 Looking at weather and load together

```
import matplotlib.dates as mdates

window = slice("2024-03-11", "2024-03-17")
fig, axes = plt.subplots(2, 1, figsize=(8.5, 4.5), sharex=True)

axes[0].plot(y_train.loc[window].index, y_train.loc[window].values / 1000,
             color="#1F4E79", linewidth=1.0)
axes[0].set_ylabel("Load [GW]")
axes[0].grid(True, color="#E5E5E5", linewidth=0.5)

axes[1].plot(weather.loc[window].index, weather.loc[window]["temperature_2m"].values,
             color="#E07B00", linewidth=1.0)
axes[1].set_ylabel("Temperature [degC]")
axes[1].set_xlabel("Time")
```

```
axes[1].grid(True, color="#E5E5E5", linewidth=0.5)
axes[1].xaxis.set_major_formatter(mdates.DateFormatter("%a %-d.%-m"))

for ax in axes:
    for s in ("top", "right"):
        ax.spines[s].set_visible(False)
fig.tight_layout()
plt.show()
```

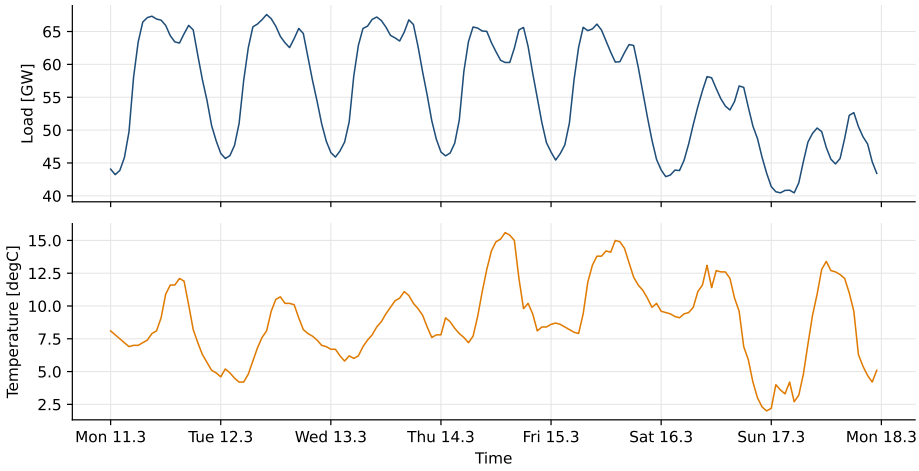


Figure 6.2: Weekly overlay in mid-March 2024: hourly load (top) and at the same time the 2-metre air temperature in Frankfurt am Main (bottom). On cold nights the load is visibly higher; on warm days the midday peak is dampened.

6.6 Concatenating calendar and weather into a single matrix

The load and the exogenous columns must have exactly the same index, otherwise the forecaster later fails. This is a direct consequence of **CR-3**: missing index rows would silently become NaN inside `pandas` — and that is exactly what we want to avoid.

```
exog = exog_cal.join(weather, how="inner")
exog = exog.loc[y_train.index.min(): y_eval.index.max()]

print(f"Joined feature matrix: {exog.shape[0]:,} hours, {exog.shape[1]} columns")
print(f"NaN values:            {int(exog.isna().sum().sum())}")

exog.to_parquet(shared / "exog.parquet")
print("Saved `exog.parquet` for follow-up chapters.")
```

```
Joined feature matrix: 8,040 hours, 45 columns
NaN values:            0
Saved `exog.parquet` for follow-up chapters.
```

Exercises (with Claude)

1. *Why RBF?* Ask Claude to show an alternative to the RBF encoding (e.g. `pd.get_dummies`) and to demonstrate on a plot of the midnight transition why this alternative violates cyclic continuity.
2. *Weather fail-safe.* Have Claude display the source of `WeatherService.get_dataframe`, specifically the spot where the `ValueError` is raised when `fill_missing=False`. What is the exact wording of the error message, and which code rule does this behaviour follow?
3. *A second city.* Together with Claude extend the pipeline so that *two*

weather locations (e.g. Frankfurt and Berlin) are queried and both included in the `exog` matrix with distinct column names. Which question could the improved model answer better?

7 Instantiate and train the forecaster

> **Learning objective**
>
> You can instantiate the class `ForecasterRecursiveLGBM`, equip it with a deterministically configured LightGBM estimator, and train it on real load and weather data. You understand why code rules **CR-2** (determinism) and **CR-4** (minimal CVE attack surface) are crucial in this step.

7.1 What happens during training?

`ForecasterRecursiveLGBM` is a wrapper around the gradient-boosting estimator *LightGBM*. During training the model looks back, for every hour t, at the last `lags` hours (time steps) and learns a function that predicts the value at hour t from these past values plus the exogenous calendar and weather features. *Recursive* means: in later prediction the output for hour t is used in turn as input for hour $t+1$.

`lags=168` (= 7 x 24) means: the model sees the last full week as context for every hour — decisive for correctly representing the weekly rhythm (weekday vs. weekend).

7.2 AI Act link

Art. 9 of the AI Act demands a risk-management system over the entire lifecycle; Art. 15 demands an appropriate level of accuracy. Both obligations presuppose

that the training run is *reproducible*: the supervisory authority must be able to reconstruct the model from its documented hyperparameters and seed. This is exactly the scope of **CR-2**.

CR-4 forbids a short negative list of dependencies in the package (`plotly`, `matplotlib`, `spotoptim`, `optuna`, `torch`, `tensorflow`). The estimator LightGBM is *not* on this list — deliberately: it is small, actively maintained, implemented in C++ and brings no additional attack surface. A migration to PyTorch or TensorFlow would violate the deny-list.

7.3 Code in action

```
import pandas as pd
from pathlib import Path

shared = Path("_cache/shared")
y_train = pd.read_parquet(shared / "y_train.parquet")["load"]
exog    = pd.read_parquet(shared / "exog.parquet")

print(f"Training series: {len(y_train):,} h from {y_train.index[0]} to {y_tra
print(f"Exog matrix:     {exog.shape}")
```

```
Training series: 7,320 h from 2024-01-01 00:00:00+00:00 to 2024-
10-31 23:00:00+00:00
Exog matrix:     (8040, 45)
```

```
from spotforecast2_safe import Period, ForecasterRecursiveLGBM
from lightgbm import LGBMRegressor

periods = [
```

```
    Period(name="hour",      n_periods=12, column="hour",      input_range=(0, 23)),
    Period(name="dayofweek", n_periods=4,  column="dayofweek", input_range=(0,  6)),
    Period(name="month",     n_periods=12, column="month",     input_range=(1, 12)),
]

model = ForecasterRecursiveLGBM(
    iteration=0,
    lags=168,
    periods=periods,
    country_code="DE",
    random_state=2026,
)

# We replace the default estimator with a richer variant so that the
# model can reliably learn the annual seasonality. All non-deterministic
# options are deliberately switched off:
model.forecaster.estimator = LGBMRegressor(
    n_estimators=400,
    learning_rate=0.05,
    num_leaves=63,
    min_child_samples=20,
    n_jobs=-1,
    verbose=-1,
    deterministic=True,
    force_col_wise=True,
    random_state=model.random_state,
)
print(f"Estimator:        {type(model.forecaster.estimator).__name__}")
print(f"Number of trees:  {model.forecaster.estimator.n_estimators}")
print(f"Learning rate:    {model.forecaster.estimator.learning_rate}")
print(f"Lags:             {model.forecaster.lags.size}")
print(f"Seed:             {model.random_state}")
```

```
Estimator:         LGBMRegressor
Number of trees:   400
Learning rate:     0.05
Lags:              168
Seed:              2026
```

> **CR-2 in practice**
>
> The three LightGBM options `deterministic=True`, `force_col_wise=True` and a fixed `random_state` together are the *bit-exact* reproducibility guarantee. Without these three options LightGBM produces slightly different trees across runs, because the order of histogram computation becomes non-deterministic — and that violates code rule CR-2.

7.4 Run the training

```python
import time

t0 = time.perf_counter()
model.fit(y=y_train, exog=exog.loc[y_train.index])
elapsed = time.perf_counter() - t0
print(f"Training completed in {elapsed:.1f} seconds.")
print(f"Training index:  {model.forecaster.training_range_[0]}  ...  "
      f"{model.forecaster.training_range_[1]}")
```

```
/Users/bartz/workspace/Lehre.d/ddmo-sose-26/.venv/lib/python3.14/site-
packages/spotforecast2_safe/forecaster/utils.py:795: UserWarning: `y` has a l
  warnings.warn(
```

```
Training completed in 6.7 seconds.
Training index:  2024-01-01 00:00:00+00:00  ...  2024-10-
31 23:00:00+00:00
```

7.5 Which features shaped the model most strongly?

LightGBM counts for every tree how often each feature was used as a split candidate. This *feature importance* is a first, traceable answer to the question: *what does the model base its forecast on?* This answerability is a core concern of Art. 13 of the AI Act (transparency).

```
import matplotlib.pyplot as plt

feat_names = model.forecaster.estimator.feature_name_
importances = model.forecaster.estimator.feature_importances_

ranking = sorted(zip(feat_names, importances), key=lambda kv: kv[1], reverse=True)[:15]

fig, ax = plt.subplots(figsize=(8.0, 4.5))
labels = [n for n, _ in ranking][::-1]
values = [v for _, v in ranking][::-1]
ax.barh(labels, values, color="#1F4E79")
ax.set_xlabel("split count (feature importance)")
ax.grid(True, axis="x", color="#E5E5E5", linewidth=0.5)
for s in ("top", "right"):
    ax.spines[s].set_visible(False)
fig.tight_layout()
plt.show()
```

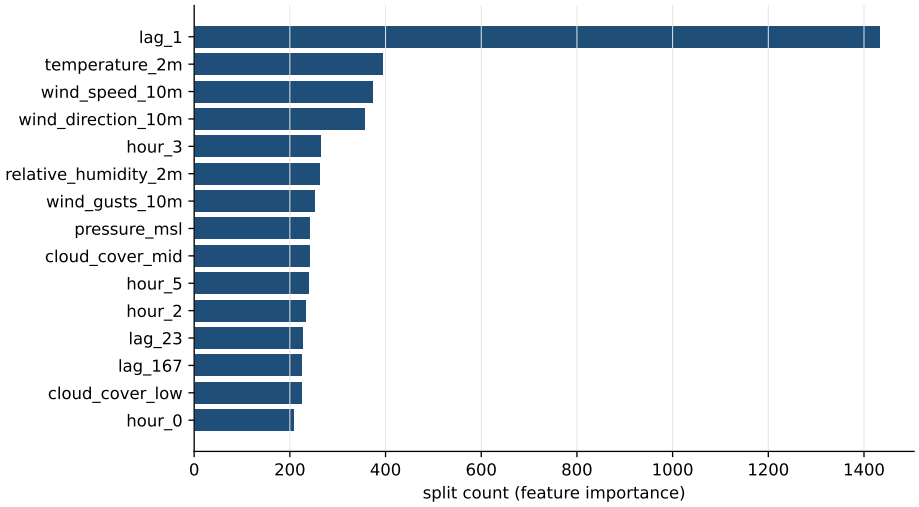


Figure 7.1: Top-15 feature importances of the trained LightGBM model, measured as 'split' frequency over all 400 trees. Lag features (`lag_*`) carry the bulk of the information; weather (temperature) and day-of-week encodings supplement them.

7.6 Persist model for follow-up chapters

We store the trained model; chapter 08 reads it back and produces the 24-h forecast.

```
import pickle

model_path = Path("_cache/forecaster.pkl")
with model_path.open("wb") as fh:
    pickle.dump(model, fh)
print(f"Model persisted: {model_path}")
```

```
Model persisted: _cache/forecaster.pkl
```

Exercises (with Claude)

1. *CR-4 deny-list.* Ask Claude to display the deny-list of forbidden dependencies in `spotforecast2-safe`'s `pyproject.toml` and to explain for every entry (`plotly`, `matplotlib`, `spotoptim`, `optuna`, `torch`, `tensorflow`) *why* it is not admissible in a safety-critical library.
2. *Check determinism.* Have Claude write a short function that trains the model twice in a row on the same data and compares the `feature_importances_` of the two runs bit-by-bit at the end. What should come out under correct CR-2 conformance?
3. *Hyperparameter awareness.* Discuss with Claude how `lags=24` should qualitatively affect the forecast compared to `lags=168`. Which weekly structure can the model no longer learn with only 24 lag hours?

8 Produce a 24-hour forecast

> **Learning objective**
>
> You understand the recursive prediction logic and can produce a daily forecast of the trained forecaster with two lines of code. You see how that forecast compares to the actual load and to a naive baseline.

8.1 Recursive multi-step forecasting

The model was trained to predict *one* hour at a time, given the last `lags=168` hours. For a 24-hour daily forecast the forecaster shifts the 168-hour lag window forward hour by hour:

$$\hat{y}_{t+1} = f(y_t, y_{t-1}, \dots, y_{t-167}, \mathbf{x}_{t+1})$$

$$\hat{y}_{t+2} = f(\hat{y}_{t+1}, y_t, \dots, y_{t-166}, \mathbf{x}_{t+2})$$

At every further step an already predicted value flows into the lag window. This recursion explains why forecast accuracy typically deteriorates with the horizon: model errors from earlier hours propagate to later ones.

8.2 AI Act link

Art. 13 of the AI Act (*transparency and provision of information to deployers*) demands that a user can understand how the forecast comes about. By writing the recursion equation out explicitly and making the lag window visible, we document the mechanism in a way that can be copied straight into a `MODEL_CARD.md`.

8.3 Code in action

```
import pandas as pd
import pickle
from pathlib import Path

shared = Path("_cache/shared")
y_train = pd.read_parquet(shared / "y_train.parquet")["load"]
y_eval  = pd.read_parquet(shared / "y_eval.parquet")["load"]
exog    = pd.read_parquet(shared / "exog.parquet")

with Path("_cache/forecaster.pkl").open("rb") as fh:
    model = pickle.load(fh)
```

We hand the forecaster the last lag window *explicitly*. For the model with `lags=168` those are the last 168 hours of the training range. That way you see immediately which past information enters the first forecast hour.

```
y_eval_24h = y_eval.iloc[:24]
last_window = y_train.iloc[-168:].copy()
# A parquet round-trip loses the `freq` attribute of the DatetimeIndex.
# The forecaster needs it to construct the prediction index:
last_window.index = pd.DatetimeIndex(last_window.index, freq="h")
```

```
y_pred = model.forecaster.predict(
    steps=24,
    last_window=last_window,
    exog=exog.loc[y_eval_24h.index],
)
y_pred.name = "forecast"
y_pred.index = y_eval_24h.index

print(f"Forecast produced: {len(y_pred)} hours, "
      f"{y_pred.index[0]} ... {y_pred.index[-1]}")
print(f"Last lag hour:    {last_window.index[-1]}")
print("First three forecast hours:")
print(y_pred.head(3).round(1).to_string())
```

```
Forecast produced: 24 hours, 2024-11-01 00:00:00+00:00 ... 2024-
11-01 23:00:00+00:00
Last lag hour:    2024-10-31 23:00:00+00:00
First three forecast hours:
Time (UTC)
2024-11-01 00:00:00+00:00    42052.3
2024-11-01 01:00:00+00:00    41891.0
2024-11-01 02:00:00+00:00    42106.2
```

8.4 Visualisation: forecast, reality, baseline

A forecast deserves the name only when it is *better* than a trivial baseline. We take *weekly persistence* as a baseline: the load of the same hour seven days earlier. If our model does not clearly beat this baseline, the whole effort was in vain.

```
import matplotlib.pyplot as plt
import matplotlib.dates as mdates

train_tail = y_train.iloc[-24:]
y_clean = pd.read_parquet(shared / "load_de_2024_clean.parquet")["load"]
persistence = y_clean.shift(168).loc[y_eval_24h.index]

fig, ax = plt.subplots(figsize=(8.5, 3.5))
ax.plot(train_tail.index, train_tail.values / 1000,
        color="#1F4E79", linewidth=1.0, label="Training (tail)")
ax.plot(y_eval_24h.index, y_eval_24h.values / 1000,
        color="#1F4E79", linewidth=1.0, linestyle=":")
ax.plot(y_pred.index, y_pred.values / 1000,
        color="#2E7D55", linewidth=1.4, label="Model forecast")
ax.plot(y_eval_24h.index, y_eval_24h.values / 1000,
        color="#E07B00", linewidth=1.0, label="observed (evaluation)")
ax.plot(persistence.index, persistence.values / 1000,
        color="#8E63B5", linewidth=1.0, linestyle="--", label="Weekly persist

split = train_tail.index[-1] + pd.Timedelta(hours=1)
ax.axvline(split, color="#555555", linestyle="--", linewidth=0.7)
ax.text(split, ax.get_ylim()[1], "  End of training",
        ha="left", va="top", fontsize=8, color="#555555")

ax.set_ylabel("Load [GW]")
ax.set_xlabel("Time")
ax.grid(True, color="#E5E5E5", linewidth=0.5)
ax.xaxis.set_major_locator(mdates.HourLocator(byhour=[0, 12]))
ax.xaxis.set_major_formatter(mdates.DateFormatter("%-d.%-m %H:%M"))
ax.legend(loc="upper left", frameon=False, fontsize=8)
for s in ("top", "right"):
    ax.spines[s].set_visible(False)
fig.tight_layout()
```

```
plt.show()
```

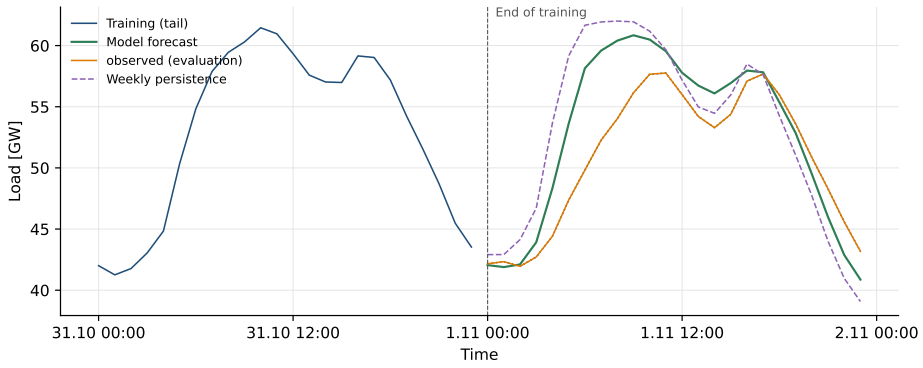


Figure 8.1: Comparison of the 24-hour forecast with the actual load and the weekly-persistence baseline (load seven days earlier, same hour). The window covers the last 24 training hours plus the first 24 evaluation hours. The vertical dashed line marks the end of the training period.

8.5 Persisting for follow-up chapters

```
y_pred.to_frame("forecast").to_parquet(shared / "y_pred_24h.parquet")
```

Exercises (with Claude)

1. *Understand recursion error.* Have Claude rebuild the recursion equation by hand in a small loop (with a trivial model such as `f(y, x) = mean(y[-24:])`) and demonstrate on a 96-hour horizon how model errors amplify over time.
2. *Change the lag window.* Discuss with Claude what happens when you

predict `steps=72` (3 days) instead of `steps=24`. Which hours among the last 24 predictions are probably the least accurate and why?

3. *Transparency for the deployer.* Ask Claude to write an example entry for a `MODEL_CARD.md` that explains *recursive forecasting* to a non-technical reader in three sentences and spells out what consequence this has for the forecast uncertainty at the end of the horizon.

9 Measuring accuracy: MAE, MSE, RMSE, MAPE

> **Learning objective**
>
> You can compute the four standard measures of point-forecast accuracy, explain their geometric meaning, and distinguish *which* measure is appropriate for a given risk question.

9.1 Why four measures instead of one?

A load forecast can go wrong in many ways: many small errors in one direction, a single huge outlier, a consistent upward or downward offset. No single measure summarises all of that. We therefore look at four:

- *Mean Absolute Error* (MAE): the average deviation in load units. Robust, easy to interpret, treats one large and many small errors with equal weight.
- *Mean Squared Error* (MSE): the average squared deviation. Penalises large outliers disproportionately. Has units of "load squared".
- *Root Mean Squared Error* (RMSE): square root of the MSE; again in load units, but with outlier sensitivity.
- *Mean Absolute Percentage Error* (MAPE): the average deviation in per cent. Allows comparisons between different load levels.

9.2 AI Act link

Art. 15 of the AI Act (*accuracy, robustness and cybersecurity*) demands that high-risk AI systems achieve "appropriate levels of accuracy" and declare those in the `MODEL_CARD.md`. What is "appropriate" depends on the use case. For a daily forecast of the electric load, a MAPE below 4% would be considered state of the art.

9.3 Code in action

```
import numpy as np
import pandas as pd
from pathlib import Path
from spotforecast2_safe.manager.demo_metrics import calculate_metrics

shared = Path("_cache/shared")
y_eval = pd.read_parquet(shared / "y_eval.parquet")["load"]
y_pred = pd.read_parquet(shared / "y_pred_24h.parquet")["forecast"]
y_eval_24h = y_eval.iloc[:24]

m = calculate_metrics(actual=y_eval_24h, predicted=y_pred)
mae  = float(m["MAE"])
mse  = float(m["MSE"])
rmse = float(np.sqrt(mse))
mape = float(np.mean(np.abs((y_eval_24h.values - y_pred.values) / y_eval_24h

print(f"MAE  = {mae:7.1f}   MW   (mean absolute error)")
print(f"MSE  = {mse:9.1f}   MW^2 (mean squared error)")
print(f"RMSE = {rmse:7.1f}   MW   (root mean squared error)")
print(f"MAPE = {mape:7.2%}        (mean absolute percentage error)")
```

```
MAE  =  2673.4   MW   (mean absolute error)
MSE  = 12446842.1   MW^2 (mean squared error)
RMSE =  3528.0   MW   (root mean squared error)
MAPE =   5.26%        (mean absolute percentage error)
```

9.4 Visualisation: geometry of the metrics

An evocative view of the measures: the *histogram of residuals* (forecast minus actual value). The MAE is the average distance from zero; the RMSE is the spread weighted by error size. Both can be drawn as vertical lines.

```python
import matplotlib.pyplot as plt

residuals = (y_pred.values - y_eval_24h.values) / 1000  # in GW

fig, ax = plt.subplots(figsize=(8.5, 3.5))
ax.hist(residuals, bins=12, color="#1F4E79", alpha=0.85, edgecolor="white")
ax.axvline(0, color="#000000", linewidth=0.8)
ax.axvline(+mae / 1000, color="#E07B00", linestyle="--", linewidth=1.2, label=f"+MAE = {mae/
ax.axvline(-mae / 1000, color="#E07B00", linestyle="--", linewidth=1.2)
ax.axvline(+rmse / 1000, color="#2E7D55", linestyle=":", linewidth=1.2, label=f"+RMSE = {rms
ax.axvline(-rmse / 1000, color="#2E7D55", linestyle=":", linewidth=1.2)
ax.set_xlabel("Residual (forecast - observed) [GW]")
ax.set_ylabel("Number of hours")
ax.legend(loc="upper right", frameon=False)
ax.grid(True, color="#E5E5E5", linewidth=0.5)
for s in ("top", "right"):
    ax.spines[s].set_visible(False)
fig.tight_layout()
plt.show()
```

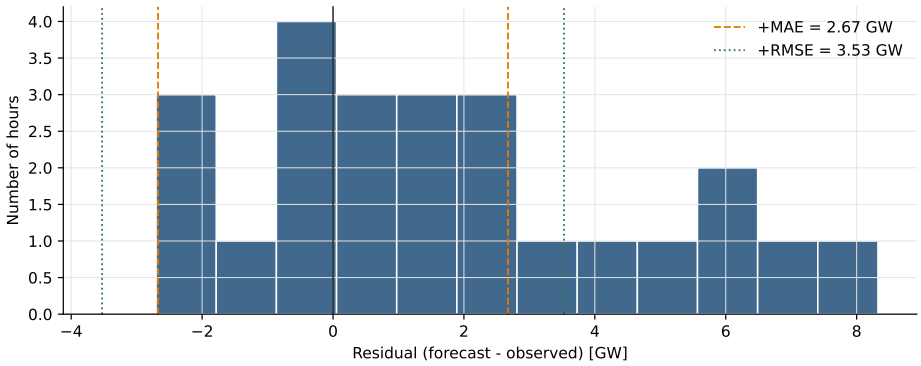


Figure 9.1: Histogram of the 24 residuals of the daily forecast. The vertical lines show MAE and RMSE as alternative spread measures. Here the MAE is the average of the absolute distances to zero; the RMSE is the square root of the average of the squared distances and is influenced more strongly by the outlier at the histogram edge.

9.5 When is which measure appropriate?

Question	Suitable measure
How much reserve energy do I need to plan in bulk?	MAE
How severe are the rare large errors?	RMSE or MSE
How does my system compare to one in another market zone?	MAPE
Which component of my risk capital is sufficient?	RMSE plus a quantile bound

9.6 Comparison with the baseline

Without comparison to a trivial baseline an accuracy number is meaningless. We compute the same measures for the weekly persistence:

```
y_clean = pd.read_parquet(shared / "load_de_2024_clean.parquet")["load"]
persistence = y_clean.shift(168).loc[y_eval_24h.index]

mae_p  = float((y_eval_24h.values - persistence.values).__abs__().mean())
mape_p = float(np.mean(np.abs((y_eval_24h.values - persistence.values) / y_eval_24h.values))

print(f"            Model       Weekly persistence")
print(f"MAE    {mae:8.1f} MW    {mae_p:8.1f} MW")
print(f"MAPE   {mape:8.2%}      {mape_p:8.2%}")
```

```
            Model       Weekly persistence
MAE      2673.4 MW      3995.0 MW
MAPE      5.26%         8.08%
```

Exercises (with Claude)

1. *Choosing measures.* Discuss with Claude a concrete use case ("We need the forecast in order to prepare our reserve gas-fired plants tomorrow at 06:00") and have it argue which of the four measures the transmission system operator should include as a KPI in its `MODEL_CARD.md`.
2. *Own implementation.* Ask Claude to program the four measures by hand with `numpy`, without `calculate_metrics`, and compare them with the library function. What happens when a single actual value is zero (division by zero in MAPE)?
3. *Robustness to outliers.* Have Claude generate a synthetic 24-h vector in which 23 values are very accurate (error $\pm$ 50 MW) and a single

value is far off ($\pm$ 5000 MW). How do MAE and RMSE behave? Which measure shows the “problem” more clearly?

10 Rolling-origin backtest

> **Learning objective**
>
> You understand why a single train/test split does not yield a robust accuracy verdict, and you can set up a rolling-origin backtest with `TimeSeriesFold` that estimates the forecast quality over several weeks.

10.1 Why backtest at all?

In the previous chapter we forecast the first 24 hours of the evaluation range and measured three accuracy metrics on it. What if those very 24 hours are unusually good — or unusually bad? A single measurement at a single point on the time axis is *not* a robust verdict on the model quality.

A *rolling-origin backtest* shifts the forecast start point step by step through the entire evaluation period and runs a 24-hour forecast at each step. The result is a whole distribution of accuracy measurements: you see not only the mean but also the spread; not only the best- and worst-case but the typical range of variation.

10.2 AI Act link

Art. 9 of the AI Act (*risk management*) demands a *testing* of the system before it is put into service. Art. 10(3) requires that validation datasets are “representative”

— a term that, without an empirical test, remains a mere claim. The backtest provides exactly that empirical test.

10.3 Code in action

```
import pandas as pd
import pickle
from pathlib import Path

shared = Path("_cache/shared")
y_clean = pd.read_parquet(shared / "load_de_2024_clean.parquet")["load"]
y_train = pd.read_parquet(shared / "y_train.parquet")["load"]
exog    = pd.read_parquet(shared / "exog.parquet")

# A parquet round-trip drops the `freq` attribute of the DatetimeIndices;
# the forecaster needs it to construct forecast indices.
y_clean.index = pd.DatetimeIndex(y_clean.index, freq="h")
y_train.index = pd.DatetimeIndex(y_train.index, freq="h")
exog.index    = pd.DatetimeIndex(exog.index, freq="h")

with Path("_cache/forecaster.pkl").open("rb") as fh:
    model = pickle.load(fh)
```

```
from spotforecast2_safe.backtesting.validation import (
    TimeSeriesFold, backtesting_forecaster
)

# We restrict the backtest to the range for which calendar and weather
# features are available (training + evaluation range from chapter 05).
# That way we avoid asking the forecaster for hours whose `exog` values
```

```
# do not exist.
y_bt = y_clean.loc[exog.index.min():exog.index.max()]

cv = TimeSeriesFold(
    steps=24,
    initial_train_size=len(y_train),
    refit=False,
    fixed_train_size=False,
    verbose=False,
)

metrics_df, predictions_df = backtesting_forecaster(
    forecaster=model.forecaster,
    y=y_bt,
    cv=cv,
    metric="mean_absolute_error",
    exog=exog,
    show_progress=False,
    verbose=False,
)
print(f"Number of folds:    {len(metrics_df)}")
print(f"Mean MAE:           {float(metrics_df.iloc[0, 0]):.1f} MW")
print(f"Predictions total:  {len(predictions_df):,} h")
print()
print("First five predictions:")
print(predictions_df.head().round(1).to_string())
```

```
Number of folds:    1
Mean MAE:           2024.8 MW
Predictions total:  720 h

First five predictions:
                           fold     pred
```

```
2024-11-01 00:00:00+00:00     0  42052.3
2024-11-01 01:00:00+00:00     0  41891.0
2024-11-01 02:00:00+00:00     0  42106.2
2024-11-01 03:00:00+00:00     0  43918.6
2024-11-01 04:00:00+00:00     0  48347.7
```

> **i** `refit=False` and CR-2
>
> With `refit=False` every fold uses the same trained model — the forecast in week 4 therefore relies on the model fitted on data up to the end of October, not on a weekly-updated model. This is a deliberate discipline: we want to measure how the model forecasts *in live operation between updates*. If we retrained before every fold, we would systematically over-estimate ourselves.

10.4 Visualisation: the folds on the time axis

```python
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import matplotlib.patches as mpatches

n_show = min(8, len(metrics_df))
fold_starts = []
for i in range(n_show):
    start = predictions_df.index[i * 24]
    end   = predictions_df.index[i * 24 + 23]
    fold_starts.append((start, end))

window_start = fold_starts[0][0] - pd.Timedelta(hours=24)
window_end   = fold_starts[-1][1] + pd.Timedelta(hours=24)
context = y_bt.loc[window_start:window_end]
```

```
fig, ax = plt.subplots(figsize=(8.5, 3.5))
ax.plot(context.index, context.values / 1000,
        color="#999999", linewidth=0.6, label="ENTSO-E load")

for start, end in fold_starts:
    span = pd.date_range(start, end, freq="h")
    ax.axvspan(start, end, color="#E07B00", alpha=0.18)
    ax.plot([start, start], [ax.get_ylim()[0], ax.get_ylim()[1]],
            color="#E07B00", linewidth=0.6)

ax.set_ylabel("Load [GW]")
ax.set_xlabel("Time")
ax.grid(True, color="#E5E5E5", linewidth=0.5)
ax.xaxis.set_major_locator(mdates.DayLocator(interval=1))
ax.xaxis.set_major_formatter(mdates.DateFormatter("%-d.%-m"))
for s in ("top", "right"):
    ax.spines[s].set_visible(False)

fig.tight_layout()
plt.show()
```

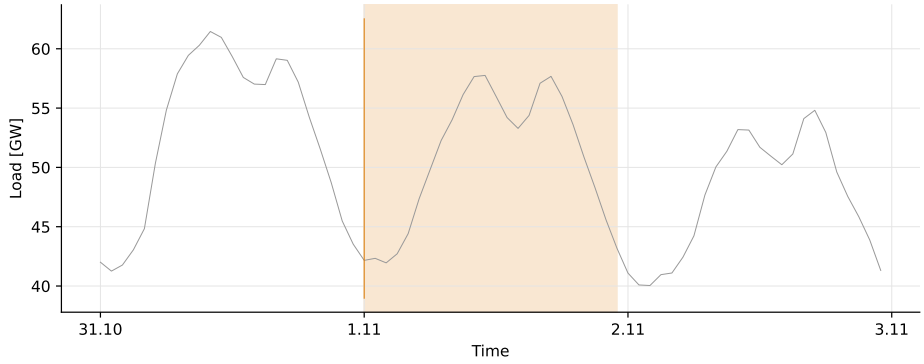


Figure 10.1: Visualisation of the rolling-origin folds. The training range (blue) ends at the orange separator line at the start of each fold. Each fold (orange-red) forecasts the following 24 hours. The ENTSO-E load in the background (grey) provides context.

10.5 Distribution of errors across folds

```
import numpy as np

# MAE per fold
mae_per_fold = []
for i in range(len(metrics_df)):
    fold_idx = predictions_df.index[i * 24 : (i + 1) * 24]
    pred_vals = predictions_df.iloc[i * 24 : (i + 1) * 24, 0].values
    actual = y_bt.loc[fold_idx].values
    mae_per_fold.append(np.abs(pred_vals - actual).mean())
mae_per_fold = np.array(mae_per_fold) / 1000   # in GW

fig, ax = plt.subplots(figsize=(8.5, 3.0))
ax.hist(mae_per_fold, bins=12, color="#1F4E79", alpha=0.85, edgecolor="white"
```

```
ax.axvline(np.median(mae_per_fold), color="#E07B00", linestyle="--",
           linewidth=1.2, label=f"Median = {np.median(mae_per_fold):.2f} GW")
ax.axvline(mae_per_fold.max(), color="#B33A3A", linewidth=1.2,
           label=f"Maximum = {mae_per_fold.max():.2f} GW")
ax.set_xlabel("MAE per 24-h fold [GW]")
ax.set_ylabel("Number of folds")
ax.legend(loc="upper right", frameon=False)
ax.grid(True, color="#E5E5E5", linewidth=0.5)
for s in ("top", "right"):
    ax.spines[s].set_visible(False)
fig.tight_layout()
plt.show()
```

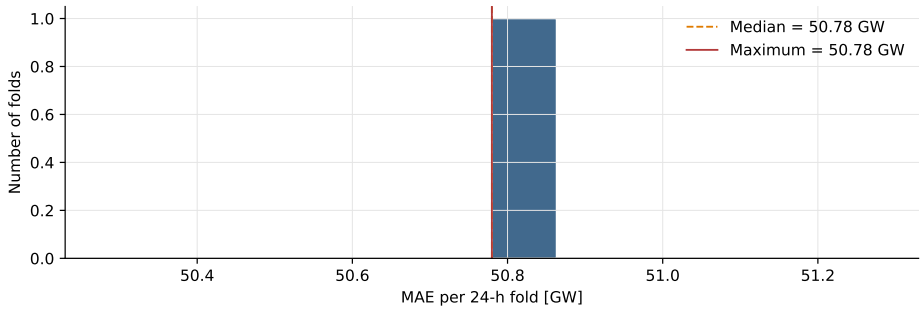


Figure 10.2: Distribution of MAE values across all folds of the backtest, plus the median (dashed) and the maximum value (red line). A narrow distribution means that the model quality is consistent across the evaluation range.

Exercises (with Claude)

1. *Change `refit`.* Have Claude explain what changes about the evaluation when `refit=True` is set — and how training cost and accuracy

trade off. In which practical situation is weekly re-training worth it?

2. *Adversarial fold.* Ask Claude to identify a fold in which the model performs particularly poorly and to discuss which data constellation might be responsible (holiday, weather event, load peak).
3. *Robustness threshold.* Discuss with Claude how you would formulate an audit threshold above which a fold is marked as "non-compliant" — and which actions should follow (model freeze, escalation to dispatcher, fallback).

11 Provenance and reproducibility

> **Learning objective**
>
> You can derive, from a trained forecaster and its audit log, the minimum amount of provenance information required by Art. 11 (technical documentation) and Art. 12 (record-keeping) of the AI Act — and you can explain how all four code rules together guarantee the reproducibility of a forecast across years.

11.1 What is this about?

Imagine that in two years' time a supervisory authority asks about a load forecast our system produced on 28 November 2024 at 06:00 for the following day. Can we, today, reproduce that forecast exactly as it was generated back then? If yes, the system is *auditable*. If not, the authority is entitled to forbid further operation.

Reproducibility presupposes that we have documented all inputs, hyperparameters and intermediate results completely enough for someone running the same code to arrive at the exact same forecast. This chapter is a retrospective: all four code rules and several AI Act articles come together here.

11.2 Which fields does `spotforecast2-safe` record automatically?

```
import json
import pandas as pd
import pickle
from pathlib import Path

shared = Path("_cache/shared")
with Path("_cache/forecaster.pkl").open("rb") as fh:
    model = pickle.load(fh)

provenance = {
    "model_name":            model.name,
    "estimator":             type(model.forecaster.estimator).__name__,
    "number_of_trees":       int(model.forecaster.estimator.n_estimators),
    "learning_rate":         float(model.forecaster.estimator.learning_rate)
    "lags":                  int(model.forecaster.lags.size),
    "seed":                  int(model.random_state),
    "training_index_from":   str(model.forecaster.training_range_[0]),
    "training_index_to":     str(model.forecaster.training_range_[1]),
    "calendar_country":      model.preprocessor.country_code,
    "number_of_periods":     len(model.preprocessor.periods),
    "exog_column_count":     int(len(model.forecaster.exog_names_in_ or []))
}
print(json.dumps(provenance, indent=2, ensure_ascii=False))
```

```
{
  "model_name": "lgbm",
  "estimator": "LGBMRegressor",
  "number_of_trees": 400,
  "learning_rate": 0.05,
```

```
  "lags": 168,
  "seed": 2026,
  "training_index_from": "2024-01-01 00:00:00+00:00",
  "training_index_to": "2024-10-31 23:00:00+00:00",
  "calendar_country": "DE",
  "number_of_periods": 3,
  "exog_column_count": 45
}
```


11.3 Data provenance from the audit log

The log switched on in Chapter 1 contains every data action: ENTSO-E download, weather fetch, gap handling, training, prediction. We extract the file names and timestamps:

```
log_dir = Path("_cache/logs")
log_files = sorted(log_dir.glob("*.log")) if log_dir.exists() else []
if log_files:
    latest = log_files[-1]
    lines = latest.read_text().splitlines()
    print(f"Log file: {latest.name}, {len(lines)} lines")
    print()
    print("First five entries:")
    for line in lines[:5]:
        print("  " + line)
else:
    print("No log files found.")
```

```
Log file: task_safe_n_to_1_20260531_215057.log, 2 lines

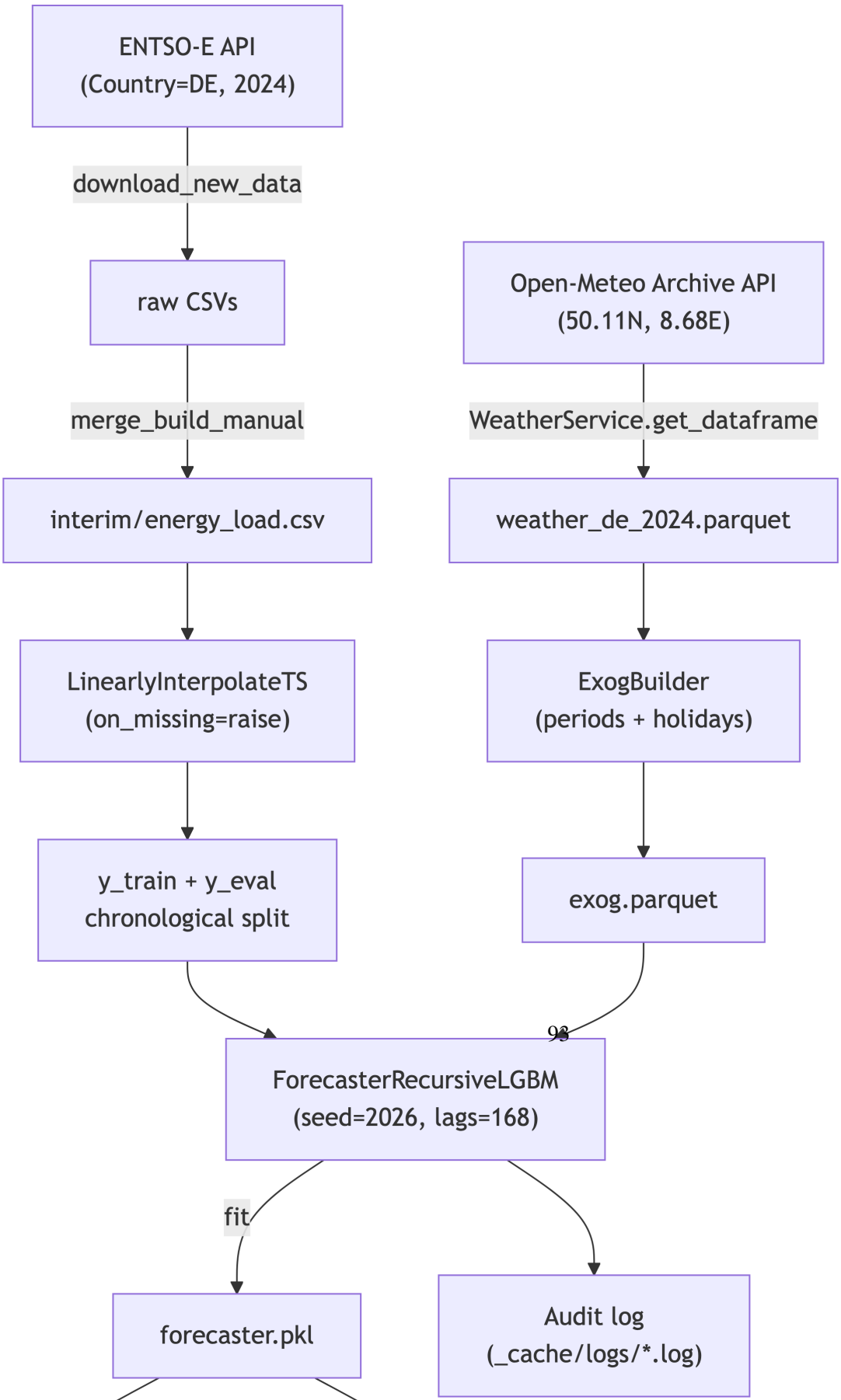
First five entries:
  {"schema_version":"1.0.0","timestamp_utc":"2026-05-31T19:50:57.585840Z","logger":"task_saf
```

{"schema_version":"1.0.0","timestamp_utc":"2026-05-31T19:50:57.586367Z","lo
forecasting pipeline (chapter 01)."}

11.4 Visualisation: the provenance diagram

The following Mermaid diagram summarises the data and control flows of the entire pipeline. In this form it can be reused directly in a `MODEL_CARD.md`.

```
flowchart TD
    A["ENTSO-E API<br/>(Country=DE, 2024)"] -->|download_new_data| B["raw CSV
    B -->|merge_build_manual| C["interim/energy_load.csv"]
    O["Open-Meteo Archive API<br/>(50.11N, 8.68E)"] -->|WeatherService.get_da
    C --> D["LinearlyInterpolateTS<br/>(on_missing=raise)"]
    D --> E["y_train + y_eval<br/>chronological split"]
    W --> F["ExogBuilder<br/>(periods + holidays)"]
    F --> G["exog.parquet"]
    E --> H["ForecasterRecursiveLGBM<br/>(seed=2026, lags=168)"]
    G --> H
    H -->|fit| M["forecaster.pkl"]
    M -->|predict| P["y_pred_24h.parquet"]
    M -->|backtesting_forecaster| BT["metrics_df + predictions_df"]
    H --> L["Audit log<br/>(_cache/logs/*.log)"]
```


ENTSO-E API
(Country=DE, 2024)
download_new_data
raw CSVs
merge_build_manual
interim/energy_load.csv
LinearlyInterpolateTS
(on_missing=raise)
y_train + y_eval
chronological split
Open-Meteo Archive API
(50.11N, 8.68E)
WeatherService.get_dataframe
weather_de_2024.parquet
ExogBuilder
(periods + holidays)
exog.parquet
ForecasterRecursiveLGBM
(seed=2026, lags=168)
fit
forecaster.pkl
Audit log
(_cache/logs/*.log)

11.5 How all four code rules come together here

Code rule	Effect in this pipeline
CR-1	Every function (`download_new_data`, `LinearlyInterpolateTS`, `ExogBuilder`, `WeatherService`, `ForecasterRecursiveLGBM`, `backtesting_forecaster`) is represented in the logging and protected by tests.
CR-2	Fixed date constants, fixed seed, `deterministic=True` in LightGBM, fixed `random_state` — bit-exact reproducibility even after years.
CR-3	NaN behaviour in `LinearlyInterpolateTS` (`on_missing='raise'`) and in `WeatherService` (`fill_missing=False`) — gaps are reported, not silently filled.
CR-4	Deny-list forbids `plotly`, `matplotlib`, `optuna`, `torch`, `tensorflow` as dependencies; LightGBM and scikit-learn are allowed because they are small and well maintained.

11.6 AI Act link, compact

Obligation	Article	Fulfilled in the pipeline by
Risk management	Art. 9	Backtest in Chapter 10, deterministic training, audit log
Data governance	Art. 10	Provenance note from chapter 02, NaN discipline from Chapter 4, weather fail-safe
Technical documentation	Art. 11	Provenance diagram, JSON entry above, MODEL_CARD.md in the package
Record-keeping	Art. 12	Audit log, persisted intermediate artefacts (`*.parquet`, `*.pkl`)

Obligation	Article	Fulfilled in the pipeline by
Transparency	Art. 13	Feature-importance plot in Chapter 7, recursion equation in Chapter 8
Accuracy/robustness	Art. 15	Four metrics in Chapter 9 plus backtest distribution

11.7 Persisting the provenance note

```
import json
out = Path("_cache/provenance.json")
out.write_text(json.dumps(provenance, indent=2, ensure_ascii=False))
print(f"Provenance note: {out}")
```

```
Provenance note: _cache/provenance.json
```

Exercises (with Claude)

1. *Reproduction in two years.* Ask Claude to sketch a concise recovery plan: an audit team wants to reproduce in 2027 the forecast from 28 November 2024 for 29 November 2024. Which five artefacts of this pipeline are needed at minimum? Which code rules make this endeavour possible at all?
2. *Gap analysis.* Discuss with Claude which additional provenance fields *beyond* the list printed here should appear in a real `MODEL_CARD.md` (e.g. hash of the training data, versions of every dependency, hardware description). Which standard expressly requires these fields (AI Act, IEC 62443, ISO 25059)?
3. *Audit script.* Have Claude write a small Python script that reads the persisted `provenance.json` and automatically checks whether all

mandatory fields are populated. Which code rule is the prerequisite for such an automated check to be meaningful at all?

11.8 Closing remark

In ten chapters you have built a complete safety-critical load-forecasting pipeline: from live data acquisition through cyclic features, weather, deterministic training, prediction, accuracy measurement and backtest, all the way to a reproducible provenance note. If you were now asked in an audit how you comply with the AI Act, you can answer with *code* instead of reassurances — and that is precisely the spirit in which `spotforecast2-safe` was developed.

12 Challenge: live load forecasting

> **Learning objective**
>
> You take the pipeline learned in chapters 01-11 into a **live** 24-hour load forecast, submit a prediction for the next day every day, and defend your submission in an auditing sense (CR-1 to CR-4, Art. 10, 12, 13, 15 of the AI Act). Evaluation is automatic against the ENTSO-E ground truth; the leaderboard is updated weekly.

12.1 Why a live challenge?

Chapters 02-11 deliberately use a **frozen** time slice (`DATA_START = 2024-01-01`, `DATA_END = 2024-12-31`) so that every student obtains byte-identical results (CR-2). That advantage disappears the moment the model is to run *in production*: new measurements arrive every hour, forecasters are retrained on a rolling window, gaps appear unpredictably, and the AI Act still demands a verifiable audit trail (Art. 12) and robustness against distribution drift (Art. 15).

The challenge simulates this live setting. You submit a forecast every day; an automatic evaluation workflow compares it the next day with the value finally published by ENTSO-E. The setup is modelled on the well-known scientific competitions:

- **GEFCom 2012/2014/2017** (Hong et al. 2014, 2016, 2019) — the Global Energy Forecasting Competition, the academic standard for load-forecasting competitions (IEEE Working Group / *International Journal of Forecasting*).
- **M4 / M5 / M6** (Makridakis et al. 2020, 2022) — the Makridakis competitions that have established conventions for rolling backtesting and point/quantile evaluation.
- **EEM Forecasting Competitions** (IEEE European Energy Market) as a recurring European annual format.
- **BDEW standard load profiles** as a German industry benchmark.

Methodologically this challenge closely follows GEFCom 2017 (hierarchical load forecasting) and M4 (rolling one-step-ahead setup).

12.2 AI Act link

Challenge rule	Code rule	AI Act
Submission deadline *before* availability of target hours	CR-3	Art. 10, 15
Completeness of submission (24 rows, no NaN)	CR-3	Art. 10
Git commit timestamps auditable in CI	CR-1	Art. 12
Score run deterministically reproducible	CR-2	Art. 12, 15
Pinned pipeline dependencies (uv.lock)	CR-4	Art. 15
Ground-truth source documented (ENTSO-E final)	CR-1	Art. 10, 13

12.3 Rules at a glance

Aspect	Definition
Target variable	DE hourly load `Actual Load` [MW] from the ENTSO-E Transparency Platform
Horizon	24 h (00:00-23:00 CET of target day *D*)
Submission cadence	daily, one entry per team
Deadline (target day *D*)	*D-1* 23:59 CET — later-arriving PRs are rejected
Ground-truth availability	*D+1* around 06:00 UTC (ENTSO-E *final values*)
Score run	daily *D+1* 07:00 UTC via GitHub Actions cron
Primary metric	**MAE** [MW] over the 24 hours
Ranking (public)	**Rolling 7-day mean MAE** — moving weekly average
Tie-break 1	cumulative MAE since kickoff (smaller wins)
Tie-break 2	number of valid submissions (more wins)
Anti-leakage	Deadline lies *before* the first target hour; commit timestamps verifiable in CI
Leaderboard refresh	after every score run (daily); weekly snapshot Sunday 23:55 UTC

For orientation: the *state-of-the-art* accuracy for the German day-ahead load forecast is around MAPE ~3 %. A simple **weekly persistence** (load exactly 168 h earlier) typically yields MAPE 5-7 % and is a useful sanity check (see Chapter 9).

12.4 Participation

- **Team size**: 2-4 people. Solo participation is not foreseen.
- **Registration**: until 12 May 2026 (Tuesday) by email to Prof. Bartz-Beielstein, with (i) team name, (ii) team ID (short, lowercase, no space, e.g. `team_lambda`), (iii) GitHub handles of all members.

- One person from the team is registered in `teams.yml` as the submission owner; however, every team member may submit via PR.

12.5 GitHub onboarding

Technically the challenge runs on GitHub. If you have never worked with it, get yourself up to speed in three steps here: create an account, find your handle, set up authentication. If you already use GitHub actively, you can skip this section and jump straight to Section 12.7.

> **Learning objective**
>
> You have a GitHub account, know your handle, have forked the leaderboard repo and cloned it locally. `git push` and `git pull` work without a password prompt.

12.5.1 What is GitHub, what is your handle?

GitHub is a public hosting service for Git repositories (https://github.com/). Every account has a unique *username*, also called *handle* or *login*. The handle is always identical to the final URL component of your profile:

```
https://github.com/<handle>
```

Format: 1-39 characters, letters/digits/hyphen, no umlauts. GitHub ignores case when comparing — internally everything is normalised to lowercase.

Why the handle matters here. Submissions go through pull requests. The CI workflow `validate-pr.yml` in the leaderboard repository reads the author's *login* from the GitHub event when a PR arrives and compares it with the `github_handles` list of the relevant team in `teams.yml`. Anyone not on that list gets:

```
ERROR: PR author 'X' not in github_handles for team 'T': [...]
```

This implements CR-1 (identity & auditability): only you and your team members can submit *on behalf of your team*.

12.5.2 Create an account

1. Open https://github.com/signup.
2. Pick an email, password and *username*. **Important:** the username is your future handle and should look professionally solid (`tina.musterhuhn` looks more serious than `xxtina4lifexx`).
3. Click through the email confirmation.
4. Activate two-factor authentication (Settings -> Password and authentication -> Two-factor authentication). A TOTP app like Aegis or 1Password suffices.

12.5.3 Find your handle (for the registration email)

Two reliable ways:

- **Click the profile picture top right** — the first menu item (“Your profile”) shows your handle.
- **In any profile URL.** Visiting https://github.com/ while logged in redirects to `https://github.com/<handle>`.

! Send handles to the lecturer

Send *all* handles of your team in the registration email, otherwise individual members will later be unable to submit PRs. Format like the example below:

> *Registration team “Lambda Forecasters”*
> *Team-ID: `team_lambda`*
> *Members and handles:*

- *Alice Mueller — GitHub:* `alice42`
- *Bob Schmidt — GitHub:* `bob99`
- *Carol Schulze — GitHub:* `carol7`

Corrections are possible at any time but cost one cycle: you report the fix by email, the lecturer commits the changed `teams.yml`, and you push a new (or empty) commit on your PR so that CI revalidates.

12.5.4 Set up authentication locally

Otherwise `git push` asks for a password every time. Choose *one* of the two variants.

Variant A — SSH key (recommended for daily use).

```
ssh-keygen -t ed25519 -C "your.address@th-koeln.de"
# Copy the public key (~/.ssh/id_ed25519.pub) to the clipboard:
cat ~/.ssh/id_ed25519.pub
```

Then on https://github.com/settings/keys -> *New SSH key* -> paste contents -> save. Test: `ssh -T git@github.com`.

Variant B — HTTPS with Personal Access Token.

GitHub -> Settings -> Developer settings -> Personal access tokens -> *Fine-grained tokens* -> *Generate new token*. Scope: read/write on the fork of the leaderboard repo. Enter the token as a password on the first `git push`; Git stores it in the credential helper.

12.5.5 Install the `gh` CLI (optional but practical)

`gh` is GitHub's official command-line tool. It lets you fork, open and merge PRs without touching the browser.

```
# macOS:
brew install gh
# Windows:
winget install --id GitHub.cli
# Linux (Debian/Ubuntu):
sudo apt install gh
```

One-off login: `gh auth login --web`. Then `gh auth status` is enough to check that everything is in place.

12.5.6 Fork and local clone

Before you can submit, you need a **fork** of the leaderboard repo — your own copy at `https://github.com/<your-handle>/challenge-leaderboard`. GitHub rejects direct pushes into the original repo (branch protection), and that is intentional: everything goes through pull requests.

Web UI:

1. Open https://github.com/bartzbeielstein/challenge-leaderboard.
2. Click *Fork* top right -> Owner = your own account -> *Create fork*.
3. On the fork page click *Code* -> copy the SSH (`git@github.com:...`) or HTTPS (`https://github.com/...`) URL.
4. Locally:

```
git clone git@github.com:<your-handle>/challenge-leaderboard.git
cd challenge-leaderboard
uv sync
```

CLI:

```
gh repo fork bartzbeielstein/challenge-leaderboard --clone
cd challenge-leaderboard
uv sync
```

`uv sync` installs the Python dependencies pinned in the repo — identical to the stack the CI uses for validation (CR-2).

12.6 Submission format

A submission is a single CSV file with the following schema:

Column	Type	Meaning
`timestamp_utc`	`str` (ISO)	hour start in UTC, exactly 24 consecutive hours of the target day D
`forecast_mw`	`float`	predicted load value in megawatts, strictly positive

Path convention in the leaderboard repository:

```
submissions/<team_id>/<YYYY-MM-DD>.csv
```

Here `<YYYY-MM-DD>` is the target day D (the date *for which* the forecast applies — not the submission day). Example: on 14 May 2026 at 17:30 team `team_lambda` submits the forecast for 15 May:

```
submissions/team_lambda/2026-05-15.csv
```

A mini example of the CSV file (excerpt):

```
import pandas as pd

sample = pd.DataFrame({
    "timestamp_utc": [
        "2026-05-15T00:00:00Z",
        "2026-05-15T01:00:00Z",
        "2026-05-15T02:00:00Z",
        "...",
        "2026-05-15T23:00:00Z",
    ],
    "forecast_mw": [48_500.0, 47_200.0, 46_400.0, "...", 52_100.0],
})
print(sample.to_string(index=False))
```

```
       timestamp_utc forecast_mw
2026-05-15T00:00:00Z     48500.0
2026-05-15T01:00:00Z     47200.0
2026-05-15T02:00:00Z     46400.0
                 ...         ...
2026-05-15T23:00:00Z     52100.0
```

For a deviating schema (too many/few rows, missing column, wrong date, `NaN` or negative values) the CI validation workflow rejects the PR with a precise error line (CR-3).

12.7 Submission workflow

Prep done (Section 12.5)? Then now follows the daily routine. It consists of a **one-time setup** and a **recurring submission cycle**.

12.7.1 One-time setup — preconditions checklist

1. Valid GitHub account and handle submitted to the lecturer (Section 12.5).
2. Fork of the leaderboard repo created and cloned locally (`challenge-leaderboard/` on your disk).
3. `uv sync` run in the repo root — the Python stack is in place.
4. `ENTSOE_API_KEY` is exported in the shell (`export ENTSOE_API_KEY=...`). Without this key the generator script cannot fetch real data.
5. The generator script `make_submission.py` from the course repo is available (see Section 12.8) — or your own.

12.7.2 Submission cycle per target day

In the following, `<team_id>` denotes your team's ID (e.g. `team_lambda`) and `<D>` the target day in `YYYY-MM-DD` format (e.g. `2026-05-13`).

12.7.2.1 1. Produce the CSV

In the root directory of your local leaderboard fork:

```
cd .../challenge-leaderboard
python .../lecture/scripts/make_submission.py \
    --team <team_id> --target-date <D>
```

The script writes the file to `submissions/<team_id>/<D>.csv`. Inspect briefly: 24 rows, plausible values (DE load typically 40 000 - 70 000 MW).

12.7.2.2 2. Create a feature branch

One separate branch per submission.

CLI:

```
git switch -c submission/<team_id>/<D>
```

Web UI: on `https://github.com/<your-handle>/challenge-leaderboard` open the branch dropdown top left, type the new branch name and confirm with *Create branch …*. Then locally `git fetch && git switch submission/<team_id>/<D>`.

12.7.2.3 3. Add the CSV and commit

> ⚠ Exactly one file per PR
>
> `validate-pr.yml` counts the changed files under `submissions/**` and rejects unless there is **exactly one**. So add only the one new CSV, no `*` and no `-A`.

```
git add submissions/<team_id>/<D>.csv
git commit -m "submission(<team_id>): forecast for <D>"
```

12.7.2.4 4. Push the branch

```
git push -u origin submission/<team_id>/<D>
```

The success message contains a direct link to create the PR.

12.7.2.5 5. Open a pull request

Web UI: GitHub shows a yellow banner *"Compare & pull request"* on the repo page right after the push. Click -> fill in title/body -> *Create pull request*. Important:

- *Base repository* is `bartzbeielstein/challenge-leaderboard` (the upstream, **not** your fork).
- *Base branch* is `main`.
- *Head repository* is your fork, *head branch* is `submission/<team_id>/<D>`.

CLI:

```
gh pr create --base main \
    --title "submission(<team_id>): <D>" \
    --body  "24h load forecast team <team_id>, target day <D>."
```

12.7.2.6 6. Wait for validation

Web UI: the PR page shows the *Checks* area at the bottom. After ~15 seconds you see either a green tick or a red cross. On red: go through the steps in Section 12.7.3.

CLI:

```
gh pr checks <pr-number> --watch
```

12.7.2.7 7. Merge

When validation is green:

Web UI: button *Squash and merge* -> *Confirm squash and merge* -> *Delete branch* (cleans up the feature branch on the server).

CLI:

```
gh pr merge <pr-number> --squash --delete-branch
```

12.7.2.8 8. Sync local `main`

```
git switch main
git pull origin main
```

The feature branch is deleted remotely; locally you can clean it up with `git branch -d submission/<team_id>/<D>`.

> **What happens after the merge?**
>
> The **daily scoring workflow** runs every day at 07:00 UTC. For every target day before its run it pulls the ENTSO-E ground truth, computes the MAE of every submitted forecast and appends the scores to `data/scores.parquet`. Then the **leaderboard workflow** rebuilds the HTML page and deploys it to GitHub Pages. You will therefore see your evaluation on the day *after* the target day.

Per target day and team: **the latest merged valid submission counts**. If you submit at 17:00 and want to take a better shot at 22:00, that is allowed — until 23:59 CET on *D-1*. For the correction add the same path file again (Git sees it as a modification; the validator accepts it).

12.7.3 When the validator is red

The most common error lines from `validate_submission.py` and how to fix them:

Error message	What to do?
`PR author 'X' not in github_handles for team 'T': [...]`	Handle missing from `teams.yml`. Short email to the lecturer. After their commit on `main`: empty commit on your PR branch (`git commit --allow-empty -m "ci: retrigger" && git push`), so that the CI recomputes with the new `teams.yml`.
`PR must change or add exactly one submission (found: N)`	Branch contains junk changes (e.g. accidentally cached `_cache/` or a second `.csv`). `git restore --staged .`, then add only the right CSV, `git commit --amend` and `git push --force-with-lease`.
`CSV: expected 24 rows, found N`	Generator bug: forecast horizon does not match. Check target day (UTC!), regenerate CSV.
`Target day in path and in file do not match`	The `YYYY-MM-DD` in the file name must match the `timestamp_utc` day in the CSV exactly.
`Negative or NaN values in forecast_mw`	Pipeline passed an endpoint NaN through. Check the trim (`y.loc[:y.last_valid_index()]`) in the generator.

For multiple errors: work through them in order, retrigger each time with an empty commit or force-push.

12.8 Baseline implementation

The following script produces a *rule-conformant* submission. It is a direct application of the pipeline from chapters 02, 04, 06 and 07, with two differences from the main book strand:

1. **Live data window** instead of the frozen 2024 slice. We load the last 30 days of ENTSO-E data at training time.
2. **Forecast for *tomorrow*** instead of for a test slice from 2024.

```
"""
make_submission.py --- produces <repo-root>/submissions/<team>/<D>.csv.

Usage (from the root directory of your challenge-leaderboard fork):
    cd .../challenge-leaderboard
    python .../make_submission.py --team team_lambda --target-date 2026-05-15

Optionally explicit:
    python .../make_submission.py --team ... --target-date ... \
        --repo-root /path/to/challenge-leaderboard

Prerequisites:
- ENTSOE_API_KEY is set in the shell.
- Directory `submissions/` exists under --repo-root.

Minimal pipeline: a pure lag model (168 h ~ weekly persistence)
without calendar/weather exogenous variables. More accurate and
course-near is the full pipeline from chapters 06-08; for the first
submission run this script is sufficient.
"""
import argparse
import os
from datetime import datetime, timedelta, timezone
from pathlib import Path

import pandas as pd

from spotforecast2_safe import ForecasterRecursiveLGBM, LinearlyInterpolateTS
from spotforecast2_safe.data.fetch_data import fetch_data, get_data_home
```

```
from spotforecast2_safe.downloader.entsoe import download_new_data


def make_submission(team: str, target_date: str, repo_root: Path) -> Path:
    target = datetime.fromisoformat(target_date).replace(tzinfo=timezone.utc)
    train_end = target - timedelta(hours=1)
    train_start = train_end - timedelta(days=30)

    api_key = os.environ["ENTSOE_API_KEY"]
    download_new_data(
        api_key=api_key,
        country_code="DE",
        start=train_start.strftime("%Y%m%d%H%M"),
        end=train_end.strftime("%Y%m%d%H%M"),
        force=True,
    )

    interim = get_data_home() / "interim" / "energy_load.csv"
    df_raw = fetch_data(filename=str(interim))
    df_raw.index = pd.to_datetime(df_raw.index, utc=True)

    load_col = next(c for c in df_raw.columns if "Actual" in c and "Load" in
    y = df_raw[load_col].astype(float).rename("load")
    if y.index.inferred_freq != "h":
        y = y.resample("h").mean()
    y = y.loc[train_start:train_end]
    # ENTSO-E publishes "Actual Load" with a few hours' delay.
    # We cut off at the last actually present hour and let the
    # forecaster bridge the gap to the end of the target day.
    y = y.loc[:y.last_valid_index()]

    # CR-3: handle gaps explicitly instead of silently interpolating
    y = LinearlyInterpolateTS(on_missing="raise").fit_transform(y)
```

```
    # Forecaster requires a DatetimeIndex with frequency set
    y.index = pd.DatetimeIndex(y.index, freq="h")

    model = ForecasterRecursiveLGBM(
        iteration=0,
        lags=168,
        random_state=2026,  # CR-2: deterministic
    )
    model.fit(y)

    target_hours = pd.date_range(target, periods=24, freq="h", tz="UTC")
    horizon = int((target_hours[-1] - y.index[-1]) / pd.Timedelta(hours=1))
    y_pred_full = model.forecaster.predict(steps=horizon)
    y_pred = y_pred_full.iloc[-24:]

    submission = pd.DataFrame({
        "timestamp_utc": target_hours.strftime("%Y-%m-%dT%H:%M:%SZ"),
        "forecast_mw": y_pred.values.round(2),
    })

    submissions_dir = repo_root / "submissions"
    if not submissions_dir.is_dir():
        raise SystemExit(
            f"Directory '{submissions_dir}' not found. "
            f"Call from the root directory of the challenge-leaderboard repo "
            f"is expected, or specify --repo-root explicitly."
        )

    out = submissions_dir / team / f"{target_date}.csv"
    out.parent.mkdir(parents=True, exist_ok=True)
    submission.to_csv(out, index=False)
    return out
```

```
if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--team", required=True)
    parser.add_argument("--target-date", required=True,
                        help="YYYY-MM-DD (UTC date of forecast target)")
    parser.add_argument("--repo-root", type=Path, default=Path.cwd(),
                        help="root directory of the challenge-leaderboard "
                             "repo (default: current working directory)")
    args = parser.parse_args()
    path = make_submission(args.team, args.target_date, args.repo_root.resolv
    print(f"Submission written: {path}")
```

⚠ Live code not executed in the book

This code block is marked with `#| eval: false`: it depends on a *live* ENTSO-E request and would violate the CR-2 invariant of the deterministic book build (see `CLAUDE.md`). Run it locally, but in the PR check in **only** the generated CSV.

12.9 Scoring methodology in detail

For every valid submission of team t for target day D the scoring script computes:

$$\mathrm{MAE}_{t,D} = \frac{1}{24}\sum_{h=0}^{23} \left| y_{t,D,h}^{(\mathrm{forecast})} - y_{D,h}^{(\mathrm{actual})} \right|$$

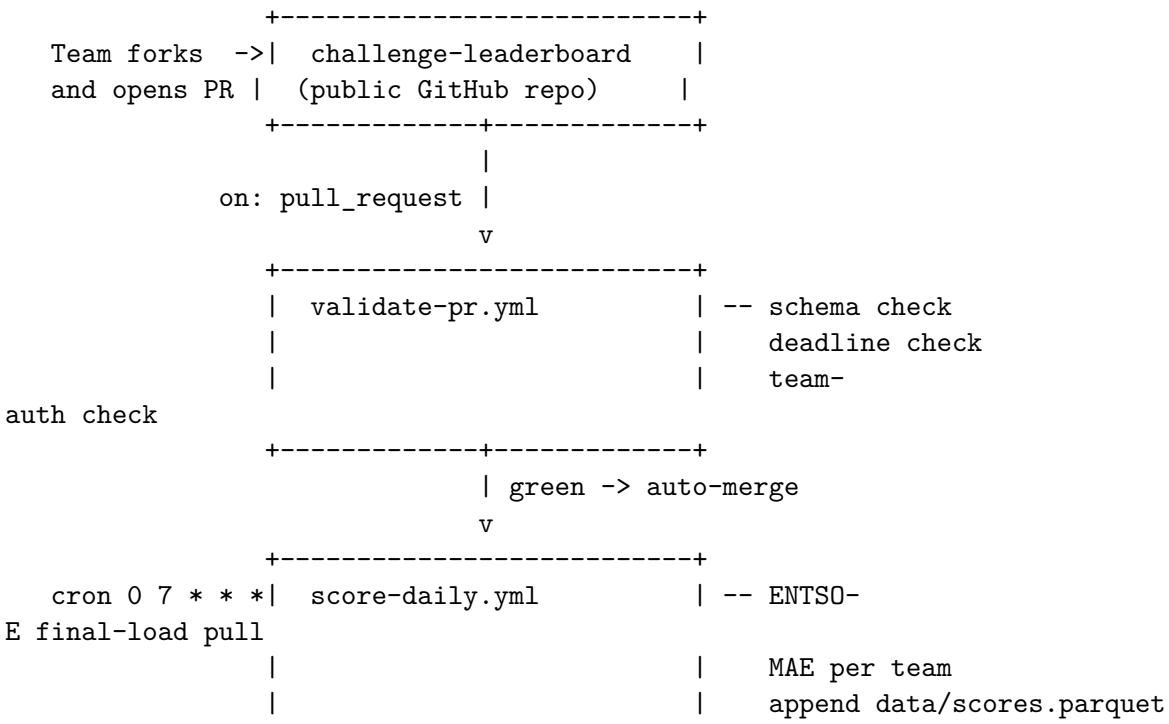
with $y_{D,h}^{(\mathrm{actual})}$ being the ENTSO-E *final value* for hour h. Additionally, RMSE and MAPE are logged (for diagnostics, not for the ranking).

The **public ranking** is the rolling 7-day mean:

$$\overline{\mathrm{MAE}}_t(D_{\text{today}}) \;=\; \frac{1}{7}\sum_{k=0}^{6} \mathrm{MAE}_{t,\,D_{\text{today}}-k}$$

(omitting days without a valid submission). Anyone who fails to reach four valid submissions in a given week does not appear in the main ranking that week but in a "quota" side ranking. The interim presentation on 2026-06-22 requires at least 30 valid submissions.

12.10 Evaluation architecture (GitHub Actions + Pages)



```
                +-------------+-------------+
                              | workflow_run
                              v
                +---------------------------+
                |  build-and-deploy.yml     | -- aggregate, HTML render
                |                           |    deploy to gh-
pages
                +-------------+-------------+
                              |
                              v
                Leaderboard on GitHub Pages,
                public, readable without login.
```

Three properties are constitutive here:

- **Auditability (Art. 12 AI Act)**: every submission, every scoring result and every leaderboard state is a commit in the Git history; nothing is overwritten.
- **Determinism (CR-2)**: the scoring workflow pins its dependencies in `uv.lock`, sets `PYTHONHASHSEED=0` and freezes the ENTSO-E response as a snapshot in the same commit that writes the score result.
- **Fail-safe (CR-3)**: if a single value is missing in the ENTSO-E pull, the validator (with `LinearlyInterpolateTS(on_missing="raise")`) aborts and does *not* publish a partial result; scoring is postponed by one day.

Complete workflow snippets live in the `challenge-leaderboard` repo (path: `.github/workflows/`).

12.11 Leaderboard

The publicly viewable page is at `https://bartzbeielstein.github.io/challenge-leaderboard/`. It contains two tables:

1. **Main ranking**: teams sorted by rolling 7-day MAE, ascending.

2. **Cum-ranking**: teams sorted by cumulative MAE since kickoff (tie-break).

Both tables are regenerated after every scoring run; the Sunday afternoon snapshot remains archived in `public/snapshots/` so that weekly winners remain traceable.

12.12 Calendar

Date	Event
2026-05-11	Challenge kickoff (slot: "Background, challenge")
2026-05-11	First possible submission (target day 2026-05-12)
2026-06-22	2nd interim presentation: at least 30 valid submissions
2026-07-19	Last scored submission (target day 2026-07-20)
2026-07-20	3rd final presentation: winners announcement
2026-07-27	Written exam (challenge code defended in the oral part)

Exercises (with Claude)

Exercises (with Claude)

1. *Beat the baseline.* Starting from the `make_submission.py` script shown above, ask Claude for three concrete levers that typically reduce the MAE (e.g. additional exogenous variables, longer training window, different lag depth). For each lever discuss whether it endangers CR-2 (determinism) — e.g. weather forecasts with non-deterministic updates.
2. *Understand block-wise CV.* Have Claude explain why classical *k-fold* cross-validation leads to over-optimistic numbers in this competition and which function in `spotforecast2_safe.model_selection`

correctly implements the rolling-origin scheme (see Chapter 10).

3. *Weekly-persistence sanity check.* Ask Claude to write a 10-line variant of the baseline script that, without training, simply submits $y_{D,h} = y_{D-7,h}$. Enter this as a "persistence team" in parallel — a team that cannot beat this baseline has a methodological problem.
4. *Recreate the validator locally.* Ask Claude to translate the schema and deadline check implemented in `challenge-leaderboard/scripts/validate_submission.py` into a local pytest test. Why is it a CR-3 violation to rely only on the server-side validator?

Appendix: maintaining `teams.yml` (for lecturers)

The team registry `teams.yml` in the leaderboard repo is the only place where permissions are checked: only people whose GitHub handle appears in `github_handles` may merge PRs for their team.

12.12.1 Fields

Field	Required	Rules
`id`	yes	lowercase letters, digits and `_` — appears in the path `submissions/<id>/...`
`display_name`	yes	free string — appears in the public leaderboard
`github_handles`	yes	list of GitHub logins **without** `@`, **case-insensitive** (validator lowercases)

12.12.2 Concrete example: handle two registrations

Email 1 (10 May 2026, 14:02):

> *Hello, we would like to register as "Team Lambda". Members: Alice Mueller (GitHub: alice42), Bob Schmidt (bob99), Carol Schulze (carol7). Best regards, …*

Email 2 (10 May 2026, 19:18):

> *Registration team "Omega-Forecaster" (Dora Weber, dora88; Eve Klein, eve12).*

1. Open the file:

```
cd .../challenge-leaderboard
$EDITOR teams.yml
```

2. Add the entries. From `teams: []` you get:

```yaml
teams:
  - id: team_lambda
    display_name: "Team Lambda"
    github_handles:
      - alice42
      - bob99
      - carol7
  - id: omega_forecaster
    display_name: "Omega-Forecaster"
    github_handles:
      - dora88
      - eve12
```

Notes on the second entry:

- `id` must **not** contain a special character — from "Omega-Forecaster" comes the ID `omega_forecaster`. `display_name` may keep the original Omega-.
- The ID later appears under `submissions/omega_forecaster/<D>.csv`, so communicate it to the team.

3. Validate locally before committing:

```
import yaml

d = yaml.safe_load(open("teams.yml"))
ids = [t["id"] for t in d["teams"]]
assert len(ids) == len(set(ids)), f"Duplicate IDs: {ids}"
for t in d["teams"]:
    assert t["id"].islower() and all(c.isalnum() or c == "_" for c in t["id"]
    assert t["github_handles"], t["id"]
print(f"OK: {len(ids)} teams: {ids}")
```

Expected: `OK: 2 teams: ['team_lambda', 'omega_forecaster']`.

4. Commit & push:

```
git add teams.yml
git commit -m "teams: registrations wave 1 (Lambda, Omega-Forecaster)"
git push
```

The `build-and-deploy.yml` workflow listens to changes in `teams.yml` and re-renders the leaderboard automatically — after that the `display_name` strings appear as headers in the ranking even when no scores yet exist.

5. Registration confirmation to the teams. Include in the reply email:

- Repository URL `https://github.com/bartzbeielstein/challenge-leaderboard`

- **Team ID** (filename-safe, use exactly as is): `team_lambda` or `omega_forecaster`
- Submission path convention `submissions/<team-id>/<YYYY-MM-DD>.csv`
- Note: only the named GitHub handles can merge PRs for the team.

12.12.3 Further registration waves

Simply append further waves to the `teams` list — the order does not matter, the leaderboard sorts by score. If a team member joins later, just add the handle to `github_handles` and push.

12.12.4 Common pitfalls

- `id` **with a hyphen** like `team-lambda` -> the path regex in the validator (`[a-z0-9_]+`) rejects it. Use only `_` as separator.
- **GitHub handle with uppercase letters** like `Alice42` -> no problem, the validator compares lowercased. Still customary to write lowercase.
- **Duplicate** `id` -> the validator itself does not check this; the local quick check in step 3 does. Duplicate IDs would overwrite submissions reciprocally.

13 Live submission to the ddmo-sose-26 challenge

> **Learning objective**
>
> You can generate today's 24-hour DE load forecast with the production pipeline (`spotforecast2` + `spotforecast2-safe`), write a validated submission CSV in the exact schema the leaderboard expects, and open the pull request that puts your team on the live ranking at https://bartzbeielstein.github.io/challenge-leaderboard/.

13.1 Overview

Previous chapters walked through every stage of the pipeline against a *frozen* 2024 evaluation window. That deterministic, reproducible sandbox is exactly what the EU AI Act asks for in the development phase. This final chapter is different in two ways: the data is *today's*, and the target is `y_0` (tomorrow), not a held-out 2024 window. Everything else, i.e., the safety contracts, the assertions, and the fail-loudly semantics, carries over.

13.2 Submission contract

The leaderboard repository `challenge-leaderboard` accepts one CSV per team per day with the following exact shape:

Table 13.1: Submission CSV schema.

Column	Type	Constraint
`timestamp_utc`	ISO 8601 string (`YYYY-MM-DDTHH:MM:SSZ`)	24 consecutive UTC hourly stamps for the target day
`forecast_mw`	`float`	strictly positive, no NaN, finite

File path: `submissions/team_4_optuna/<YYYY-MM-DD>.csv` — the filename is the *target* day (the day being forecast), not the submission day. Deadline: `D-1 23:59 UTC` (the validator rejects later PRs). Scoring: per-day mean absolute error in MW (`MAE = (1/24) Σ |ŷ_h - y_h|`). Public ranking: rolling 7-day mean MAE, smallest wins; ties broken first by cumulative MAE, then by number of valid submissions.

The implementation is auditable. The load-bearing contract anchors are published on github.com (https://github.com/bartzbeielstein/challenge-leaderboard/tree/main/scripts) and visible in the codebase:

- `validate_schema()` at `scripts/validate_submission.py:53`
- `validate_deadline()` at `scripts/validate_submission.py:79`
- `score_submission()` at `scripts/score_day.py:112`
- `aggregate()` at `scripts/build_leaderboard.py:35`
- The daily scoring cron at `.github/workflows/score-daily.yml:5`

13.3 Registered teams

The team registry lives in `challenge-leaderboard/teams.yml`. The cell below reads it from the sibling clone and renders the current roster. This chapter is wired for the registered team `team_4_optuna` (display name "Team 4 Optuna", handle `bartzbeielstein`); swap that `team_id` and the GitHub handle for the row that matches your own team when you adapt the code:

```
from pathlib import Path
import yaml, pandas as pd

teams_yml = Path.home() / "workspace" / "challenge-leaderboard" / "teams.yml"
teams = yaml.safe_load(teams_yml.read_text())["teams"]
pd.DataFrame(
    [{"team_id": t["id"],
      "display_name": t["display_name"],
      "github_handles": ", ".join(t["github_handles"])}
     for t in teams]
)
```

	team_id	display_name	github_handles
0	a_team	Das A Team	obecher, Math1s0, JannhTH, MarkDT551, Kradid655
1	syntaxerror	Syntaxerror	lriethk, timmyhaeger, linus-thk
2	team_fabinalii	Team Fabinalii	akiev, feith, zakharuk
3	team_4	Team 4 SpotOptim	bartzbeielstein
4	team_4_optuna	Team 4 Optuna	bartzbeielstein
5	entsoe	Entso-E	bartzbeielstein
6	neura	Team Neura	Shervinfmz, Mayank07-Git, ravishankarbhojani12...
7	hot_rod	Hot Rod	soheilayati, msteffge, erikbirkholz, hennioo
8	ddkast	DDKAST	bachmak, KareemYassen, INDYROD
9	eigen_squad	Eigen-Squad	Slakshmi-m, kpnair99, hareemmustafa, SereenaJe...
10	team_gladiators	Team Gladiators	gauravlokhande1100-prog

The `validate_authorship()` check in `scripts/validate_submission.py:89` rejects PRs whose author is not in the team's `github_handles` list, so your local `gh auth login` must be the same account.

13.4 Step-by-step student walkthrough

13.4.1 Prerequisites

Python 3.13 with uv for environment management, `git` and the GitHub CLI (`gh`) authenticated as your registered handle, a personal ENTSO-E Transparency Platform API key (free; request via *My Account Settings* → *Web API Security Token*), and about 1 GB of free disk for the cached interim ENTSO-E CSV.

13.4.2 Step 1: Clone the lecture and fork the leaderboard

Assume git@git-ce.rwth-aachen.de:spotseven-lab/ddmo-sose-26.git as the lecture repo URL. The forked leaderboard is where you write your submission CSV and open the PR; the lecture repo is where you run the pipeline and render this chapter.

```
mkdir -p ~/workspace && cd ~/workspace
git clone git@git-ce.rwth-aachen.de:spotseven-lab/ddmo-sose-26.git ddmo-sose
gh repo fork bartzbeielstein/challenge-leaderboard --clone --remote
```

The fork lets you push your submission branch without write access to the upstream. Your PR will come from `bartzbeielstein/challenge-leaderboard:submission/team_4_optuna-<YYYY-MM-DD>-<HHMMSS>` into `bartzbeielstein/challenge-leaderboard:main`.

13.4.3 Step 2: Install the pipeline

```
cd ~/workspace/ddmo-sose-26
uv sync
```

This installs `spotforecast2-safe>=13.4.1`, `spotforecast2>=2.2.2`, and the rest of the dependencies into a local `.venv/`. Quarto reads `QUARTO_PYTHON` to find this interpreter (see the lecture's `Makefile`).

13.4.4 Step 3: Export your ENTSO-E API key

```
export ENTSOE_API_KEY="<your-personal-key-from-transparency.entsoe.eu>"
```

The live cells below read the key from the environment, never from a hard-coded literal in the rendered HTML.

13.4.5 Step 4: Set your team id

This chapter uses the registered team `team_4_optuna` throughout. The `TEAM_ID` constant lives in the submission write cell at Section 13.5; edit it once if you adapt the chapter for a different team.

13.4.6 Step 5: Generate the forecast and write the CSV

Render this chapter (`make render` at the lecture root, or just `quarto render lecture/13_live_submission.qmd`). The render writes a real CSV at `~/workspace/challenge-leaderboard/submissions/team_4_optuna/<tomorrow>.csv`.

13.4.7 Step 6: Validate locally before pushing

```
cd ~/workspace/challenge-leaderboard
uv run python scripts/validate_submission.py \
    --path submissions/team_4_optuna/<YYYY-MM-DD>.csv \
    --pr-author bartzbeielstein
```

Exit code 0 means schema, deadline, and authorship all check out. The last live cell at Section 13.5 runs this same validator from inside the notebook so you see the green output before you ever push.

13.4.8 Step 7: Open the submission PR

```
cd ~/workspace/challenge-leaderboard
TARGET=<YYYY-MM-DD>            # e.g. tomorrow's UTC date
# Unique branch per attempt (avoids colliding with an earlier merged branch
# if you re-submit for the same target day).
BRANCH="submission/team_4_optuna-${TARGET}-$(date -u +%H%M%S)"
git switch -c "$BRANCH" main
git add "submissions/team_4_optuna/${TARGET}.csv"
git commit -m "submission(team_4_optuna): forecast for ${TARGET}"
git push -u origin "$BRANCH"
gh pr create --base main --repo bartzbeielstein/challenge-leaderboard \
    --head "$BRANCH" \
    --title "submission(team_4_optuna): forecast for ${TARGET}" \
    --body "Generated by the ddmo-sose-26 live submission chapter."
```

If validation passes, auto-merge squashes the submission onto `main`. The next morning at 07:00 UTC, `score-daily.yml` scores it against ENTSO-E final actuals and the leaderboard refreshes within a minute.

> **Deadline**
>
> Submissions for target day `D` must be merged before `D-1 23:59 UTC` — the validator enforces this strictly. The full pipeline (render → validate → push → PR → auto-merge) typically completes in under two minutes, but ENTSO-E API hiccups during the data refresh can occasionally cost a few more. Submit at least 30 minutes before the deadline to leave yourself recovery room.

13.5 Live example — generate ŷ end-to-end

The cells below are the *live* pipeline: every render of this chapter downloads today's ENTSO-E actuals, trains a fresh Optuna-tuned LightGBM forecaster on the available data, and writes a validated CSV into your forked leaderboard's submission slot for tomorrow. Optuna drives a Bayesian (TPE) search over the LightGBM hyperparameters; the other three approaches (Lazy, Defaults, SpotOptim) are introduced for comparison in chapters 07 through 10.

```
import logging, os
from pathlib import Path
import pandas as pd

logging.basicConfig(level=logging.WARNING)

# Isolate this chapter's storage from chapter 14 -----------------------------
# download_new_data() writes the merged ENTSO-E mirror to
# <SPOTFORECAST2_DATA>/interim/energy_load.csv, and MultiTask persists tuned
# models, tuning results, and logs under <SPOTFORECAST2_CACHE>. Both default to
# a single shared location in $HOME, so chapters 13 and 14 would otherwise read
# and write the *same* interim download. Pinning per-chapter homes here - before
# any download or MultiTask call - gives this chapter its own download, cache,
# and log directory. The package reads both env vars at call time via
```

```
# get_data_home() / get_cache_home(), so setting them now is sufficient.
os.environ["SPOTFORECAST2_DATA"]  = str(Path.home() / "spotforecast2_data"  /
os.environ["SPOTFORECAST2_CACHE"] = str(Path.home() / ".spotforecast2_cache"

from spotforecast2_safe.data import Period
from spotforecast2_safe.downloader.entsoe import download_new_data
from spotforecast2_safe.data.fetch_data import get_data_home
from spotforecast2_safe.configurator.config_entsoe import ConfigEntsoe
from spotforecast2.tasks.task_entsoe import (
    entsoe_data_loader,
    entsoe_test_data_loader,
    entsoe_lgbm_factory,
)
from spotforecast2.multitask import MultiTask
```

13.5.1 Dynamic date constants

Every date is derived from `pd.Timestamp.now(tz="UTC").normalize()` so the chapter stays correct on every render. The convention follows the canonical `bart26b/CLAUDE.md` *Date-range rule*:

- `y_{-1}` = yesterday's 24 hourly DE Actual Load values (UTC day `D-1`)
- `ŷ_{-1}` = the model's prediction for the same window (used by chapters 08–10 as the scorable exemplar)
- `y_0` = today's 24 h (UTC day `D`); the *real* live target — not scorable until `~D+1 06:00 UTC` when ENTSO-E publishes final actuals

```
NOW_UTC = pd.Timestamp.now(tz="UTC")
TODAY_UTC = NOW_UTC.normalize()
YESTERDAY_UTC = TODAY_UTC - pd.Timedelta(days=1)
TOMORROW_UTC = TODAY_UTC + pd.Timedelta(days=1)
```

```
START_DOWNLOAD = "202201010000"
END_DOWNLOAD = NOW_UTC.strftime("%Y%m%d%H%M")   # exclusive upper bound = now (UTC) → includ

print(f"NOW_UTC       = {NOW_UTC}")
print(f"TODAY_UTC     = {TODAY_UTC}")
print(f"TOMORROW_UTC  = {TOMORROW_UTC}  (target day = y_0)")
print(f"END_DOWNLOAD  = {END_DOWNLOAD}  (ENTSO-E fetch cutoff = now, UTC)")
```

```
NOW_UTC       = 2026-05-31 20:08:58.326983+00:00
TODAY_UTC     = 2026-05-31 00:00:00+00:00
TOMORROW_UTC  = 2026-06-01 00:00:00+00:00  (target day = y_0)
END_DOWNLOAD  = 202605312008  (ENTSO-E fetch cutoff = now, UTC)
```

13.5.2 Download today's ENTSO-E actuals

`download_new_data` is incremental: subsequent calls only fetch ranges not already on disk. After the call, we verify coverage and abort loudly if the merged interim CSV does not extend through `D-1 23:00 UTC` — a stale download would silently shorten the prediction window.

```
api_key = os.environ.get("ENTSOE_API_KEY")
assert api_key, "ENTSOE_API_KEY must be set in the environment (see Step 3 above)."

download_new_data(
    api_key=api_key,
    country_code="DE",
    start=START_DOWNLOAD,
    end=END_DOWNLOAD,
    force=False,
)

interim_csv = get_data_home() / "interim" / "energy_load.csv"
```

```
interim = pd.read_csv(interim_csv, index_col=0, parse_dates=True)
interim.index = pd.to_datetime(interim.index, utc=True)

required_last = TODAY_UTC - pd.Timedelta(hours=1)
if interim.index.max() < required_last:
    raise RuntimeError(
        f"ENTSO-E coverage is stale: last interim row is {interim.index.max()
        f"but the chapter requires at least {required_last}. "
        "Re-run download_new_data(..., force=True) or check the ENTSO-E API."
    )
print(f"interim CSV last row: {interim.index.max()}")
print(f"Actual Load tail (last 6 quarter-hours):")
print(interim["Actual Load"].dropna().tail(6))
```

```
interim CSV last row: 2026-05-31 19:45:00+00:00
Actual Load tail (last 6 quarter-hours):
Time (UTC)
2026-05-31 18:00:00+00:00    47815.13431
2026-05-31 18:15:00+00:00    47346.56433
2026-05-31 18:30:00+00:00    47349.94149
2026-05-31 18:45:00+00:00    47007.55604
2026-05-31 19:00:00+00:00    43577.29066
2026-05-31 19:15:00+00:00    43384.91911
Name: Actual Load, dtype: float64
```

13.5.3 Determine the live training cutoff and forecast horizon

At submission time the ENTSO-E intra-day actuals usually cover only part of today, so we cannot anchor `end_train_default` at today 23:00 UTC. Instead we anchor at the latest hour for which `Actual Load` is non-NaN, floor it to a clean hour boundary, and compute how many hourly forecast steps we need to reach the

end of tomorrow. After training we slice tomorrow's 24 hourly steps out of the longer forecast — those are the `y_0` rows we submit.

```
LAST_TARGET = TOMORROW_UTC + pd.Timedelta(hours=23)
LAST_FULL_HOUR = interim["Actual Load"].dropna().index.max().floor("h")
FIRST_PRED = LAST_FULL_HOUR + pd.Timedelta(hours=1)
LIVE_N_STEPS = int((LAST_TARGET - FIRST_PRED).total_seconds() // 3600) + 1

print(f"end_train (inclusive): {LAST_FULL_HOUR}")
print(f"first forecast step  : {FIRST_PRED}")
print(f"last forecast step   : {LAST_TARGET}")
print(f"predict_size         : {LIVE_N_STEPS}  (slice last 24 for y_0)")
```

```
end_train (inclusive): 2026-05-31 19:00:00+00:00
first forecast step  : 2026-05-31 20:00:00+00:00
last forecast step   : 2026-06-01 23:00:00+00:00
predict_size         : 28  (slice last 24 for y_0)
```

Why is `predict_size` usually larger than 24?

`predict_size` is **not** "how many hours we want to submit." It is the number of hourly steps the recursive forecaster must produce, starting at the first hour *after* the last known actual, until it reaches the end of tomorrow. The 24 rows we submit are then *sliced out of* that longer forecast.
The math, step by step:

$$\texttt{FIRST_PRED} = \texttt{LAST_FULL_HOUR} + 1\,\text{h}$$
$$\texttt{LAST_TARGET} = \texttt{TOMORROW_UTC} + 23\,\text{h}$$
$$\texttt{LIVE_N_STEPS} = \frac{\texttt{LAST_TARGET} - \texttt{FIRST_PRED}}{1\,\text{h}} + 1$$

The `+ 1` is because both endpoints are inclusive (we need the hour

FIRST_PRED *and* the hour LAST_TARGET).
Decomposing the horizon into two segments makes it obvious where the extra steps come from:

$$\underbrace{\texttt{LIVE_N_STEPS}}_{\text{total recursion}} = \underbrace{(24 - \texttt{LAST_FULL_HOUR.hour} - 1)}_{\text{bridge: rest of today}} + \underbrace{24}_{y_0\text{: tomorrow}}$$

A worked example. Suppose the chapter is rendered at 18:30 UTC and ENTSO-E's intra-day feed currently publishes `Actual Load` through **17:00 UTC** today. Then:

- `LAST_FULL_HOUR` = today 17:00 UTC
- `FIRST_PRED` = today 18:00 UTC
- `LAST_TARGET` = tomorrow 23:00 UTC
- bridge hours = `24 - 17 - 1 = 6` (today 18:00, 19:00, 20:00, 21:00, 22:00, 23:00)
- target hours = 24 (tomorrow 00:00 … 23:00)
- `LIVE_N_STEPS` = `6 + 24 = 30`

Why can't we just ask the model for 24 steps and start at tomorrow 00:00? Because the LightGBM forecaster behind `entsoe_lgbm_factory` is **recursive**: its feature row for hour $h+1$ uses the lag-1 value $\hat{y}_h$ that it produced one step earlier (together with lag-2, lag-24, calendar, and weather features). It cannot "jump" over the 6 bridge hours; each prediction is built from the chain of preceding predictions. The bridge hours are scaffolding the recursion needs to reach tomorrow — once it has, the next cell slices them off:

```
y0 = future.loc[TOMORROW_UTC:LAST_TARGET]   # exactly 24 rows
```

So `LIVE_N_STEPS` floats with how fresh ENTSO-E's feed happens to be at render time:

Render time (UTC)	`LAST_FULL_HOUR`	Bridge hours	`LIVE_N_STEPS`
00:30 (feed current through 23:00 of *previous* day)	yesterday 23:00	24 (all of today)	48
12:30 (feed current through 11:00)	today 11:00	12	36
18:30 (feed current through 17:00)	today 17:00	6	30
23:30 (feed current through 22:00)	today 22:00	1	25
just after midnight (feed already current through 23:00 of *previous* day)	yesterday 23:00	24	48

The submission CSV still has exactly 24 rows in every case — only the amount of throwaway bridge forecasting changes.

13.5.4 Build the live `ConfigEntsoe`

All inputs to the pipeline travel on a single configuration object. The hooks (`data_loader`, `test_data_loader`, `forecaster_factory`) are the same ones the bart26b publication uses. The training-window parameters and tuning budgets follow the canonical defaults. The `ConfigEntsoe`

documentation is at https://sequential-parameter-optimization.github.io/spotforecast2-safe/docs/reference/configurator.config_entsoe.ConfigEntsoe.html

We pass `cv_block_size=PREDICT_SIZE` (= 24) so the rolling cross-validation scores a fixed 24-hour-ahead block, decoupled from the render-dependent live horizon `predict_size=LIVE_N_STEPS`. Chapter 14 derives why this matters at Section 14.6.2.

```
TRAIN_SIZE = pd.Timedelta(days=365 * 3)
PREDICT_SIZE = 24
REFIT_SIZE = 7
NUMBER_FOLDS = 10
DELTA_VAL = pd.Timedelta(hours=PREDICT_SIZE * REFIT_SIZE * NUMBER_FOLDS)

live_config = ConfigEntsoe(
    country_code="DE",
    data_filename="interim/energy_load.csv",
    targets=["Actual Load"],
    agg_weights=[1.0],
    bounds=[(-1e9, 1e9)],
    data_loader=entsoe_data_loader,
    test_data_loader=entsoe_test_data_loader,
    forecaster_factory=entsoe_lgbm_factory,
    periods=[Period(name='daily', n_periods=12, column='hour', input_range=(
    lags_consider=[1, 2, 24],
    train_size=TRAIN_SIZE,
    end_train_default=LAST_FULL_HOUR.isoformat(),
    delta_val=DELTA_VAL,
    predict_size=LIVE_N_STEPS,
    cv_block_size=PREDICT_SIZE,
    refit_size=REFIT_SIZE,
    n_trials_optuna=30,
    n_trials_spotoptim=7,
    n_initial_spotoptim=5,
```

```
    random_state=42,
    on_weather_failure="skip",
)
live_config.data_frame_name = "ddmo-live"
print("live_config ready:", live_config.data_frame_name,
      "predict_size=", live_config.predict_size,
      "end_train_default=", live_config.end_train_default)
```

```
live_config ready: ddmo-live predict_size= 28 end_train_default= 2026-
05-31T19:00:00+00:00
```

13.5.5 Train with Optuna and predict forward through tomorrow

The MultiTask orchestrator runs the full pipeline: prepare data, detect outliers, impute, build exogenous (calendar / day-night / weather / holiday) features, tune hyperparameters with Optuna, fit the final forecaster, and return a *prediction package*. The raw forecast covers `[LAST_FULL_HOUR + 1 h, LAST_TARGET]`; we then slice the 24 hourly rows for tomorrow.

```
live_mt = MultiTask(live_config, task="optuna")
live_mt.prepare_data()
live_mt.detect_outliers()
live_mt.impute()
live_mt.build_exogenous_features()
live_mt.run(show=False)

future = live_mt.results["optuna"]["Actual Load"]["future_pred"]
print(f"raw forecast: {len(future)} hourly steps "
      f"| {future.index.min()} → {future.index.max()}")

y0 = future.loc[TOMORROW_UTC:LAST_TARGET]
```

```
print(f"y_0 slice   : {len(y0)} hourly steps "
      f"| {y0.index.min()} → {y0.index.max()}")
```

```
INFO:spotforecast2.multitask.base.MultiTask:Pipeline data shape after prepara
INFO:spotforecast2.multitask.base.MultiTask:Forecasted Load: 362 outliers aut
INFO:spotforecast2.multitask.base.MultiTask:Actual Load: 371 outliers automat
INFO:spotforecast2.multitask.base.MultiTask:Outlier detection complete.
INFO:spotforecast2.multitask.base.MultiTask:apply_imputation: NaN cells befor
INFO:spotforecast2.multitask.base.MultiTask:Applying weighted imputation (n2n
INFO:spotforecast2.multitask.base.MultiTask:Weight function created with 3868
INFO:spotforecast2.multitask.base.MultiTask:apply_imputation: NaN cells after
INFO:spotforecast2.multitask.base.MultiTask:Imputation complete (method=weigh
INFO:spotforecast2.multitask.base.MultiTask:Building exogenous features...
INFO:spotforecast2.multitask.base.MultiTask:  Weather features: (38712, 105)
INFO:spotforecast2.multitask.base.MultiTask:  Calendar features: (38712, 4)
INFO:spotforecast2.multitask.base.MultiTask:  Day/night features: (38712, 4)
INFO:spotforecast2.multitask.base.MultiTask:  Holiday features: (38712, 1)
INFO:spotforecast2.multitask.base.MultiTask:Combined exogenous features shape
INFO:spotforecast2.multitask.base.MultiTask:Selected 27 exogenous features fo
INFO:spotforecast2.multitask.base.MultiTask:Merged data shape: (38684, 28)
INFO:spotforecast2.multitask.base.MultiTask:Exogenous prediction covariates s
INFO:spotforecast2.multitask.base.MultiTask:Training window: 2023-
06-01 19:00:00+00:00 to 2026-05-31 19:00:00+00:00
INFO:spotforecast2.multitask.base.MultiTask:[task 3] Target 'Actual Load': ta
INFO:spotforecast2.multitask.base.MultiTask:  Best params: {'estimator__num_l
INFO:spotforecast2.multitask.base.MultiTask:  Best lags: [ 1  2  3  4  5  6
 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48]
INFO:spotforecast2.multitask.base.MultiTask:  Saved tuning results: /Users/ba
live_Actual Load_optuna_20260531_201913.json
INFO:spotforecast2.multitask.base.MultiTask:  Saved model: /Users/bartz/.spot
live/ddmo-live_Actual Load_optuna_20260531_201953.joblib

raw forecast: 28 hourly steps | 2026-05-31 20:00:00+00:00 → 2026-
```

```
06-01 23:00:00+00:00
y_0 slice   : 24 hourly steps | 2026-06-01 00:00:00+00:00 → 2026-
06-01 23:00:00+00:00
```

13.5.6 Visual Inspection

Before writing the CSV, plot the prediction package with `spotforecast2.plots.plotter.make_plot`. The figure overlays the actual ENTSO-E load (training window plus whatever final actuals are already available), the model's in-sample fit, the out-of-sample forecast through tomorrow, and a last-week shift for weekly-seasonality context. The vertical dashed line at `end_train` separates the historical region from the live forecast — the 24 rows we are about to submit sit to the right of it.

```
from spotforecast2.plots.plotter import make_plot

pkg = live_mt.results["optuna"]["Actual Load"]
print("future_pred :", len(pkg["future_pred"]), "rows,",
      pkg["future_pred"].index.min(), "→", pkg["future_pred"].index.max())
print("metrics_future         :", pkg.get("metrics_future"))
print("metrics_future_one_day :", pkg.get("metrics_future_one_day"))

fig = make_plot(pkg)
fig.show()
```

```
future_pred : 28 rows, 2026-05-31 20:00:00+00:00 → 2026-06-
01 23:00:00+00:00
metrics_future         : {}
metrics_future_one_day : {}
```

```
Unable to display output for mime type(s): text/html
```

(a) Live ENTSO-E DE load forecast — training fit (left of the dashed line) and the recursive forecast through tomorrow 23:00 UTC (right). The 24 hourly values for `y_0` are the rightmost block.

```
Unable to display output for mime type(s): text/html
```

(b)

Figure 13.1

13.5.7 Validate the schema and write the submission CSV

Four contract assertions guard the write: 24 rows, the first stamp must be `TOMORROW 00:00 UTC`, every value strictly positive, no NaN. Any violation aborts before the file is opened, so a partially-written CSV can never reach the leaderboard. After the assertions, the cell writes the file into your local clone of the leaderboard fork.

```
assert len(y0) == 24,           f"expected 24 hourly steps for y_0, got {len
assert y0.index.min() == TOMORROW_UTC, (
    f"first step {y0.index.min()} != TOMORROW {TOMORROW_UTC}"
)
assert (y0 > 0).all(),          "non-positive forecast value - spec requires
assert y0.notna().all(),        "NaN in forecast - spec forbids"

TEAM_ID = "team_4_optuna"       # registered team; edit once if you adapt thi
SUBMISSION_DIR = Path.home() / "workspace" / "challenge-leaderboard" / "submi
SUBMISSION_DIR.mkdir(parents=True, exist_ok=True)
submission_path = SUBMISSION_DIR / f"{TOMORROW_UTC.date().isoformat()}.csv"

submission_df = pd.DataFrame({
    "timestamp_utc": y0.index.strftime("%Y-%m-%dT%H:%M:%SZ"),
```

```
    "forecast_mw":   y0.round(2).values,
})
submission_df.to_csv(submission_path, index=False)
print(f"wrote {submission_path}")
print(f"  rows: {len(submission_df)}")
print(f"  range: {submission_df.forecast_mw.min():.1f}-{submission_df.forecast_mw.max():.1f}
submission_df.head(3)
```

```
wrote /Users/bartz/workspace/challenge-leaderboard/submissions/team_4_optuna/2026-
06-01.csv
  rows: 24
  range: 36892.5-56005.6 MW
```

	timestamp_utc	forecast_mw
0	2026-06-01T00:00:00Z	37032.63
1	2026-06-01T01:00:00Z	36892.50
2	2026-06-01T02:00:00Z	37582.75

13.5.8 Local validation with the leaderboard's own script

A green exit code from `validate_submission.py` is the same check that runs on the PR in CI. We pass `--skip-deadline` because the chapter may be rendered inside the deadline window (the live CI uses the strict form without the flag).

```
import subprocess
lb_repo = Path.home() / "workspace" / "challenge-leaderboard"
result = subprocess.run(
    [
        "uv", "run", "python", "scripts/validate_submission.py",
        "--path", str(submission_path.relative_to(lb_repo)),
```

```
        "--skip-deadline",
    ],
    cwd=lb_repo,
    capture_output=True,
    text=True,
)
print("exit code:", result.returncode)
print("stdout   :", result.stdout.strip())
if result.stderr.strip():
    print("stderr   :", result.stderr.strip())
```

```
exit code: 0
stdout   : OK: team=team_4_optuna target_date=2026-06-
01 file=submissions/team_4_optuna/2026-06-01.csv
```

13.6 What the render did NOT do

The cells above only write the CSV. They do not commit, push, or open a PR — those are deliberate operator steps so a careless render cannot spam the leaderboard. The bash recipe in Section 13.4 step 7 is the manual hand-off; once the PR is opened, the leaderboard's CI (see Section 13.7) takes over.

13.7 Tomorrow's scoring

The leaderboard's daily cron is configured at `.github/workflows/score-daily.yml:5`:

```
cron: "0 7 * * *"
```

That is 07:00 UTC every day. The cron fetches ENTSO-E final actuals for yesterday (= the target day of the submission you just opened), computes per-team MAE, opens a bot PR that updates `data/scores.parquet`, and triggers `build-and-deploy.yml` which republishes https://bartzbeielstein.github.io/challenge-leaderboard/. End-to-end latency from your PR merge to a refreshed leaderboard row is typically under 90 seconds after `D+1 07:00 UTC`.

Exercises (with Claude)

1. *Swap in your own model.* Replace `forecaster_factory=entsoe_lgbm_factory` in the live config with a custom factory that returns a `ForecasterRecursive(estimator=…)` of your choice (XGBoost, Prophet, a naïve last-week baseline). Anything that produces 24 strictly positive hourly values for the target UTC day will score.
2. *Tighten the tuning budget.* Change `n_trials_optuna` from 15 to 3, re-render, and see whether the resulting forecast deviates noticeably. Discuss with Claude how a small budget can still beat defaults if the search space is well-shaped.
3. *Read the failure modes.* Look up the S1–S7 failure-mode taxonomy in the bart26b publication (table after `@sec-exo-failure-modes`). Which scenario applies if Open-Meteo is unreachable during the cell above? Trace what happens to your CSV in that case.

14 Team 4 live submission — advanced setup

> **Learning objective**
>
> You can render `team_4`'s signed-off live submission: a 24-hour DE load forecast produced by a richer feature/lag setup than the generic baseline in Chapter 13. The chapter writes the validated CSV into the leaderboard fork and documents the exact `git` / `gh` commands `team_4` uses to open the submission PR.

14.1 Overview

Chapter 13 is a *generic* template: any team can render it, substitute a placeholder, and submit. This chapter is *team_4's tuned variant*. Every safety contract from chapter 13 carries over (deadline check, 24-row schema, strictly-positive forecasts, fail-loud ENTSO-E coverage assertion); only the modelling configuration changes.

What this chapter changes relative to Chapter 13:

- All five seasonal `Periods` instead of `daily` alone — weekly, monthly, quarterly, and yearly RBF encodings join the daily one.
- Candidate lags chosen *dynamically* from the data — the PACF-significant lags (1) rather than the baseline's hard-coded `[1, 2, 24]`.

- `include_weather_windows=True`, `include_holiday_features=True`, `poly_features_degree=2` — temperature, DE/NW public holidays, and bilinear calendar-by-weather interactions enter the exogenous matrix. The interactions are capped at `max_poly_features=10`, keeping only the ten most informative ones (by mutual information).
- SpotOptim budget kept small for a fast live render — 6 / 5 (trials / initial evaluations) with `number_folds=10`. This is the one knob still dialled down; raising it deepens the search at the cost of render time.

14.2 Submission contract

Unchanged — see Section 13.2 for the CSV schema, deadline, and scoring rules. The leaderboard validator that runs against this chapter's output is byte-identical to the one used by Chapter 13.

14.3 Team identity

`team_4` is hardcoded throughout this chapter, there is no placeholder to edit. The cell below confirms the team is registered in `challenge-leaderboard/teams.yml`.

```
from pathlib import Path
import yaml

teams_yml = Path.home() / "workspace" / "challenge-leaderboard" / "teams.yml"
teams = yaml.safe_load(teams_yml.read_text())["teams"]
team4 = next((t for t in teams if t["id"] == "team_4"), None)
assert team4 is not None, "team_4 is not in challenge-leaderboard/teams.yml"
print(f"team_4 display_name : {team4['display_name']}")
print(f"team_4 github       : {', '.join(team4['github_handles'])}")
```

```
team_4 display_name : Team 4 SpotOptim
team_4 github       : bartzbeielstein
```

`validate_authorship()` in `scripts/validate_submission.py:89` rejects PRs whose author is not in this `github_handles` list, so the `gh auth login` account used in Section 14.7 must match.

14.4 Prerequisites and walkthrough

Identical to Section 13.4: Python 3.13 + uv, a personal ENTSO-E API key exported as `ENTSOE_API_KEY`, `gh auth login` as a registered `team_4` handle, and a sibling clone of `challenge-leaderboard` at `~/workspace/challenge-leaderboard/`.

The only difference from the generic walkthrough is that `TEAM_ID = "team_4"` is already baked into the submission cell at Section 14.6.12 — nothing to edit.

14.5 Live pipeline

The cells below are the *live* team_4 pipeline. Every render downloads today's ENTSO-E actuals, tunes an LGBM forecaster with the advanced configuration, and writes a validated CSV for tomorrow.

```
import logging, os
from pathlib import Path
import pandas as pd

logging.basicConfig(level=logging.WARNING)

# Isolate this chapter's storage from chapter 13 ------------------------------
# download_new_data() writes the merged ENTSO-E mirror to
```

```
# <SPOTFORECAST2_DATA>/interim/energy_load.csv, and MultiTask persists tuned
# models, tuning results, and logs under <SPOTFORECAST2_CACHE>. Both default
# a single shared location in $HOME, so chapters 13 and 14 would otherwise re
# and write the *same* interim download. Pinning per-chapter homes here - bei
# any download or MultiTask call - gives this chapter its own download, cache
# and log directory. The package reads both env vars at call time via
# get_data_home() / get_cache_home(), so setting them now is sufficient.
os.environ["SPOTFORECAST2_DATA"]  = str(Path.home() / "spotforecast2_data"  /
os.environ["SPOTFORECAST2_CACHE"] = str(Path.home() / ".spotforecast2_cache"

from spotforecast2_safe.data import Period
from spotforecast2_safe.downloader.entsoe import download_new_data
from spotforecast2_safe.data.fetch_data import get_data_home
from spotforecast2_safe.configurator.config_entsoe import ConfigEntsoe
from spotforecast2.tasks.task_entsoe import (
    entsoe_data_loader,
    entsoe_test_data_loader,
    entsoe_lgbm_factory,
)
from spotforecast2.multitask import MultiTask
```

14.5.1 Dynamic date constants

Every date is derived from `pd.Timestamp.now(tz="UTC").normalize()` so the chapter stays correct on every render — same convention as Section 13.5.

```
NOW_UTC = pd.Timestamp.now(tz="UTC")
TODAY_UTC = NOW_UTC.normalize()
YESTERDAY_UTC = TODAY_UTC - pd.Timedelta(days=1)
TOMORROW_UTC = TODAY_UTC + pd.Timedelta(days=1)

START_DOWNLOAD = "202201010000"
```

```
# Fetch through *tomorrow* so the interim file lands two things: (a) the most
# current ENTSO-E actuals already published today --- which shrinks the
# recursive bridge and horizon --- and (b) ENTSO-E's own day-ahead
# `Forecasted Load` for the target day, used only for the comparison plot in
# @sec-team4-vs-entsoe (the model never sees it). Future timestamps simply
# carry no Actual Load yet, so the training cutoff and horizon are unchanged.
END_DOWNLOAD = (TOMORROW_UTC + pd.Timedelta(days=1)).strftime("%Y%m%d%H%M")

print(f"NOW_UTC       = {NOW_UTC}")
print(f"TODAY_UTC     = {TODAY_UTC}")
print(f"TOMORROW_UTC  = {TOMORROW_UTC}  (target day = y_0)")
print(f"END_DOWNLOAD  = {END_DOWNLOAD}  (covers tomorrow for ENTSO-E's day-ahead forecast)")
```

```
NOW_UTC       = 2026-05-31 19:37:53.771067+00:00
TODAY_UTC     = 2026-05-31 00:00:00+00:00
TOMORROW_UTC  = 2026-06-01 00:00:00+00:00  (target day = y_0)
END_DOWNLOAD  = 202606020000  (covers tomorrow for ENTSO-
E's day-ahead forecast)
```

14.5.2 Download today's ENTSO-E actuals and ENTSO-E's own day-ahead `Forecasted Load`

```
api_key = os.environ.get("ENTSOE_API_KEY")
assert api_key, "ENTSOE_API_KEY must be set in the environment."

download_new_data(
    api_key=api_key,
    country_code="DE",
    start=START_DOWNLOAD,
    end=END_DOWNLOAD,
    force=True,
```

```
    keep_forecast_future=True,
)
```

`keep_forecast_future=True` is the one argument that differs from the training chapters' download. By default `download_new_data` drops every row after `now` when it builds the interim file — the leakage-free input a model must train on. That default would also erase ENTSO-E's day-ahead `Forecasted Load` for the target day, leaving the baseline comparison in Section 14.6.11 with nothing to plot. The flag keeps those future rows, so tomorrow's published forecast survives; the future `Actual Load` is still blank, so no future target leaks into training (the leakage guard in Section 14.6.9 re-checks this).

14.5.3 The interim energy-load frame

`download_new_data` merges every ENTSO-E pull into one tidy file, `interim/energy_load.csv`, under the package data home (`get_data_home()`). That file is the single source the rest of the chapter reads from. Its shape is worth knowing before the cells below slice it:

- Index — `Time (UTC)` parsed to a timezone-aware `DatetimeIndex`, stepping in 15-minute (quarter-hour) increments. It runs from `START_DOWNLOAD` (`2022-01-01 00:00 UTC`) through `END_DOWNLOAD` (the day after tomorrow), so the frame deliberately extends past *now* into the target day.
- `Actual Load` — ENTSO-E's German *Actual Total Load* in MW, the forecaster's target. It is populated only up to the last quarter-hour ENTSO-E has actually published; every timestamp after that — including all of the target day — is blank (`NaN`). The last non-blank stamp is what Section 14.5.6 turns into `end_train`.
- `Forecasted Load` — ENTSO-E's own day-ahead load forecast in MW, published around midday `D-1`. Because it is a forecast, it covers the future hours `Actual Load` does not, which is exactly why `END_DOWNLOAD` reaches

into tomorrow. The model never trains on this column (the leakage guard in Section 14.6.9 enforces that); it serves only as the independent baseline in Section 14.6.11 and the overlay in Figure 14.2.

- `Actual` — a legacy ENTSO-E column carried through the download verbatim; it is empty for DE load and the pipeline ignores it.

The cell below loads the file, coerces the index to UTC, and runs two fail-loud coverage checks. The first guards the frame's *index*: it must reach at least `D-1 23:00 UTC` so the exogenous features and the forecast bridge have somewhere to land. That index is kept fresh by `Forecasted Load` (which extends into the future), so it does *not* speak to the target's freshness. The second check therefore guards the target directly. It cannot demand the actuals reach `D-1 23:00`, because ENTSO-E publishes the German actual load with a normal lag of a few hours — early in the UTC day the actuals legitimately trail that bar. Instead it allows the last *published* `Actual Load` stamp to lag `now` by up to `MAX_ACTUAL_LAG` (24 h here, easily raised), failing only when the feed is genuinely stale — the index looking current while the training history quietly ended a day or more early.

```
interim_csv = get_data_home() / "interim" / "energy_load.csv"
interim = pd.read_csv(interim_csv, index_col=0, parse_dates=True)
interim.index = pd.to_datetime(interim.index, utc=True)

required_last = TODAY_UTC - pd.Timedelta(hours=1)
if interim.index.max() < required_last:
    raise RuntimeError(
        f"ENTSO-E coverage is stale: last interim row is {interim.index.max()} "
        f"but the chapter requires at least {required_last}. "
        "Re-run download_new_data(..., force=True) or check the ENTSO-E API."
    )

# ENTSO-E publishes the German actual load with a normal lag of a few hours,
# so the actuals legitimately trail `now` (especially early in the UTC day).
# Guard only against a genuinely stale feed: the last published actual must lie
# within MAX_ACTUAL_LAG of now. Raise the tolerance if your feed lags more.
```

```
MAX_ACTUAL_LAG = pd.Timedelta(hours=36)
last_actual = interim["Actual Load"].dropna().index.max()
if last_actual < NOW_UTC - MAX_ACTUAL_LAG:
    lag_h = int((NOW_UTC - last_actual) / pd.Timedelta(hours=1))
    raise RuntimeError(
        f"Actual Load is stale: last published actual is {last_actual}, "
        f"{lag_h} h before now ({NOW_UTC}); tolerance is "
        f"{int(MAX_ACTUAL_LAG / pd.Timedelta(hours=1))} h. "
        "Re-run download_new_data(..., force=True) or wait for ENTSO-E to "
        "publish more recent actuals."
    )
print(f"interim CSV last row: {interim.index.max()}")
print(interim["Actual Load"].dropna().tail(6))
```

```
interim CSV last row: 2026-06-01 21:45:00+00:00
Time (UTC)
2026-05-31 17:30:00+00:00    47723.15097
2026-05-31 17:45:00+00:00    47780.42240
2026-05-31 18:00:00+00:00    47824.03431
2026-05-31 18:15:00+00:00    47354.53933
2026-05-31 18:30:00+00:00    47359.06649
2026-05-31 18:45:00+00:00    47026.90704
Name: Actual Load, dtype: float64
```

14.5.4 Extract ENTSO-E's day-ahead `Forecasted Load`

ENTSO-E's own day-ahead forecast is used twice downstream — as the overlay in Figure 14.1 and as the scored baseline in Section 14.6.11 — so we pull it out of the `interim` frame once, here, and keep it as a standalone Series. It is resampled onto the hourly grid (the mean of each hour's four quarter-hour values, matching how `prepare_data()` aggregates the actuals in Section 14.6.3), so it lines up directly with the forecaster's hourly output for error computations. The trailing

`.dropna()` keeps only the hours ENTSO-E has actually published a forecast for. The model never trains on these values — the leakage guard in Section 14.6.9 enforces that; `predictions_entsoe` is an independent baseline only.

```
predictions_entsoe = interim["Forecasted Load"].resample("h").mean().dropna()
predictions_entsoe.name = "Forecasted Load"

print(f"predictions_entsoe: {len(predictions_entsoe)} hourly values "
      f"| {predictions_entsoe.index.min()} → {predictions_entsoe.index.max()}")
print(predictions_entsoe.tail(6))
```

```
predictions_entsoe: 38710 hourly values | 2022-01-01 00:00:00+00:00 → 2026-
06-01 21:00:00+00:00
Time (UTC)
2026-06-01 16:00:00+00:00    58422.032589
2026-06-01 17:00:00+00:00    58017.811127
2026-06-01 18:00:00+00:00    56567.745495
2026-06-01 19:00:00+00:00    53892.870363
2026-06-01 20:00:00+00:00    50815.118195
2026-06-01 21:00:00+00:00    48307.254943
Freq: h, Name: Forecasted Load, dtype: float64
```

14.5.5 Native resolution: 15-minute actuals aggregated to hourly

ENTSO-E publishes the German *Actual Total Load* at 15-minute (quarter-hour) resolution, and `download_new_data` stores it verbatim — the `interim/energy_load.csv` index steps in 15-minute increments, not hourly. The hourly grid the forecaster trains on is produced later, inside `prepare_data()` (Section 14.6.3): it calls `agg_and_resample_data(rule="h", by="mean")` (in `spotforecast2_safe.preprocessing.curate_data`), which replaces the four quarter-hour values in each hour with their mean. The mean — not the

sum — is the correct aggregation because load is an average power level over the interval, not an accumulated quantity. The cell below confirms the native cadence of the freshly downloaded `interim` frame.

```
native_step = interim.index.to_series().diff().dropna().mode().iloc[0]
print(f"interim rows           : {len(interim)}")
print(f"native cadence (modal) : {native_step}   <- 15-min ENTSO-E DE Actual
print(interim[["Actual Load"]].dropna().head(4))
```

```
interim rows           : 154836
native cadence (modal) : 0 days 00:15:00   <- 15-min ENTSO-
E DE Actual Load
                           Actual Load
Time (UTC)
2022-01-01 00:00:00+00:00     42195.81
2022-01-01 00:15:00+00:00     41803.94
2022-01-01 00:30:00+00:00     41140.73
2022-01-01 00:45:00+00:00     41002.15
```

14.5.6 Training cutoff and forecast horizon

Identical mechanics to Section 13.5: anchor `end_train` at the last full hour of `Actual Load`, then forecast forward through tomorrow 23:00 UTC and slice the 24 rows we submit.

```
LAST_TARGET = TOMORROW_UTC + pd.Timedelta(hours=23)
LAST_FULL_HOUR = interim["Actual Load"].dropna().index.max().floor("h")
FIRST_PRED = LAST_FULL_HOUR + pd.Timedelta(hours=1)
LIVE_N_STEPS = int((LAST_TARGET - FIRST_PRED).total_seconds() // 3600) + 1

print(f"end_train (inclusive): {LAST_FULL_HOUR}")
print(f"first forecast step  : {FIRST_PRED}")
```

```
print(f"last forecast step   : {LAST_TARGET}")
print(f"predict_size         : {LIVE_N_STEPS}  (slice last 24 for y_0)")
```

```
end_train (inclusive): 2026-05-31 18:00:00+00:00
first forecast step  : 2026-05-31 19:00:00+00:00
last forecast step   : 2026-06-01 23:00:00+00:00
predict_size         : 29  (slice last 24 for y_0)
```

14.5.7 Data availability vs. the requested interval

It is tempting to assume that at render time `now`, the `Actual Load` series reaches `now - 1 h` — the last completed hour. ENTSO-E does not work that way: the German actual total load is published with a lag, so the last *observed* hour `y_e` can sit well before `now - 1 h`. The span between them is publication lag, not missing data, and `y_e` is exactly the `end_train` cutoff fixed in Section 14.5.6.

This matters because of how the frame is built. `download_new_data` extends the merged index into the future via `Forecasted Load`, so `interim` runs to roughly `now`, while `Actual Load` stops at `y_e`. When `prepare_data()` (Section 14.6.3) resamples to an hourly grid it spans that full index, so the hours after `y_e` carry no target and show up as `NaN`. They are *not* gaps in the historical record — they are the recent tail ENTSO-E has not published yet, the very interval the forecast bridge is built to span. The curation and imputation steps (Section 14.6.4, Section 14.6.5) will encounter them, but they are harmless: `end_train` slices them out of the training target `y_train`, so the model never fits them.

The table below makes these boundaries explicit so the trailing `NaN` count seen later is not mistaken for corrupt history. (The symbol `y_e` here marks a data-availability boundary — the last observed actual hour — and is distinct from the forecast notation `y_0` / `y_{-1}`, which name the target day and the day before it.)

```
from IPython.display import Markdown

naive_last = NOW_UTC.floor("h") - pd.Timedelta(hours=1)
last_actual_qh = interim["Actual Load"].dropna().index.max()
lag_h = int((naive_last - LAST_FULL_HOUR) / pd.Timedelta(hours=1))

hourly_actual = interim[["Actual Load"]].resample("h").mean()
interior_gaps = int(hourly_actual.loc[:LAST_FULL_HOUR, "Actual Load"].isna()
trailing_hours = int(hourly_actual.loc[FIRST_PRED:, "Actual Load"].isna().sum

availability = pd.DataFrame(
    [
        ("render time (now)",                              "",       f"{NOW_UTC}
        ("naive last available hour (now - 1 h)",      "y_{-1}", f"{naive_las
        ("merged-frame index max (via Forecasted Load)", "",     f"{interim.
        ("last published Actual Load (15-min)",            "",       f"{last_actu
        ("last observed actual hour = end_train",      "y_e",    f"{LAST_FULL
        ("ENTSO-E publication lag (y_{-1} - y_e)",     "",       f"{lag_h} h
        ("interior gaps in observed range",            "",       f"{interior_
        ("trailing unpublished hours (y_e -> index max)", "",    f"{trailing_
        ("first forecast step",                            "",       f"{FIRST_PRE
        ("last forecast step",                             "",       f"{LAST_TARG
        ("forecast horizon length",                    "predict_size", f"{LIV
    ],
    columns=["quantity", "symbol", "value"],
)
Markdown(availability.to_markdown(index=False))
```

Table 14.1: Data availability vs. the requested interval. The last observed actual hour `y_e` (= `end_train`) lags the naive `now - 1 h`; the hours between `y_e` and the merged-frame index max carry no target and appear as trailing `NaN`, not as interior gaps.

quantity	symbol	value
render time (now)		2026-05-31 19:37:53.771067+00:00
naive last available hour (now - 1 h)	y_{-1}	2026-05-31 18:00:00+00:00
merged-frame index max (via Forecasted Load)		2026-06-01 21:45:00+00:00
last published Actual Load (15-min)		2026-05-31 18:45:00+00:00
last observed actual hour = end_train	y_e	2026-05-31 18:00:00+00:00
ENTSO-E publication lag (y_{-1} - y_e)		0 h
interior gaps in observed range		0
trailing unpublished hours (y_e -> index max)		27
first forecast step		2026-05-31 19:00:00+00:00
last forecast step		2026-06-01 23:00:00+00:00
forecast horizon length	predict_size	29

14.5.8 Visualization of the training data and ENTSO-E's own day-ahead `Forecasted Load`

Figure 14.1 shows the full training series the forecaster is tuned on. It is assembled in a single line, `train_view = interim.loc[:LAST_FULL_HOUR, ["Actual Load"]].dropna()`, which is worth unpacking. The `.loc[:LAST_FULL_HOUR, ["Actual Load"]]` part takes a label-based slice of the downloaded frame: every row from the first downloaded timestamp up to and including `LAST_FULL_HOUR` (the training cutoff `end_train`, Section 14.5.6), keeping only the `Actual Load` column. Label slicing with `.loc` is inclusive on both ends, so `LAST_FULL_HOUR` itself is retained; the double brackets return a one-column DataFrame rather than a Series, which is the shape `visualize_ts_plotly` expects. The trailing `.dropna()` drops hours ENTSO-E has not yet published, so the plot shows only observed load — the raw, 15-minute "what we downloaded" view (see Section 14.6.3 for the hourly aggregation), before the pipeline's own outlier removal (Section 14.6.4) and imputation (Section 14.6.5).

The full series spans years of hourly load, so the seasonal shape is visible but the recent days blur together. Figure 14.2 therefore zooms in on the final week — the data closest to the live forecast bridge — with the actuals restricted to the seven days ending at `LAST_FULL_HOUR` (`LAST_WEEK_START = LAST_FULL_HOUR - pd.Timedelta(days=7)`, then `train_view.loc[LAST_WEEK_START:LAST_FULL_HOUR]`).

Overlaid on the same axes is ENTSO-E's own day-ahead `Forecasted Load`, renamed onto the shared `Actual Load` column so `visualize_ts_plotly` draws it as a second trace. In the full-series figure it is clipped to `end_train` to line up with the actuals; in the last-week figure it is sliced with no right bound (`predictions_entsoe.loc[LAST_WEEK_START:]`), so its most recent values run past `end_train` into the published day-ahead forecast for the hours the actuals do not yet cover. It is shown purely for visual context — the forecaster never trains on it (Section 14.6.11).

```
train_view = interim.loc[:LAST_FULL_HOUR, ["Actual Load"]].dropna()
# Reuse predictions_entsoe (@sec-team4-entsoe-predictions), sliced to the
# training window. visualize_ts_plotly overlays one trace per dict entry but
# requires the plotted column to exist in every frame, so the forecast is
# renamed onto the shared "Actual Load" column; the dict key supplies its
# legend label.
entsoe_view = predictions_entsoe.loc[:LAST_FULL_HOUR].rename("Actual Load").to_frame()
print(f"training view: {len(train_view)} rows "
      f"| {train_view.index.min()} → {train_view.index.max()}")
print(f"ENTSO-E forecast view: {len(entsoe_view)} rows "
      f"| {entsoe_view.index.min()} → {entsoe_view.index.max()}")
```

```
training view: 154725 rows | 2022-01-01 00:00:00+00:00 → 2026-
05-31 18:00:00+00:00
ENTSO-E forecast view: 38683 rows | 2022-01-01 00:00:00+00:00 → 2026-
05-31 18:00:00+00:00
```

```
from spotforecast2.plots.time_series_visualization import visualize_ts_plotly
visualize_ts_plotly(
    {
        "Actual Load (train)":      train_view,
        "ENTSO-E Forecasted Load":  entsoe_view,
    },
    columns=["Actual Load"],
    title_suffix="- team_4 training data (ENTSO-E DE)",
)
```

```
from spotforecast2.plots.time_series_visualization import visualize_ts_plotly

# Actuals: the final seven days ending at LAST_FULL_HOUR (end_train).
LAST_WEEK_START = LAST_FULL_HOUR - pd.Timedelta(days=7)
train_view_last_week = train_view.loc[LAST_WEEK_START:LAST_FULL_HOUR]
```

```
Unable to display output for mime type(s): text/html
```

(a) team_4 training data — the 15min ENTSO-E DE `Actual Load` series the advanced LGBM forecaster is tuned on (blue), overlaid with ENTSO-E's own day-ahead `Forecasted Load` (orange) over the same window. The forecast is shown only for context; the model never sees it (Section 14.6.11).

```
Unable to display output for mime type(s): text/html
```

(b)

Figure 14.1

```
# ENTSO-E forecast: slice predictions_entsoe directly with no right bound, so
# the most recent day-ahead values past end_train are kept (they extend into
# the future hours the actuals do not cover yet). Renamed onto the shared
# "Actual Load" column so visualize_ts_plotly draws it as a second trace.
entsoe_view_last_week = (
    predictions_entsoe.loc[LAST_WEEK_START:].rename("Actual Load").to_frame()
)

visualize_ts_plotly(
    {
        "Actual Load (train)":      train_view_last_week,
        "ENTSO-E Forecasted Load":  entsoe_view_last_week,
    },
    columns=["Actual Load"],
    title_suffix="- team_4 training data, last week (ENTSO-E DE)",
)
```

```
Unable to display output for mime type(s): text/html
```

Figure 14.2: team_4 training data, last week — the most recent seven days of the 15min ENTSO-E DE `Actual Load` series the advanced LGBM forecaster is tuned on (blue, ending at `end_train`), overlaid with ENTSO-E's own day-ahead `Forecasted Load` (orange), which is drawn in full and therefore runs past `end_train` into the published forecast for the hours the actuals do not yet cover. The forecast is shown only for context; the model never sees it (Section 14.6.11).

14.6 Lag structure: select `key_lags` from the data

The autoregressive lags the forecaster considers should follow the data, not a hard-coded guess. Before building the config we read the lag structure straight off the hourly `Actual Load` and let it pick the lags.

The autocorrelation function (ACF) and partial autocorrelation function (PACF) expose that structure. We select lags with the standard rule: the 95% white-noise significance band is $\pm 1.96/\sqrt{N}$, and any correlation outside it is statistically non-zero. We rank candidates by the *partial* autocorrelation, not the raw ACF — the PACF strips out the correlation already explained by shorter lags, so its significant spikes are the textbook Box–Jenkins signal for which lags carry *direct* autoregressive information. A raw-ACF threshold (say `> 0.8`) is unhelpful here: load autocorrelation decays so slowly that nearly every lag would clear it. We keep the strongest few significant lags by `|PACF|` and feed them into `ConfigEntsoe(lags_consider=key_lags)` below. They typically recover the hourly persistence (lags 1–3), the daily cycle (around 24), and the weekly cycle (around 168). This works on the raw downloaded series (the hourly mean of the 15-minute actuals, before the pipeline's own curation), so it diagnoses the genuine lag structure; as Section 14.6.1 explains, the tuned model's final lags are still chosen by SpotOptim.

```
import numpy as np
import matplotlib.pyplot as plt
from spotforecast2.stats.autocorrelation import calculate_lag_autocorrelation

# Hourly Actual Load over the training side only (mean of each hour's four
# quarter-hour values), matching the grid prepare_data() will build later.
acf_series = interim.loc[:LAST_FULL_HOUR, "Actual Load"].resample("h").mean()
acf = calculate_lag_autocorrelation(acf_series, n_lags=200)

# Data-driven key lags. The 95% white-noise band is ±1.96/sqrt(N); a lag is
# "significant" when its correlation falls outside it. We rank by the PARTIAL
# autocorrelation (PACF), which removes the correlation already carried by
# shorter lags and so flags only the lags with *direct* predictive value ---
# the standard Box-Jenkins rule. (A raw-ACF threshold is useless here: the lo
# ACF decays so slowly that almost every lag clears the band.)
conf = 1.96 / np.sqrt(len(acf_series))
significant = acf[acf["partial_autocorrelation_abs"] > conf]
key_lags = sorted(significant.nlargest(8, "partial_autocorrelation_abs")["lag
if not key_lags:                          # degenerate fallback (e.g. very short
    key_lags = [1, 2, 24, 168]

print(f"N = {len(acf_series)}  |  95% significance band = ±{conf:.4f}")
print(f"significant PACF lags (|pacf| > band): {len(significant)} of {len(acf
print(f"key_lags (top {len(key_lags)} by |PACF|, in lag order): {key_lags}")
print(acf[acf["lag"].isin(key_lags)][
    ["lag", "autocorrelation", "partial_autocorrelation"]
].to_string(index=False))

fig, ax = plt.subplots(figsize=(8.5, 3.0))
ax.bar(acf["lag"], acf["autocorrelation"], width=0.9, color="#1F4E79")
ax.axhline( conf, color="#999999", lw=0.8, ls="--")
ax.axhline(-conf, color="#999999", lw=0.8, ls="--")
for lag in key_lags:
```

```
    row = acf[acf["lag"] == lag]
    if not row.empty:
        val = float(row["autocorrelation"].iloc[0])
        ax.annotate(f"lag {lag}", xy=(lag, val),
                    xytext=(lag, val + 0.08), ha="center", fontsize=8,
                    color="#E07B00",
                    arrowprops=dict(arrowstyle="->", color="#E07B00", lw=0.8))
ax.set_xlabel("lag (hours)")
ax.set_ylabel("autocorrelation")
ax.set_title("ACF of Actual Load (annotated: data-selected key_lags; dashed = 95% band)")
ax.grid(True, color="#E5E5E5", linewidth=0.5)
for s in ("top", "right"):
    ax.spines[s].set_visible(False)
fig.tight_layout()
plt.show()
```

```
N = 38683  |  95% significance band = ±0.0100
significant PACF lags (|pacf| > band): 163 of 200
key_lags (top 8 by |PACF|, in lag order): [1, 2, 3, 15, 24, 25, 26, 169]
 lag  autocorrelation  partial_autocorrelation
   1         0.968940                 0.968965
   2         0.887719                -0.836711
  24         0.798884                -0.420846
  25         0.764812                -0.366218
   3         0.775323                 0.336359
 169         0.872321                -0.316312
  15         0.137298                 0.302143
  26         0.684706                 0.282528
```

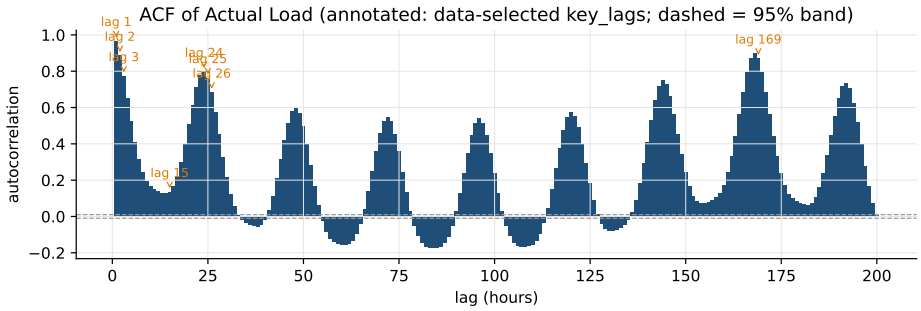


Figure 14.3: Autocorrelation of the hourly `Actual Load` up to lag 200, with the 95% white-noise significance band (±1.96/√N, dashed grey). The annotated `key_lags` are chosen dynamically — the significant lags with the largest *partial* autocorrelation — and are passed to `ConfigEntsoe(lags_consider=...)`. SpotOptim still selects the model's final lags (Section 14.6.1).

14.6.1 Advanced `ConfigEntsoe`

Each change relative to Section 13.5.4 has a one-line rationale:

- All five seasonal periods — captures multi-scale seasonality the recursive LGBM cannot recover from hourly lags alone.
- Data-selected lags — the PACF-significant lags from 1 (typically hourly 1–3, daily ~24, weekly ~168), not a fixed set.
- `include_weather_windows=True` — temperature is the dominant load driver below ~5 °C and above ~15 °C.
- `include_holiday_features=True` — DE/NW public holidays shift weekday load by 20–30 %.
- `poly_features_degree=2` — bilinear weather × calendar interactions, capped at `max_poly_features=10` (top ten by mutual information) so the feature space stays compact.

- `n_trials_spotoptim=6`, `n_initial_spotoptim=5`, `number_folds=10` — a deliberately small search budget that keeps the live render fast; raise it for a more thorough tuning run.

```
TRAIN_SIZE = pd.Timedelta(days=365 * 3)
PREDICT_SIZE = 24
REFIT_SIZE = 7
NUMBER_FOLDS = 10
DELTA_VAL = pd.Timedelta(hours=PREDICT_SIZE * REFIT_SIZE * NUMBER_FOLDS)
# Gap-penalty zone (hours) for weighted imputation, decoupled from the
# rolling-feature `window_size` (default 72). A narrower zone reduces the
# "all sample weights zero -> uniform weighting" warning on short CV folds.
IMPUTATION_WINDOW_SIZE = 24

ADVANCED_PERIODS = [
    Period(name="daily",     n_periods=12, column="hour",      input_range=(1, 24)),
    Period(name="weekly",    n_periods=7,  column="dayofweek", input_range=(0, 6)),
    Period(name="monthly",   n_periods=12, column="month",     input_range=(1, 12)),
    Period(name="quarterly", n_periods=4,  column="quarter",   input_range=(1, 4)),
    Period(name="yearly",    n_periods=12, column="dayofyear", input_range=(1, 365)),
]

team4_config = ConfigEntsoe(
    country_code="DE",
    data_filename="interim/energy_load.csv",
    targets=["Actual Load"],
    agg_weights=[1.0],
    bounds=None,
    data_loader=entsoe_data_loader,
    test_data_loader=entsoe_test_data_loader,
    forecaster_factory=entsoe_lgbm_factory,
    periods=ADVANCED_PERIODS,
    lags_consider=key_lags,
```

```
    train_size=TRAIN_SIZE,
    end_train_default=LAST_FULL_HOUR.isoformat(),
    delta_val=DELTA_VAL,
    predict_size=LIVE_N_STEPS,
    cv_block_size=PREDICT_SIZE,
    refit_size=REFIT_SIZE,
    number_folds=NUMBER_FOLDS,
    imputation_window_size=IMPUTATION_WINDOW_SIZE,
    n_trials_optuna=15,
    n_trials_spotoptim=30,
    n_initial_spotoptim=10,
    warm_start_lags=True,
    include_weather_windows=True,
    include_holiday_features=True,
    poly_features_degree=2,
    max_poly_features=10,
    state="NW",
    random_state=42,
    on_weather_failure="skip",
)
team4_config.data_frame_name = "ddmo-live-team4"
print("team4_config ready:", team4_config.data_frame_name,
      "| predict_size=", team4_config.predict_size,
      "| n_trials_spotoptim=", team4_config.n_trials_spotoptim,
      "| number_folds=", team4_config.number_folds)
```

```
team4_config ready: ddmo-live-team4 | predict_size= 29 | n_trials_spotoptim=
```

14.6.1.1 What each constant and argument means

The cell defines two things: a handful of local constants that fix the cross-validation geometry and the feature menu, and the `ConfigEntsoe` object that records every

choice the pipeline reads. The value shown for each item is the one used here.

Local constants — the cross-validation geometry:

- `TRAIN_SIZE` — length of the rolling training window, a `pd.Timedelta`. Here `pd.Timedelta(days=365 * 3)`, i.e. three years. Example: a fold whose `end_train` is `2026-05-20 23:00` reaches back to roughly `2023-05-21 00:00`.
- `PREDICT_SIZE` — length of one forecast block in hours. `24` is one day. It feeds `DELTA_VAL` only; the config's live horizon is `predict_size=LIVE_N_STEPS` (below), not this fixed `24`.
- `REFIT_SIZE` — spacing, in days, at which the model is refit while backtesting. `7` means a weekly refit.
- `NUMBER_FOLDS` — number of time-ordered cross-validation folds. `10`.
- `DELTA_VAL` — total validation span reserved for the rolling CV, `pd.Timedelta(hours=PREDICT_SIZE * REFIT_SIZE * NUMBER_FOLDS)` = `24 × 7 × 10 = 1680` h = 70 days. The most recent 70 days of history are held out for evaluation. This constant is explained in detail in Section 14.6.2.
- `ADVANCED_PERIODS` — a list of `Period` cyclical encodings, one per seasonal scale. Each `Period(name, n_periods, column, input_range)` reads `column` off the timestamp, treats its values as living on `input_range`, and encodes them with `n_periods` smooth basis functions. Example: `Period("daily", 12, "hour", (1, 24))` turns the hour-of-day into 12 daily encodings; `Period("weekly", 7, "dayofweek", (0, 6))` gives a seven-way day-of-week encoding.

(The autoregressive lags are no longer a constant here: `key_lags` is selected from the data in 1 and passed straight to `lags_consider` below.)

`ConfigEntsoe` — identity and data source:

- `country_code="DE"` — ISO 3166-1 alpha-2 code; selects both the ENTSO-E query and the national holiday calendar.

- `data_filename="interim/energy_load.csv"` — path, under the data home, of the merged interim frame to read (the file profiled in Section 14.5.3).
- `targets=["Actual Load"]` — the column(s) to forecast; one target here.
- `agg_weights=[1.0]` — per-target weight used when several targets are aggregated into one score; with a single target it is `1.0`.
- `bounds=None` — per-target outlier limits, left unset here on purpose. A fixed pair like `±1e9` MW would never flag anything; instead the bounds are *derived from the long load history* (relaxed `median ± 5·IQR`, lower floored at 0 MW) and assigned just before detection in Section 14.6.4. These hard bounds are the only step that marks anomalies; the IsolationForest pass is report-only logging (see Section 14.6.4).

`ConfigEntsoe` — pipeline wiring (callables):

- `data_loader=entsoe_data_loader` — function `loader(config)` that returns the training DataFrame; invoked by `prepare_data()`.
- `test_data_loader=entsoe_test_data_loader` — function returning ground-truth values for the horizon, used to populate `test_actual` and `metrics_future` when they exist.
- `forecaster_factory=entsoe_lgbm_factory` — factory that builds the LightGBM recursive forecaster the search then tunes.

`ConfigEntsoe` — features:

- `periods=ADVANCED_PERIODS` — wire in the seasonal encodings above.
- `lags_consider=key_lags` — the data-selected candidate lags computed in

    1. Paired with `warm_start_lags=True` (below), this list is injected as a candidate in the tuner's lag search and seeded as its first evaluation; see the note after this list for exactly how it is consumed.

- `include_weather_windows=True` — add rolling weather-window features.

- include_holiday_features=True — add public-holiday indicators, regionalised by state.
- poly_features_degree=2 — generate pairwise bilinear interaction terms (1 would disable interactions).
- max_poly_features=10 — keep only the ten most informative interaction columns, ranked by mutual information with the target.
- state="NW" — the German subdivision code (North Rhine-Westphalia) driving the regional holiday calendar.

Where key_lags is used — and who really picks the lags

A natural question is whether lags_consider=key_lags fixes the lags the model forecasts on, or whether the tuner decides. In the tuned spotoptim run this chapter performs (MultiTask(team4_config, task="spotoptim")), the tuner still chooses the winner, but with warm_start_lags=True the data-selected list now genuinely competes. Three steps consume it:

1. The forecaster factory entsoe_lgbm_factory (spotforecast2/multitask/factories.py) builds the *initial* forecaster with lags=config.lags_consider[-1] — only the largest entry. With the data-selected key_lags (whose maximum is the weekly lag, around 168), initialize_lags expands that maximum to the full contiguous range 1…max, not the individual selected lags.
2. SpotOptim treats the lag configuration as a categorical hyperparameter and searches a candidate set. By default that set is hardcoded (_default_spotoptim_search_space in spotforecast2/multitask/search_spaces.py — e.g. "24", "[1, 2, 24, 48]", "[1, 2, 11, 12, 23, 24, 167, 168]"). With warm_start_lags=True, SpotOptimStrategy *prepends* str(key_lags) to that list, so the PACF-selected lags become a real, selectable candidate.

3. The same flag seeds the surrogate search: `build_warm_start_x0` constructs the optimizer's first evaluation point at `key_lags` (combined with the factory's estimator defaults) and passes it as SpotOptim's `x0`, so the seeded configuration is guaranteed to be evaluated rather than left to chance in the random initial design. The winner is applied with `set_lags`, logged as "Best lags", and inspectable via exercise 3.

A consequence worth flagging: `initialize_lags` expands an *integer* candidate to the contiguous range `1…max`, but leaves a *list* candidate untouched. So when the logged "Best lags" come back gapless (e.g. `1…24`), it means the integer candidate `"24"` won the back-test — not that the search cannot carry gaps. The candidate set above includes genuine non-contiguous sets such as `[1, 2, 11, 12, 23, 24, 167, 168]`, and the winning lags are free to be one of them; the weekly lag survives in the search whether or not it wins. So under `warm_start_lags=True` the ACF analysis in 1 no longer only *motivates* candidate lags — it injects them into the search and seeds the optimizer with them, while the tuner still has the final say among all candidates. With `warm_start_lags=False` (the default), `lags_consider` reverts to its passive role: only its maximum (`lags_consider[-1]`) sizes the pre-tuning forecaster and the minimum history the back-test reserves, and the tuner picks from the hardcoded set alone.

`ConfigEntsoe` — training window, horizon, refit:

- `train_size=TRAIN_SIZE`, `delta_val=DELTA_VAL`, `refit_size=REFIT_SIZE`, `number_folds=NUMBER_FOLDS` — hand the CV geometry above to the config.
- `end_train_default=LAST_FULL_HOUR.isoformat()` — ISO timestamp anchoring the end of training: the last full hour of published actuals (Section 14.5.6). Example: `"2026-05-20T23:00:00+00:00"`.
- `predict_size=LIVE_N_STEPS` — the live horizon in hours, i.e. the recursive bridge from `end_train` through tomorrow 23:00 UTC, which is longer than the fixed `PREDICT_SIZE=24`.

- `cv_block_size=PREDICT_SIZE` — the cross-validation test-block width (24 h), fixed and decoupled from the render-dependent live `predict_size`. Leaving it unset would make the CV inherit `predict_size=LIVE_N_STEPS`; pinning it at 24 keeps the fold geometry stable across renders (Section 14.6.2).

`ConfigEntsoe` — search budget and reproducibility:

- `n_trials_optuna=15` — Optuna Bayesian-search trial budget.
- `n_trials_spotoptim=6` — SpotOptim surrogate-search trial budget.
- `n_initial_spotoptim=5` — SpotOptim initial random evaluations before the surrogate model takes over.
- `warm_start_lags=True` — inject `key_lags` as a candidate lag set and seed the surrogate search's first evaluation with it (see the note above).
- `random_state=42` — RNG seed fixing tuning and model randomness, so the render is reproducible (CR-2).
- `on_weather_failure="skip"` — if the Open-Meteo fetch fails, log a warning and continue with empty weather features rather than abort; `"raise"` (the default) would fail loud instead.

Finally, the post-construction assignment `team4_config.data_frame_name = "ddmo-live-team4"` names the cache subdirectory, the saved-model files, and the per-dataset log for this run.

The `MultiTask` orchestrator runs the full pipeline in one call, but a safety-critical forecaster is only trustworthy if its inputs are auditable. The next six subsections run the pipeline one method at a time and inspect the data each step produces — endogenous series, curation, imputation, exogenous features, the tuned forecast, and the feature attributions. Each method mutates `team4_mt` in place, so the cells must run top to bottom.

14.6.2 Cross-validation geometry recap

The four constants defined at the top of Section 14.6.1 — `PREDICT_SIZE`, `REFIT_SIZE`, `NUMBER_FOLDS`, and the `DELTA_VAL` they combine into — do one job: they fix the *cross-validation geometry* that SpotOptim scores every candidate model against. The config explainer above quoted the result ("the most recent 70 days are held out"); this section derives that number from scratch and shows exactly how the rolling back-test that produces SpotOptim's objective is laid out. It is self-contained — it reuses the chronological-split idea of Chapter 5 and the rolling-origin back-test of Chapter 10, but every term needed is defined here.

14.6.2.1 Definitions

The pipeline reasons about one contiguous, hourly, gap-free load series. The following terms pin down how that series is carved up for tuning.

Training series the full hourly `Actual Load` the tuner is allowed to look at. It runs from `start_train` to `end_train`.

`end_train` the last timestamp the model may train on — here `LAST_FULL_HOUR`, the last full hour of published actuals (Section 14.5.6). Everything after it is the live forecast, never part of tuning.

`train_size` the length of the training series as a duration. `start_train = end_train - train_size`. Here three years.

`predict_size` the *live* forecast horizon — how many hours the model is asked to forecast forward in production. The live config sets it to `LIVE_N_STEPS` (Section 14.5.6), the recursive bridge to tomorrow 23:00, so it is render-dependent (≈ 32). It does *not* set the cross-validation block; that is `cv_block_size`.

`cv_block_size` the width of each cross-validation test block, in hours — the horizon the folds actually score. The config sets it to `PREDICT_SIZE = 24` (one day), fixed and independent of the render-dependent `predict_size`. Decoupling the two is deliberate: the CV scores a stable 24-hour-ahead block while the live forecast still runs out to `LIVE_N_STEPS`. (When

`cv_block_size` is left unset it falls back to `predict_size` — the behaviour chapters 02–11 rely on, where the two coincide at 24.)

`delta_val` the validation span: the stretch of the most recent history reserved for evaluation and withheld from the model's training. It is computed, not chosen directly — see below.

`end_cv` the cross-validation training cutoff, `end_cv = end_train - delta_val`. The model is trained on `[start_train, end_cv]` and validated on `(end_cv, end_train]`.

Fold one (train, test) pair. Every fold trains on data ending at or before its test block and predicts the next `cv_block_size` hours — so a fold never peeks at its own future, the discipline introduced in Chapter 5.

Fold stride how far the test block advances between consecutive folds. Here it equals `cv_block_size = 24`, so the folds are back-to-back and non-overlapping: together they tile the whole validation span exactly once.

Rolling-origin back-test the procedure (Chapter 10) that slides the forecast origin forward one stride at a time across the validation span, collecting a forecast at each step. With `refit=False` the forecaster is fit *once* on `[start_train, end_cv]` and then reused to produce every fold's forecast — fast, and a deterministic choice (CR-2).

Mean absolute error (MAE) the score. The fold forecasts are pooled into one vector $\hat{y}$, compared with the matching actuals y, and averaged: $\text{MAE} = \frac{1}{n}\sum_{t=1}^{n}|y_t - \hat{y}_t|$. This single number, over all validation hours, is what SpotOptim minimises.

14.6.2.2 Computing `DELTA_VAL`

`DELTA_VAL` is the product of the other three constants, read as hours:

$$\texttt{DELTA_VAL} = \underbrace{\texttt{PREDICT_SIZE}}_{24\text{ h/fold}} \times \underbrace{\texttt{REFIT_SIZE}}_{7} \times \underbrace{\texttt{NUMBER_FOLDS}}_{10} = 24{\times}7{\times}10 = 1680\,\text{h} = 70\,\text{days}.$$

This span is then sliced off the *tail* of the training series. With `end_cv = end_train - delta_val`, the series splits cleanly into a training part and a validation part whose lengths add up to `train_size`:

$$\underbrace{\texttt{train_size}}_{3\ \text{yr} = 26280\ \text{h}} = \underbrace{(\texttt{train_size} - \texttt{delta_val})}_{\text{CV training window}} + \underbrace{\texttt{delta_val}}_{\text{validation span}} = 24600\,\text{h} + 1680\,\text{h}.$$

The fold count then falls straight out of the geometry. The folds are non-overlapping blocks of `cv_block_size = 24` hours that tile the validation span, so their number is

$$\#\text{folds} = \frac{\texttt{delta_val}}{\texttt{cv_block_size}} = \frac{1680}{24} = \texttt{REFIT_SIZE} \times \texttt{NUMBER_FOLDS} = 7 \times 10 = 70,$$

and the total validation hours scored is $70 \times 24 = 1680 =$ `delta_val`. The `70/70` progress bars at Section 14.6.8 are exactly this count, the same for every render. The clean identity holds *because* we set `cv_block_size = PREDICT_SIZE`. Had the CV instead inherited the live `predict_size = LIVE_N_STEPS` (≈ 32 h) — as it did before the `cv_block_size` field existed — the block would tile the span into `⌊1680 / 32⌋ = 53` folds, and the count would drift render to render with the feed's freshness (Section 14.5.6). Decoupling the CV block from the live horizon is exactly what `cv_block_size` buys.

A note on the names, so the arithmetic is not mistaken for behaviour the code does not have. In *this* cross-validation the model is fit once and reused (`refit=False`), so `REFIT_SIZE` does not trigger any refitting — it is one of two factors whose product, with `PREDICT_SIZE`, sizes the held-out span. Likewise `NUMBER_FOLDS` enters only through that product; because we set `cv_block_size = PREDICT_SIZE`, the real fold count is the 70 derived above, not 10. The constants are a convenient way to *spell* a 70-day window as "ten weekly blocks of daily forecasts", not a description of a ten-fold refit loop. (Internally `cv_ts` in `spotforecast2` reads `cv_block_size` — falling back to

`predict_size` when it is unset — sets `end_cv = end_train - delta_val`, and returns a `TimeSeriesFold(steps=cv_block_size, refit=False, allow_incomplete_fold=True)` — the same `TimeSeriesFold` machinery used in Chapter 10.)

14.6.2.3 A toy example you can check by hand

The real case (dozens of folds) is hard to draw, so first shrink every constant. Take `predict_size = 2`, `refit_size = 2`, `number_folds = 3`, and a training series of `train_size = 30` time units. Then

$$\texttt{delta_val} = 2{\times}2{\times}3 = 12, \quad \text{CV training window} = 30{-}12 = 18, \quad \#\text{folds} = \tfrac{12}{2} = 6.$$

Index the series $0, 1, \ldots, 29$. Training is the fixed block $[0, 18)$; the six test blocks tile the last twelve positions, each two wide:

fold	trains on	tests on
1	[0, 18)	[18, 20)
2	[0, 18)	[20, 22)
3	[0, 18)	[22, 24)
4	[0, 18)	[24, 26)
5	[0, 18)	[26, 28)
6	[0, 18)	[28, 30)

The model is fit once on $[0, 18)$, makes six successive 2-step forecasts that march across $[18, 30)$, and the MAE is taken over all $6 \times 2 = 12$ predicted positions. In this toy the block (`cv_block_size = 2`) equals the unit that sizes `delta_val`, so the fold count is the clean `delta_val / cv_block_size = 6`. The schematic below draws exactly this layout; the real configuration is the same picture with 70 daily (24-hour) blocks instead of six 2-step ones — the live `predict_size`

= LIVE_N_STEPS plays no part in the CV geometry, only in the production forecast.

```
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches

# Toy constants (small enough to draw every block); the real run scales these
# to PREDICT_SIZE=24, REFIT_SIZE=7, NUMBER_FOLDS=10 -> 1680 h (70-day) span,
# tiled into cv_block_size=24-h blocks -> 1680/24 = 70 folds (render-independ
PS, RS, NF = 2, 2, 3          # cv_block_size, refit_size, number_folds
TRAIN = 30                    # train_size, in the same time units as the fo
DV = PS * RS * NF             # delta_val = 12
CVT = TRAIN - DV              # CV training window = 18
NFOLDS = DV // PS             # number of folds = 6

BLUE, ORANGE, GREY, GRID = "#1F4E79", "#E07B00", "#999999", "#E5E5E5"

fig, (axA, axB) = plt.subplots(
    2, 1, figsize=(8.5, 4.6), gridspec_kw={"height_ratios": [1.0, 1.5]}
)

# --- Panel A: the split of the training series ----------------------------
axA.add_patch(mpatches.Rectangle((0, 0), CVT, 1, facecolor=BLUE,
                                 edgecolor="white", label="CV training windo
for k in range(NFOLDS):
    axA.add_patch(mpatches.Rectangle((CVT + k * PS, 0), PS, 1, facecolor=ORA
                                     edgecolor="white"))
# marker lines + labels for the three anchors
for x, lab in [(0, "start_train"), (CVT, "end_cv"), (TRAIN, "end_train")]:
    axA.axvline(x, color="#555555", lw=0.7, ls="--")
    axA.text(x, -0.35, lab, ha="center", va="top", fontsize=8, color="#33333
# delta_val span annotation
axA.annotate("", xy=(CVT, 1.35), xytext=(TRAIN, 1.35),
             arrowprops=dict(arrowstyle="<->", color=ORANGE, lw=1.0))
```

```
axA.text((CVT + TRAIN) / 2, 1.5, f"delta_val = {DV}", ha="center",
         fontsize=8, color=ORANGE)
# one predict_size block annotation
axA.annotate("", xy=(CVT, 1.12), xytext=(CVT + PS, 1.12),
             arrowprops=dict(arrowstyle="<->", color="#333333", lw=0.8))
axA.text(CVT + PS / 2, 1.2, f"predict_size = {PS}", ha="left", fontsize=7,
         color="#333333")
axA.set_xlim(-1.5, TRAIN + 1.5)
axA.set_ylim(-0.9, 1.8)
axA.set_yticks([])
axA.set_title(f"Series split: train_size {TRAIN} = CV-train {CVT} + delta_val {DV}"
              f"   (-> {NFOLDS} folds; real run: 1680/24 = 70 folds over 70 days)",
              fontsize=9)
for s in ("top", "right", "left"):
    axA.spines[s].set_visible(False)

# --- Panel B: the rolling-origin folds, one per row ---------------------------
for k in range(NFOLDS):
    y = NFOLDS - 1 - k                          # fold 1 on top
    axB.add_patch(mpatches.Rectangle((0, y + 0.1), CVT, 0.8, facecolor=BLUE,
                                     edgecolor="white"))
    axB.add_patch(mpatches.Rectangle((CVT + k * PS, y + 0.1), PS, 0.8,
                                     facecolor=ORANGE, edgecolor="white"))
    axB.text(-0.5, y + 0.5, f"fold {k + 1}", ha="right", va="center", fontsize=8)
axB.axvline(CVT, color="#555555", lw=0.7, ls="--")
axB.text(CVT, -0.4, "end_cv", ha="center", va="top", fontsize=8, color="#333333")
axB.text(CVT / 2, NFOLDS + 0.25, "fit once (refit=False), reused for every fold",
         ha="center", fontsize=8, color=BLUE)
axB.set_xlim(-1.5, TRAIN + 1.5)
axB.set_ylim(-0.9, NFOLDS + 0.7)
axB.set_yticks([])
axB.set_xlabel("position in the training series (time -->)", fontsize=8)
axB.grid(True, axis="x", color=GRID, linewidth=0.5)
```

```
for s in ("top", "right", "left"):
    axB.spines[s].set_visible(False)

# shared legend
handles = [mpatches.Patch(color=BLUE, label="training window"),
           mpatches.Patch(color=ORANGE, label="test block (one fold)")]
axA.legend(handles=handles, loc="upper left", fontsize=7, frameon=False)

fig.tight_layout()
plt.show()
```

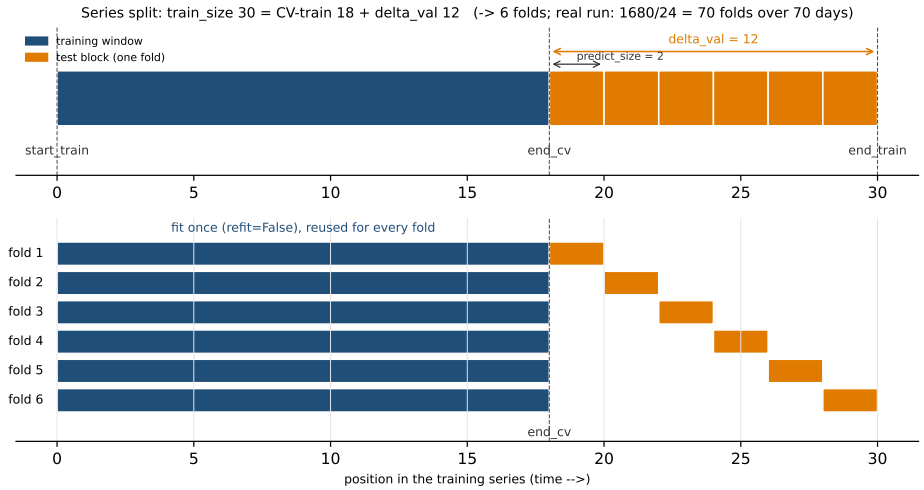


Figure 14.4: Cross-validation geometry behind `DELTA_VAL`, drawn with the toy constants `predict_size=2`, `refit_size=2`, `number_folds=3`, `train_size=30` (so `delta_val=12` and 6 folds). Top: the series splits into a fixed training window `[start_train, end_cv]` (blue) and the held-out validation span `delta_val` (orange), tiled by non-overlapping `predict_size`-wide folds. Bottom: the rolling-origin back-test — the model is fit once and reused, the test block sliding forward one stride per fold. The real run uses the same picture with 70 daily (`cv_block_size=24`) folds over the 70-day span, independent of the live `predict_size`.

14.6.2.4 The real configuration

Substituting the chapter's constants reproduces the numbers quoted at Section 14.6.1:

- validation span: `delta_val = 24 × 7 × 10 = 1680` h = 70 days, held out as `(end_cv, end_train]` with `end_cv = end_train − 70 days`;

- CV training window: `26280 - 1680 = 24600` h ≈ 2.81 years, the fixed block the forecaster is fit on once;
- folds: `1680 / cv_block_size = 1680 / 24 = 70` non-overlapping daily blocks tiling the 70-day span — the same count for every render, because `cv_block_size` is fixed at 24 and does not track the live `predict_size`;
- scored hours: `70 × 24 = 1680 = delta_val`, pooled into the single MAE SpotOptim drives down.

14.6.2.5 How SpotOptim consumes it

This geometry is the fixed yardstick of the search. Each SpotOptim trial proposes one hyperparameter candidate — including a lag set, the warm-started `key_lags` among them (Section 14.6.1) — builds the forecaster, runs the 70-fold rolling back-test above, and returns its pooled MAE. The surrogate optimiser then picks the next candidate to try, always against the *same* 1680-hour validation window tiled the *same* way every render, so trials are directly comparable. Exercise 3 inspects which lag configuration ultimately won.

Why `cv_block_size` is a separate knob

The config carries two horizons that are easy to confuse. `predict_size = LIVE_N_STEPS` is the *live* forecast horizon — the recursive bridge to tomorrow 23:00, which floats with the ENTSO-E feed's freshness at render time (≈ 25–48 h). `cv_block_size = 24` is the *cross-validation* block — the horizon each fold scores. Setting them apart matters for two reasons:

- The search minimises the pooled MAE of `cv_block_size`-step-ahead forecasts. Fixing that at a 24-hour day keeps the tuning objective aligned with the leaderboard's 24-hour metric, instead of a longer, render-dependent horizon.
- Because `cv_block_size` is constant, the validation protocol (block width and the `1680/24 = 70` fold count) is identical across renders

— the reproducibility (CR-2) the frozen chapters enjoy, now kept for the live one.

Were the two coupled — `cv_block_size` left unset, so the CV inherited `predict_size = LIVE_N_STEPS` — the block would be ≈ 32 h, the fold count ⌊1680/32⌋ = 53, and both would drift between renders. That coupling is exactly what the `cv_block_size` field (added in `spotforecast2-safe`) removes; Exercise 4 probes how much the choice of block actually moves the tuned model.

14.6.3 Step 1 — Endogenous data: prepare and profile

`prepare_data()` loads the ENTSO-E DE `Actual Load` series and aggregates the 15-minute actuals onto the hourly grid the forecaster expects (the hourly mean of each hour's four quarter-hour values). We profile it before any curation so later steps have a baseline to compare against.

```
team4_mt = MultiTask(team4_config, task="spotoptim")
team4_mt.prepare_data()

endo = team4_mt.df_pipeline
print(f"df_pipeline shape: {endo.shape}")
print(f"index range      : {endo.index.min()} → {endo.index.max()}")
print(f"cadence          : {endo.index.to_series().diff().dropna().mode().iloc[0]}  (hourly
print(endo["Actual Load"].describe())
```

```
INFO:spotforecast2.multitask.base.MultiTask:Pipeline data shape after preparation: (38710, 2
```

```
df_pipeline shape: (38710, 2)
index range      : 2022-01-01 00:00:00+00:00 → 2026-06-
01 21:00:00+00:00
```

```
cadence          : 0 days 01:00:00  (hourly mean of the 15-
min actuals)
count    38683.000000
mean     53579.879057
std       9355.244530
min      30902.685000
25%      45900.622500
50%      53618.930000
75%      60567.817375
max      78680.352500
Name: Actual Load, dtype: float64
```

```
from spotforecast2.plots.time_series_visualization import visualize_ts_plotly

visualize_ts_plotly(
    {"Actual Load": team4_mt.df_pipeline[["Actual Load"]]},
    columns=["Actual Load"],
)
```

```
Unable to display output for mime type(s): text/html
```

Figure 14.5: Endogenous target after `prepare_data()` — the hourly ENTSO-E DE `Actual Load` series the advanced LGBM forecaster is tuned on, before outlier curation or imputation.

Figure 14.6 zooms in on the last week of the endogenous target, which is the most recent data the model trains on and the closest to the forecast bridge. It is a sanity check that this window contains no unexpected gaps or anomalies before we hand it to the outlier detection step in Section 14.6.4.

```
from spotforecast2.plots.time_series_visualization import visualize_ts_plotly

last_week = team4_mt.df_pipeline.loc[
```

```
    team4_mt.df_pipeline.index.max() - pd.Timedelta(days=7):, ["Actual Load"]
]
visualize_ts_plotly(
    {"Actual Load": last_week},
    columns=["Actual Load"],
)
```

```
Unable to display output for mime type(s): text/html
```

Figure 14.6: Last week data of the endogenous target after `prepare_data()`

14.6.4 Step 2 — Outlier detection and curation

Curation needs a bound. A fixed pair such as `±1e9` MW flags nothing, so it removes nothing; a tight pair risks deleting genuine demand — the January cold-snap peaks and the holiday troughs are real records the model must see. We want the opposite of both: bounds wide enough to keep every plausible value, narrow enough to excise the physically impossible ones (a reported 0 MW national load, a sensor spike to ten times the peak).

We therefore derive the bounds from the load history itself, using a robust rule that the anomalies cannot distort. Let m be the median of the series and IQR its inter-quartile range (the 75th minus the 25th percentile). Median and IQR are *robust*: a handful of absurd records barely moves them, so the band they define is not inflated by the very points we mean to catch. The bounds are

$$\text{lower} = \max(0,\ m - K \cdot \text{IQR}), \qquad \text{upper} = m + K \cdot \text{IQR},$$

with $K = 5$ — a deliberately *relaxed* width. (For reference, Tukey's "far out" fence uses $K = 3$; ours is wider still.) For German load this places the band at roughly $[0,\ 130]$ GW, so only records outside any plausible national demand are

flagged. The lower bound is floored at 0 MW because load cannot be negative. `OUTLIER_IQR_K` is exposed as a constant: lower it to curate more aggressively, raise it to curate only the most extreme records.

We compute the bounds from the prepared history and assign them to `team4_mt.config.bounds` *before* detection runs.

```
OUTLIER_IQR_K = 5

s = team4_mt.df_pipeline["Actual Load"].dropna()
med = s.median()
iqr = s.quantile(0.75) - s.quantile(0.25)
low = max(0.0, med - OUTLIER_IQR_K * iqr)
high = med + OUTLIER_IQR_K * iqr
team4_mt.config.bounds = [(low, high)]

n_outside = int(((s < low) | (s > high)).sum())
print(f"median             : {med:,.0f} MW")
print(f"IQR                : {iqr:,.0f} MW")
print(f"derived bounds (K={OUTLIER_IQR_K}): [{low:,.0f}, {high:,.0f}] MW")
print(f"history points out : {n_outside} of {len(s)}")
```

```
median             : 53,619 MW
IQR                : 14,667 MW
derived bounds (K=5): [0, 126,955] MW
history points out : 0 of 38683
```

`detect_outliers()` snapshots the pre-curation series into `df_pipeline_original`, then replaces every value outside these bounds with NaN in `df_pipeline` via `manual_outlier_removal` (the values are *replaced*, not clipped to the bound). The second branch of `detect_outliers()` runs an IsolationForest pass (`get_outliers`, `contamination=0.01`), but that call only *reports* candidate anomalies to the log — it does not alter the series. The hard bounds are therefore

the sole mechanism that marks NaN. Removing implausible values here is a direct Art. 10 data governance obligation: the model must not learn from corrupted records.

What happens to a flagged record next is the substance of curation, and it is not deletion. The recursive LightGBM forecaster reads autoregressive lags off a *contiguous* hourly index; dropping rows outright would misalign every downstream lag. Instead, Step 3 (Section 14.6.5) imputes the NaN with the default `weighted` strategy: it forward/back-fills the gap so the index stays gap-free, *and* it builds a sample-weight function that assigns weight 0 to each imputed timestamp and the surrounding `imputation_window_size` h (here 24 h). That weight function is threaded into the forecaster (`ForecasterRecursive(weight_func=…)`), so during `fit` the curated rows contribute nothing to the training loss and nothing to the validation score. A flagged record is thus replaced by NaN, filled only to preserve lag continuity, and *excluded from training and validation* by zero weight — the intended Art. 10 outcome, reached without breaking the recursive feature construction.

`imputation_window_size` is a dedicated knob: it sets the gap-penalty width *independently* of `window_size` (default 72 h), which still controls the LightGBM rolling-mean feature window. During tuning SpotOptim evaluates many short cross-validation folds, and a fold lying entirely inside a penalty zone has every row at weight 0; the forecaster then logs *"all sample weights for the requested index are zero … using uniform weighting"* and falls back to uniform weights for that fold. This is benign — it is the safe degenerate case, not an error — but a narrower `imputation_window_size` makes it rarer without widening the rolling-feature window.

```
team4_mt.detect_outliers()

n_removed = int(
    (team4_mt.df_pipeline["Actual Load"].isna()
     & team4_mt.df_pipeline_original["Actual Load"].notna()).sum()
)
```

```
nan_to_impute = int(team4_mt.df_pipeline["Actual Load"].isna().sum())
imputed_idx = team4_mt.df_pipeline.index[team4_mt.df_pipeline["Actual Load"]
# Split genuine interior gaps from the unpublished trailing tail (@sec-team4-
# anything at or before end_train (LAST_FULL_HOUR) is a real gap; anything af
# simply the recent tail ENTSO-E has not published yet, which end_train exclu
interior_idx = imputed_idx[imputed_idx <= LAST_FULL_HOUR]
tail_idx = imputed_idx[imputed_idx > LAST_FULL_HOUR]
contamination = getattr(team4_mt.config, "contamination", None)

print(f"outliers removed   : {n_removed}")
print(f"NaN to impute next : {nan_to_impute}")
print(f"  interior gaps    : {len(interior_idx)}  (real gaps in observed rang
print(f"  trailing tail    : {len(tail_idx)}  (unpublished, after end_train)"
print(f"contamination      : {contamination}")
```

```
INFO:spotforecast2.multitask.base.MultiTask:Forecasted Load: 366 outliers aut
INFO:spotforecast2.multitask.base.MultiTask:Actual Load: 388 outliers automat
INFO:spotforecast2.multitask.base.MultiTask:Outlier detection complete.
```

```
outliers removed   : 0
NaN to impute next : 27
  interior gaps    : 0  (real gaps in observed range)
  trailing tail    : 27  (unpublished, after end_train)
contamination      : 0.01
```

```
from spotforecast2.plots.plotter import plot_with_outliers

plot_with_outliers(team4_mt.df_pipeline, team4_mt.df_pipeline_original, team4
```

```
Unable to display output for mime type(s): text/html
```

Figure 14.7: Outlier curation as performed by the pipeline: the pre-curation series (df_pipeline_original) overlaid with the curated series (df_pipeline), with removed points marked.

14.6.5 Step 3 — Missing-data imputation

impute() fills the gaps left by curation (and any pre-existing gaps) so the recursive forecaster sees a gap-free series. Under Art. 10 every filled value is a modelling assumption, so we verify the count goes to zero rather than trusting the call silently.

Of the nan_to_impute missing values, only interior_idx are genuine gaps inside the observed range; the rest (tail_idx) is the unpublished trailing tail described in Section 14.5.7 — the forecast-bridge hours after end_train that ENTSO-E has not published yet. Filling the tail is harmless: end_train excludes it from the training target y_train, so the model never fits the imputed tail values. The visualization below therefore zooms in only on the genuine interior gaps; on a typical render there are none, and the fallback view of the most recent hours is shown instead.

```
team4_mt.impute()

nan_after = int(team4_mt.df_pipeline["Actual Load"].isna().sum())
print(f"NaN before impute: {nan_to_impute}")
print(f"NaN after impute : {nan_after}  (must be 0)")
```

```
INFO:spotforecast2.multitask.base.MultiTask:apply_imputation: NaN cells before imputation: 2
INFO:spotforecast2.multitask.base.MultiTask:Applying weighted imputation (n2n style)...
INFO:spotforecast2.multitask.base.MultiTask:Weight function created with 38710 entries.
INFO:spotforecast2.multitask.base.MultiTask:apply_imputation: NaN cells after imputation: 0
INFO:spotforecast2.multitask.base.MultiTask:Imputation complete (method=weighted).
```

```
NaN before impute: 27
NaN after impute : 0  (must be 0)
```

```
print(f"interior gaps to plot: {len(interior_idx)}  "
      f"(trailing tail of {len(tail_idx)} unpublished hours not shown)")
if len(interior_idx) > 0:
    # Reconstruct the pre-imputation (gapped) series by re-inserting NaN at t
    # interior gap positions; the imputed series is df_pipeline after impute
    gapped = team4_mt.df_pipeline[["Actual Load"]].copy()
    gapped.loc[gapped.index.isin(interior_idx), "Actual Load"] = float("nan")
    imputed = team4_mt.df_pipeline[["Actual Load"]]

    print(f"plotting {len(interior_idx)} interior gaps (±12 h window each)")
    for i, t in enumerate(interior_idx, start=1):
        w = slice(t - pd.Timedelta(hours=12), t + pd.Timedelta(hours=12))
        visualize_ts_plotly(
            {
                "before (gaps)":   gapped.loc[w],
                "after (imputed)": imputed.loc[w],
            },
            columns=["Actual Load"],
            title_suffix=f"- gap {i}/{len(interior_idx)} @ {t:%Y-%m-%d %H:%M}
        )
else:
    print("no interior gaps; showing the last 72 hours of the imputed series")
    tail = team4_mt.df_pipeline[["Actual Load"]].tail(72)
    visualize_ts_plotly({"after (imputed)": tail}, columns=["Actual Load"])
```

```
interior gaps to plot: 0  (trailing tail of 27 unpublished hours not shown)
no interior gaps; showing the last 72 hours of the imputed series
```

```
Unable to display output for mime type(s): text/html
```

Figure 14.8: One panel per genuine interior gap: the curated series with the gap (before) overlaid with the imputed series (after) on a 24-hour window (±12 h) centred on each imputed timestamp. The unpublished trailing tail (Section 14.5.7) is excluded. If there are no interior gaps, the last 72 hours of the imputed series are shown instead.

14.6.6 Step 4 — Exogenous features: weather, holidays, calendar, polynomial

build_exogenous_features() encodes every exogenous family enabled in team4_config — weather and its rolling windows, DE/NW public holidays, the five seasonal radial basis function (RBF) / cyclical encodings, and (because poly_features_degree=2) the bilinear polynomial interactions, capped to the max_poly_features=10 most informative ones by mutual information. We group the generated columns by family at runtime (the exact encoded names depend on the configuration) and count each family.

```
team4_mt.build_exogenous_features()

exog_all = team4_mt.exogenous_features
# print the full list of exogenous features for manual inspection, then count by family
print("exogenous features:")
for col in exog_all.columns:
    print(f"  {col}")

print(f"exogenous_features shape : {exog_all.shape}")
print(f"selected for training    : {len(team4_mt.exog_feature_names)} features")

def family_of(col):
    c = col.lower()
    if "holiday" in c:
```

```
        return "holiday"
    if "poly" in c:
        return "polynomial"
    if "window" in c:
        return "weather_window"
    if any(k in c for k in ("sin", "cos", "rbf")):
        return "cyclical/RBF"
    if c.startswith("lag_") or c.startswith("lag"):
        return "lag"
    return "weather/other"

families = {}
for col in exog_all.columns:
    families.setdefault(family_of(col), []).append(col)

counts = pd.DataFrame(
    sorted(((fam, len(cols)) for fam, cols in families.items()),
           key=lambda kv: kv[1], reverse=True),
    columns=["family", "count"],
)
print(counts.to_string(index=False))
```

```
INFO:spotforecast2.multitask.base.MultiTask:Building exogenous features...
INFO:spotforecast2.multitask.base.MultiTask:  Weather features: (38739, 105)
INFO:spotforecast2.multitask.base.MultiTask:  Calendar features: (38739, 4)
INFO:spotforecast2.multitask.base.MultiTask:  Day/night features: (38739, 4)
INFO:spotforecast2.multitask.base.MultiTask:  Holiday features: (38739, 1)
INFO:spotforecast2.multitask.base.MultiTask:Capped polynomial features: kept
INFO:spotforecast2.multitask.base.MultiTask:Combined exogenous features shape
INFO:spotforecast2.multitask.base.MultiTask:Selected 127 exogenous features i
INFO:spotforecast2.multitask.base.MultiTask:Merged data shape: (38710, 128)
INFO:spotforecast2.multitask.base.MultiTask:Exogenous prediction covariates s
```

```
exogenous features:
  month
  week
  day_of_week
  hour
  sunrise_hour
  sunset_hour
  daylight_hours
  is_daylight
  temperature_2m
  relative_humidity_2m
  precipitation
  rain
  snowfall
  weather_code
  pressure_msl
  surface_pressure
  cloud_cover
  cloud_cover_low
  cloud_cover_mid
  cloud_cover_high
  wind_speed_10m
  wind_direction_10m
  wind_gusts_10m
  temperature_2m_window_1D_mean
  temperature_2m_window_1D_max
  temperature_2m_window_1D_min
  relative_humidity_2m_window_1D_mean
  relative_humidity_2m_window_1D_max
  relative_humidity_2m_window_1D_min
  precipitation_window_1D_mean
  precipitation_window_1D_max
  precipitation_window_1D_min
  rain_window_1D_mean
```

```
rain_window_1D_max
rain_window_1D_min
snowfall_window_1D_mean
snowfall_window_1D_max
snowfall_window_1D_min
weather_code_window_1D_mean
weather_code_window_1D_max
weather_code_window_1D_min
pressure_msl_window_1D_mean
pressure_msl_window_1D_max
pressure_msl_window_1D_min
surface_pressure_window_1D_mean
surface_pressure_window_1D_max
surface_pressure_window_1D_min
cloud_cover_window_1D_mean
cloud_cover_window_1D_max
cloud_cover_window_1D_min
cloud_cover_low_window_1D_mean
cloud_cover_low_window_1D_max
cloud_cover_low_window_1D_min
cloud_cover_mid_window_1D_mean
cloud_cover_mid_window_1D_max
cloud_cover_mid_window_1D_min
cloud_cover_high_window_1D_mean
cloud_cover_high_window_1D_max
cloud_cover_high_window_1D_min
wind_speed_10m_window_1D_mean
wind_speed_10m_window_1D_max
wind_speed_10m_window_1D_min
wind_direction_10m_window_1D_mean
wind_direction_10m_window_1D_max
wind_direction_10m_window_1D_min
wind_gusts_10m_window_1D_mean
wind_gusts_10m_window_1D_max
```

```
wind_gusts_10m_window_1D_min
temperature_2m_window_7D_mean
temperature_2m_window_7D_max
temperature_2m_window_7D_min
relative_humidity_2m_window_7D_mean
relative_humidity_2m_window_7D_max
relative_humidity_2m_window_7D_min
precipitation_window_7D_mean
precipitation_window_7D_max
precipitation_window_7D_min
rain_window_7D_mean
rain_window_7D_max
rain_window_7D_min
snowfall_window_7D_mean
snowfall_window_7D_max
snowfall_window_7D_min
weather_code_window_7D_mean
weather_code_window_7D_max
weather_code_window_7D_min
pressure_msl_window_7D_mean
pressure_msl_window_7D_max
pressure_msl_window_7D_min
surface_pressure_window_7D_mean
surface_pressure_window_7D_max
surface_pressure_window_7D_min
cloud_cover_window_7D_mean
cloud_cover_window_7D_max
cloud_cover_window_7D_min
cloud_cover_low_window_7D_mean
cloud_cover_low_window_7D_max
cloud_cover_low_window_7D_min
cloud_cover_mid_window_7D_mean
cloud_cover_mid_window_7D_max
cloud_cover_mid_window_7D_min
```

```
cloud_cover_high_window_7D_mean
cloud_cover_high_window_7D_max
cloud_cover_high_window_7D_min
wind_speed_10m_window_7D_mean
wind_speed_10m_window_7D_max
wind_speed_10m_window_7D_min
wind_direction_10m_window_7D_mean
wind_direction_10m_window_7D_max
wind_direction_10m_window_7D_min
wind_gusts_10m_window_7D_mean
wind_gusts_10m_window_7D_max
wind_gusts_10m_window_7D_min
is_holiday
month_sin
month_cos
week_sin
week_cos
day_of_week_sin
day_of_week_cos
hour_sin
hour_cos
sunrise_hour_sin
sunrise_hour_cos
sunset_hour_sin
sunset_hour_cos
poly_hour_cos__temperature_2m_window_1D_mean
poly_hour_cos__temperature_2m_window_1D_max
poly_hour_cos__temperature_2m_window_7D_mean
poly_hour_cos__temperature_2m_window_7D_max
poly_hour_cos__weather_code_window_7D_max
poly_hour_cos__pressure_msl_window_7D_max
poly_hour_cos__pressure_msl_window_7D_min
poly_hour_cos__surface_pressure_window_7D_max
poly_hour_cos__surface_pressure_window_7D_min
```

```
  poly_hour_cos__temperature_2m
exogenous_features shape : (38739, 136)
selected for training    : 127 features
        family  count
weather_window     90
 weather/other     23
  cyclical/RBF     12
    polynomial     10
       holiday      1
```

14.6.7 Processing of the Weather Data

`build_exogenous_features()` pulls weather from the Open-Meteo API through `get_weather_features()` (in `spotforecast2_safe.weather.features`). It fetches hourly `temperature_2m` — the dominant load driver below ~5 °C and above ~15 °C — plus fourteen other variables for the Dortmund default location (51.5136 °N, 7.4653 °E; the `state="NW"` setting drives the *holiday* calendar, not the weather coordinates), caches them as a parquet file, and derives the rolling 1-day and 7-day mean/max/min window features enabled by `include_weather_windows=True`. The result is stored as `team4_mt.weather_aligned` and plotted against load below.

The interesting part is the recent edge of the series, because Open-Meteo serves two endpoints with a coverage seam. The *archive* endpoint carries quality-checked history but lags about five days; the *forecast* endpoint carries today and the days ahead, and — via its `past_days` parameter — a configurable number of recent days backwards. `_fetch_hybrid()` stitches them together: it takes the archive up to `now - 5 days` and the forecast with enough `past_days` to bridge that five-day lag plus enough `forecast_days` to reach the prediction horizon. This is the weather analogue of the ENTSO-E publication lag in Section 14.5.7: the most recent days are not yet in the historical record.

A worked example makes the seam concrete. The cell below recomputes, for this render, the two numbers that close it.

```
archive_cutoff = NOW_UTC - pd.Timedelta(days=5)
past_days = (NOW_UTC.normalize() - archive_cutoff.normalize()).days + 1

print(f"now (render)       : {NOW_UTC}")
print(f"archive reaches    : {archive_cutoff}   (Open-Meteo archive lags ~5 d
print(f"forecast past_days : {past_days}         (bridges the 5-day archive l
print(f"ffill tolerance    : 24 h                (longer gaps are refused, no
```

```
now (render)       : 2026-05-31 19:37:53.771067+00:00
archive reaches    : 2026-05-26 19:37:53.771067+00:00   (Open-
Meteo archive lags ~5 days)
forecast past_days : 6         (bridges the 5-day archive lag)
ffill tolerance    : 24 h                (longer gaps are refused, not filled
```

For a render at `now = 2026-05-31 14:00 UTC` the archive reaches only `2026-05-26`, and the forecast is requested with `past_days = 6`, so it returns from `2026-05-25 00:00` onward (with `forecast_days` reaching the next day's horizon). The archive window `[…, 2026-05-26]` and the forecast window `[2026-05-25, 2026-06-03]` overlap and jointly cover the requested range with no hole. Were `past_days` omitted, the forecast would start at `2026-05-31 00:00` and the five days `2026-05-26`…`2026-05-30` (≈120 h) would be fetched by neither endpoint.

That seam is exactly where missing weather data must be handled correctly. After fetching, the frame is reindexed onto a regular hourly grid spanning `[data_start, cov_end]`, where `cov_end` is `end_train` plus the `predict_size` horizon. Two fail-loud tiers then govern any remaining gap, both active because the safe default `fill_missing=False` applies:

- The fetch layer (`_finalize_df`) rejects any interior `NaN` or absent hour and reports the offending timestamps, rather than filling them. Example: a single missing hour `2026-05-20 03:00` raises with that timestamp listed.

- The alignment layer (`get_weather_features`) forward-fills only *short* gaps, up to `MAX_FFILL_GAP_PERIODS = 24` consecutive hours; a longer run raises `WeatherFetchError`. Last observation carried forward (LOCF) is acceptable for a brief outage but not for days. Example: a 3-hour gap holds the `02:00` reading (say 8.4 °C) across `03:00`–`05:00` — a run of $3 \le 24$, tolerated. A 120-hour gap (`2026-05-26`…`2026-05-30`) is a run of $120 > 24$, so it raises `WeatherFetchError`, which `on_weather_failure="skip"` catches: the model then trains *without* weather for that run instead of on a five-day flat line.

This refusal is a direct Art. 10 obligation: a carried-forward value must not masquerade as a measurement, and a multi-day flat temperature would corrupt the single most important exogenous driver. The same data-governance stance appears in Section 14.6.4 and Section 14.6.5.

Because the recent window is now genuinely fetched, Figure 14.9 shows real diurnal temperature variation across the last week of training data — cold spells lifting the load — rather than a flat tail.

```
import matplotlib.pyplot as plt

weather = team4_mt.weather_aligned
temp_cols = [c for c in weather.columns if "temp" in c.lower()]
load = team4_mt.df_pipeline["Actual Load"].dropna()

if temp_cols and len(load) > 0:
    temp_col = temp_cols[0]
    end = load.index.max()
    w = slice(end - pd.Timedelta(days=7), end)
    load_w = load.loc[w]
    temp_w = weather.loc[w, temp_col]

    fig, ax_load = plt.subplots(figsize=(8.5, 3.5))
    ax_load.plot(load_w.index, load_w.values / 1000, color="#1F4E79", linewidth=1.0)
```

```
    ax_load.set_ylabel("Load [GW]", color="#1F4E79")
    ax_load.tick_params(axis="y", labelcolor="#1F4E79")
    ax_load.set_xlabel("Time")
    ax_load.grid(True, color="#E5E5E5", linewidth=0.5)

    ax_temp = ax_load.twinx()
    ax_temp.plot(temp_w.index, temp_w.values, color="#E07B00", linewidth=1.0)
    ax_temp.set_ylabel(f"{temp_col} [degC]", color="#E07B00")
    ax_temp.tick_params(axis="y", labelcolor="#E07B00")

    ax_load.set_title("Load vs. temperature (last week of training data)")
    for s in ("top",):
        ax_load.spines[s].set_visible(False)
        ax_temp.spines[s].set_visible(False)
    fig.autofmt_xdate()
    fig.tight_layout()
    plt.show()
else:
    print("no temperature-like weather column available; skipping weather plo
```

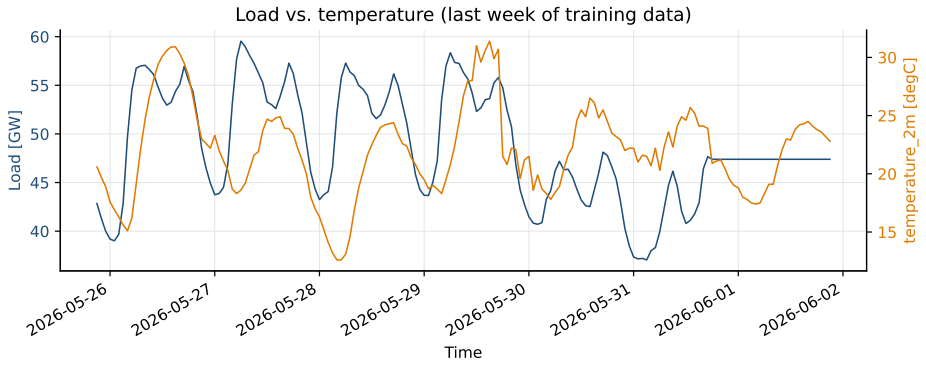


Figure 14.9: Weekly overlay of temperature and load on twin y-axes: the temperature-like weather column (orange, right axis) against the curated `Actual Load` (blue, left axis). Cold spells lift the load; mild days dampen the midday peak.

```
import matplotlib.pyplot as plt

rbf_cols = [c for c in exog_all.columns
            if any(k in c.lower() for k in ("sin", "cos", "rbf"))]

if rbf_cols:
    end = exog_all.index.max()
    w = slice(end - pd.Timedelta(days=2), end)
    block = exog_all.loc[w, rbf_cols]
    fig, ax = plt.subplots(figsize=(8.5, 3.0))
    for c in rbf_cols[:12]:
        ax.plot(block.index, block[c].values, linewidth=0.9, alpha=0.7)
    ax.set_xlabel("Time")
    ax.set_ylabel("encoding value")
    ax.set_title(f"Cyclical/RBF encodings ({len(rbf_cols)} columns, first 12 shown)")
    ax.grid(True, color="#E5E5E5", linewidth=0.5)
    for s in ("top", "right"):
```

```
        ax.spines[s].set_visible(False)
    fig.autofmt_xdate()
    fig.tight_layout()
    plt.show()
else:
    print("no cyclical/RBF columns found; skipping RBF plot")
```

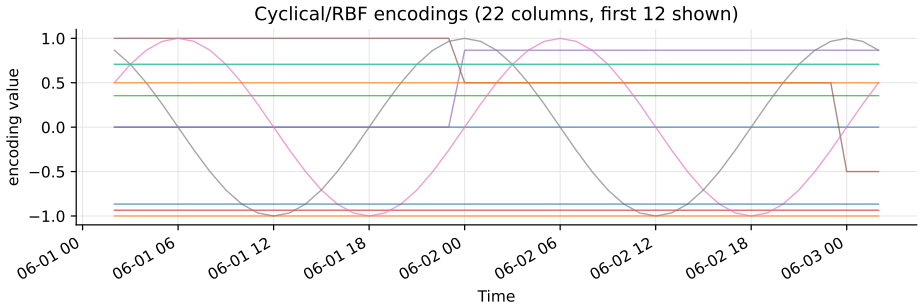


Figure 14.10: Cyclical / radial basis function (RBF) calendar encodings over a two-day window. Each curve is one basis function; together they encode time-of-day and day-of-week position so the model sees cyclic continuity across the midnight and week boundaries.

```
import matplotlib.pyplot as plt

holiday_cols = [c for c in exog_all.columns if "holiday" in c.lower()]
load = team4_mt.df_pipeline["Actual Load"].dropna()

if holiday_cols and len(load) > 0:
    hol_col = holiday_cols[0]
    hol = exog_all[hol_col]
    total_holiday_hours = int((hol > 0).sum())
    print(f"holiday column     : {hol_col}")
    print(f"total holiday hours: {total_holiday_hours}")
```

```
    # Window around the most recent holiday hour, falling back to the tail.
    recent_hol = hol[hol > 0]
    if not recent_hol.empty:
        center = recent_hol.index.max()
    else:
        center = load.index.max()
    w = slice(center - pd.Timedelta(days=7), center + pd.Timedelta(days=3))
    load_w = load.loc[load.index.isin(load.loc[w].index)]
    hol_w = hol.loc[w]

    fig, ax = plt.subplots(figsize=(8.5, 3.0))
    ax.plot(load_w.index, load_w.values / 1000, color="#1F4E79", linewidth=1.0)
    in_holiday = hol_w[hol_w > 0].index
    for t in in_holiday:
        ax.axvspan(t, t + pd.Timedelta(hours=1), color="#E07B00", alpha=0.2)
    ax.set_xlabel("Time")
    ax.set_ylabel("Load [GW]")
    ax.set_title("Load with holiday hours shaded")
    ax.grid(True, color="#E5E5E5", linewidth=0.5)
    for s in ("top", "right"):
        ax.spines[s].set_visible(False)
    fig.autofmt_xdate()
    fig.tight_layout()
    plt.show()
else:
    print("no holiday column found; skipping holiday plot")
```

```
holiday column     : is_holiday
total holiday hours: 1200
```

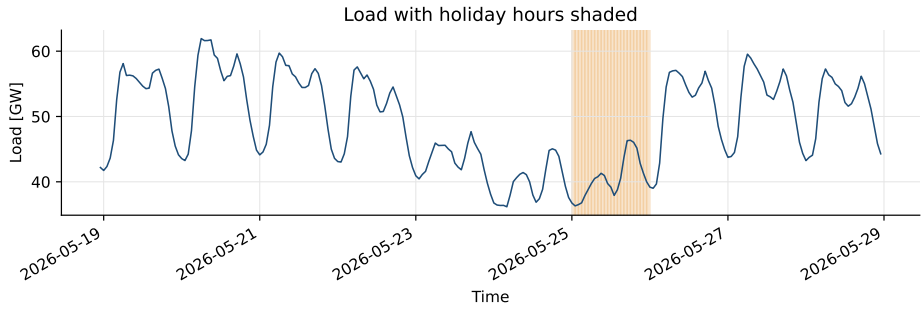


Figure 14.11: Holiday indicator over a representative window: the curated `Actual Load` with holiday hours highlighted. Public holidays shift weekday load downward, which the holiday feature lets the model encode.

```
import matplotlib.pyplot as plt

# With poly_features_degree=2 the pipeline emits poly_<a>__<b> interaction
# columns, then keeps the top max_poly_features by mutual information.
poly_cols = [c for c in exog_all.columns if c.startswith("poly_")]
selected_poly = [c for c in team4_mt.exog_feature_names if c.startswith("poly
print(f"poly_* columns generated : {len(poly_cols)}")
print(f"poly_* selected (cap = {team4_config.max_poly_features}): {len(select
for c in selected_poly:
    print(f"  {c}")

# Demonstrate that a poly_<a>__<b> column equals the product of its two bases
demonstrated = False
for poly_col in selected_poly or poly_cols:
    parts = poly_col[len("poly_"):].split("__")
    if len(parts) == 2 and parts[0] in exog_all.columns and parts[1] in exog_
        a, b = parts
        product = exog_all[a] * exog_all[b]
        corr = float(exog_all[poly_col].corr(product))
```

```
        print(f"{poly_col} == {a} * {b}  (corr = {corr:.4f})")
        fig, ax = plt.subplots(figsize=(8.5, 3.0))
        ax.scatter(product.values, exog_all[poly_col].values, s=3, alpha=0.3, color="#1F4E79
        ax.set_xlabel(f"{a} * {b}")
        ax.set_ylabel(poly_col)
        ax.set_title("Bilinear interaction equals the product of its base features")
        ax.grid(True, color="#E5E5E5", linewidth=0.5)
        for s in ("top", "right"):
            ax.spines[s].set_visible(False)
        fig.tight_layout()
        plt.show()
        demonstrated = True
        break
if not demonstrated:
    print("no recoverable poly_<a>__<b> column to demonstrate")

print(f"\n`select_exogenous_features` pruned {exog_all.shape[1]} encoded columns "
      f"down to the {len(team4_mt.exog_feature_names)} features actually used for training."
```

```
poly_* columns generated : 10
poly_* selected (cap = 10): 10
  poly_hour_cos__temperature_2m_window_1D_mean
  poly_hour_cos__temperature_2m_window_1D_max
  poly_hour_cos__temperature_2m_window_7D_mean
  poly_hour_cos__temperature_2m_window_7D_max
  poly_hour_cos__weather_code_window_7D_max
  poly_hour_cos__pressure_msl_window_7D_max
  poly_hour_cos__pressure_msl_window_7D_min
  poly_hour_cos__surface_pressure_window_7D_max
  poly_hour_cos__surface_pressure_window_7D_min
  poly_hour_cos__temperature_2m
poly_hour_cos__temperature_2m_window_1D_mean == hour_cos * temperature_2m_window_1D_mean  (c
```

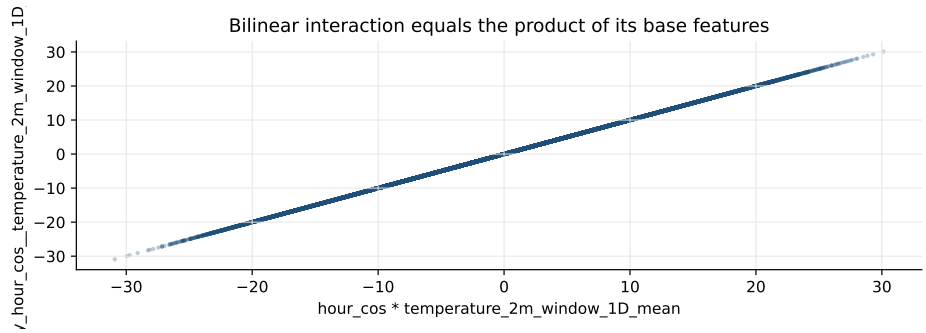


Figure 14.12: Bilinear polynomial expansion produced by the pipeline at `poly_features_degree=2`. Each `poly_<a>__<b>` column is the elementwise product of two base features; the cap keeps only the `max_poly_features` most informative ones by mutual information. The scatter confirms one such column equals the product of its two bases (correlation ≈ 1).

```
`select_exogenous_features` pruned 136 encoded columns down to the 127 featu
```

14.6.8 Step 5 — Train with SpotOptim and predict through tomorrow

With the curated, gap-free target and the selected exogenous matrix in place, `run()` executes the SpotOptim search and refits the LGBM forecaster. With the advanced configuration this cell typically takes 5–10 minutes on a recent laptop — mostly the SpotOptim trials against the CV folds.

> **Reading the SpotOptim progress bars**
>
> Because the cell below calls `run(show=True)`, SpotOptim prints a `tqdm` progress bar every time it scores a candidate model — one bar per rolling

cross-validation back-test. The numbers to the right of a finished bar, e.g. `70/70 [00:00<00:00, 259.84it/s]`, are `tqdm`'s standard read-out:

- `70/70` — steps completed out of the total; here all 70 are done (100 %).
- `[00:00<00:00, 259.84it/s]` follows the pattern `[elapsed<remaining, rate]`:
  - the first `00:00` is the wall-clock time already spent on the bar (`mm:ss`, or `h:mm:ss` once it passes an hour);
  - the second `00:00` is `tqdm`'s estimate of the time still to go — zero here, because the bar has just finished;
  - `259.84it/s` is the throughput, about 260 iterations per second; below one per second `tqdm` switches to its reciprocal `s/it` (seconds per iteration).

A bar caught mid-run, `55/70 [00:00<00:00, 240.00it/s]`, would then read: 55 of 70 steps done, at about 240 steps per second.
The total, 70, is the number of *folds* in the back-test that scores one candidate. The search reserves a validation span of `delta_val = PREDICT_SIZE × REFIT_SIZE × NUMBER_FOLDS = 24 × 7 × 10 = 1680` hours (70 days, Section 14.6.2) and tiles it into blocks of `cv_block_size` hours. The config sets `cv_block_size = PREDICT_SIZE = 24`, decoupled from the live `predict_size = LIVE_N_STEPS` (Section 14.5.6), so

$$\#\text{folds} = \frac{\texttt{delta_val}}{\texttt{cv_block_size}} = \frac{1680}{24} = 70.$$

Because `cv_block_size` is fixed, this total is the *same on every render* — it does not drift with the live horizon. One such bar appears for roughly every candidate the search scores (`n_initial_spotoptim + n_trials_spotoptim`). Section 14.6.2 derives this geometry in full and explains why the CV block is `cv_block_size`, not the live `predict_size`. These bars are progress reporting only. They do not affect the result. Passing `show=False` (as Chapter 13 does) suppresses them entirely.

```
team4_mt.run(show=True)

future = team4_mt.results["spotoptim"]["Actual Load"]["future_pred"]
print(f"raw forecast: {len(future)} hourly steps "
      f"| {future.index.min()} → {future.index.max()}")

y0 = future.loc[TOMORROW_UTC:LAST_TARGET]
print(f"y_0 slice   : {len(y0)} hourly steps "
      f"| {y0.index.min()} → {y0.index.max()}")
```

```
INFO:spotforecast2.multitask.base.MultiTask:Training window: 2023-
06-01 18:00:00+00:00 to 2026-05-31 18:00:00+00:00
INFO:spotforecast2.multitask.base.MultiTask:[task 4] Target 'Actual Load': ta
INFO:spotforecast2.multitask.base.MultiTask:  Warm-start lags seeded: [1, 2,


  0%|          | 0/70 [00:00<?, ?it/s]


  0%|          | 0/70 [00:00<?, ?it/s]


  0%|          | 0/70 [00:00<?, ?it/s]


  0%|          | 0/70 [00:00<?, ?it/s]


  0%|          | 0/70 [00:00<?, ?it/s]


  0%|          | 0/70 [00:00<?, ?it/s]
```

```
0%|          | 0/70 [00:00<?, ?it/s]

0%|          | 0/70 [00:00<?, ?it/s]

0%|          | 0/70 [00:00<?, ?it/s]

0%|          | 0/70 [00:00<?, ?it/s]

0%|          | 0/70 [00:00<?, ?it/s]

0%|          | 0/70 [00:00<?, ?it/s]

0%|          | 0/70 [00:00<?, ?it/s]

0%|          | 0/70 [00:00<?, ?it/s]

0%|          | 0/70 [00:00<?, ?it/s]

0%|          | 0/70 [00:00<?, ?it/s]

0%|          | 0/70 [00:00<?, ?it/s]
```

```
  0%|          | 0/70 [00:00<?, ?it/s]

  0%|          | 0/70 [00:00<?, ?it/s]

  0%|          | 0/70 [00:00<?, ?it/s]

  0%|          | 0/70 [00:00<?, ?it/s]

  0%|          | 0/70 [00:00<?, ?it/s]

  0%|          | 0/70 [00:00<?, ?it/s]

  0%|          | 0/70 [00:00<?, ?it/s]

  0%|          | 0/70 [00:00<?, ?it/s]

  0%|          | 0/70 [00:00<?, ?it/s]

  0%|          | 0/70 [00:00<?, ?it/s]

  0%|          | 0/70 [00:00<?, ?it/s]

  0%|          | 0/70 [00:00<?, ?it/s]
```

```
  0%|          | 0/70 [00:00<?, ?it/s]



  0%|          | 0/70 [00:00<?, ?it/s]


INFO:spotforecast2.multitask.base.MultiTask:  Best params: {'estimator__num_leaves': 122, 'e
INFO:spotforecast2.multitask.base.MultiTask:  Best lags: [ 1  2 23 24 47 48]
INFO:spotforecast2.multitask.base.MultiTask:  Saved tuning results: /Users/bartz/.spotforeca
live-team4_Actual Load_spotoptim_20260531_195113.json


`Forecaster` refitted using the best-found lags and parameters, and the whole data set:
  Lags: [ 1  2 23 24 47 48]
  Parameters: {'estimator__num_leaves': 122, 'estimator__max_depth': 5, 'estimator__learning
  Backtesting metric: 1935.868793601971



Unable to display output for mime type(s): text/html


INFO:spotforecast2.multitask.base.MultiTask:  Saved model: /Users/bartz/.spotforecast2_cache
live-team4/ddmo-live-team4_Actual Load_spotoptim_20260531_195118.joblib


raw forecast: 29 hourly steps | 2026-05-31 19:00:00+00:00 → 2026-
06-01 23:00:00+00:00
y_0 slice   : 24 hourly steps | 2026-06-01 00:00:00+00:00 → 2026-
06-01 23:00:00+00:00
```

14.6.9 Step 6 — Feature importance and SHAP

The fitted forecaster lives in the results dict. We reconstruct its training design matrix — lags plus selected exogenous features — and read two complementary attributions: LightGBM's split-count importance and SHapley Additive exPlanations (SHAP). Both answer the Art. 13 transparency question *what does the model base its forecast on?* The cell also asserts a key data-governance invariant: ENTSO-E's own `Forecasted Load` is held out as test data and never enters the model — only the post-processing comparison in Section 14.6.11 may read it.

```
fc = team4_mt.results["spotoptim"]["Actual Load"]["forecaster"]

X_tr, y_int = fc.create_train_X_y(
    y=team4_mt.data_with_exog["Actual Load"],
    exog=team4_mt.data_with_exog[team4_mt.exog_feature_names],
)
assert list(X_tr.columns) == list(fc.estimator.feature_name_), \
    "design-matrix columns do not match the fitted estimator's feature names"

feat_names = list(fc.estimator.feature_name_)
print(f"trained features: {len(feat_names)}")

# Safety invariant (CR-3): ENTSO-E's own day-ahead forecast must NEVER reach
# the model. It is neither a target nor a selected exogenous feature, so
# `merge_data_and_covariates` drops it before training --- we assert that
# here, fail-loud, so any future regression that let it leak would break the
# render rather than silently bias the model.
LEAKAGE_COLS = {"Forecasted Load", "Actual"}
assert not (LEAKAGE_COLS & set(team4_mt.data_with_exog.columns)), \
    "ENTSO-E forecast leaked into the training frame (data_with_exog)!"
assert not (LEAKAGE_COLS & set(team4_mt.exog_feature_names)), \
    "ENTSO-E forecast leaked into the selected exogenous features!"
assert not (LEAKAGE_COLS & set(feat_names)), \
    "ENTSO-E forecast leaked into the fitted model's features!"
```

```
print("leakage guard passed: ENTSO-E forecast absent from target, exog, and model features")

fam_counts = {}
for col in feat_names:
    fam_counts[family_of(col)] = fam_counts.get(family_of(col), 0) + 1
for fam, n in sorted(fam_counts.items(), key=lambda kv: kv[1], reverse=True):
    print(f"  {fam:<16}: {n}")
```

```
trained features: 134
leakage guard passed: ENTSO-E forecast absent from target, exog, and model features
  weather_window  : 90
  weather/other   : 16
  cyclical/RBF    : 12
  polynomial      : 10
  lag             : 6
```

```
import matplotlib.pyplot as plt

importances = fc.estimator.feature_importances_
ranking = sorted(zip(feat_names, importances), key=lambda kv: kv[1], reverse=True)[:20]

family_color = {
    "lag":            "#B22222",
    "weather/other":  "#E07B00",
    "weather_window": "#F0A33C",
    "cyclical/RBF":   "#1F4E79",
    "holiday":        "#2E8B57",
    "polynomial":     "#7B5EA7",
}

labels = [n for n, _ in ranking][::-1]
values = [v for _, v in ranking][::-1]
```

```
colors = [family_color.get(family_of(n), "#888888") for n in labels]

fig, ax = plt.subplots(figsize=(8.5, 5.5))
ax.barh(labels, values, color=colors)
ax.set_xlabel("split count (feature importance)")
ax.set_title("Top-20 feature importances (coloured by family; lags in red)")
ax.grid(True, axis="x", color="#E5E5E5", linewidth=0.5)
handles = [plt.Rectangle((0, 0), 1, 1, color=c) for c in family_color.values
ax.legend(handles, family_color.keys(), fontsize=7, loc="lower right")
for s in ("top", "right"):
    ax.spines[s].set_visible(False)
fig.tight_layout()
plt.show()
```

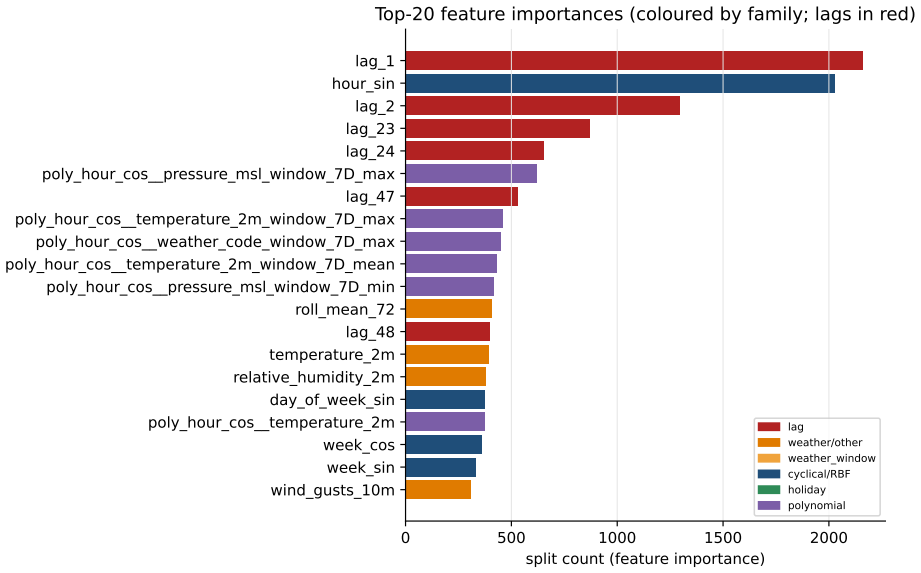


Figure 14.13: Top-20 LightGBM split-count feature importances of the tuned forecaster, coloured by feature family. Lag features (highlighted) carry the bulk of the information; weather, calendar, holiday, and polynomial features supplement them.

```
import shap
import matplotlib.pyplot as plt

step = max(1, len(X_tr) // 2000)
X_sample = X_tr.iloc[::step]

explainer = shap.TreeExplainer(fc.estimator)
sv = explainer.shap_values(X_sample)

shap.summary_plot(sv, X_sample, plot_type="bar", show=False)
plt.tight_layout()
```

```
plt.show()
```



Figure 14.14: Mean absolute SHapley Additive exPlanations (SHAP) value per feature over a ≈2000-row deterministic sample of the training design matrix. SHAP measures each feature's average contribution to the prediction, complementing the split-count ranking above.

Split-count importance ranks features by how often LightGBM branches on them, while mean absolute SHAP ranks them by their average effect on the forecast — the two orderings can disagree, and reporting both is the kind of layered explanation Art. 13 transparency expects.

14.6.10 Visual inspection

`make_plot` (its `PredictionFigure` builder) already knows how to draw a benchmark line: whenever the prediction package carries a `future_forecast` Series, the figure adds a "Benchmark Forecast (e.g., ENTSOE)" trace for free. So overlaying ENTSO-E's own day-ahead forecast is a one-liner — assign `predictions_entsoe` (Section 14.5.4) to that key before calling `make_plot`, rather than appending a Plotly trace by hand. The series must be in raw MW (the same units `future_pred` uses) and is reindexed onto the forecast horizon so it lines up hour-for-hour. The benchmark legend also reads optional `metrics_forecast` / `metrics_forecast_one_day` entries; we leave them unset — the target day is not yet scorable (Section 14.6.11) — so those MAE/MAPE figures render as zero placeholders until the actuals arrive.

```
from spotforecast2.plots.plotter import make_plot

pkg = team4_mt.results["spotoptim"]["Actual Load"]

# Built-in benchmark trace: PredictionFigure draws a "Benchmark Forecast" li
# whenever the package carries a `future_forecast` Series (raw MW, like
# `future_pred`). Reuse predictions_entsoe, reindexed onto the forecast horiz
pkg["future_forecast"] = predictions_entsoe.reindex(pkg["future_pred"].index)

print("future_pred     :", len(pkg["future_pred"]), "rows,",
      pkg["future_pred"].index.min(), "→", pkg["future_pred"].index.max())
print("future_forecast :", int(pkg["future_forecast"].notna().sum()),
      "of", len(pkg["future_forecast"]), "ENTSO-E hours populated")
print("metrics_future         :", pkg.get("metrics_future"))
```

```
print("metrics_future_one_day :", pkg.get("metrics_future_one_day"))

fig = make_plot(pkg)
fig.show()
```

```
future_pred     : 29 rows, 2026-05-31 19:00:00+00:00 → 2026-
06-01 23:00:00+00:00
future_forecast : 27 of 29 ENTSO-E hours populated
metrics_future         : {}
metrics_future_one_day : {}
```

```
Unable to display output for mime type(s): text/html
```

Figure 14.15: team_4 live ENTSO-E DE load forecast — advanced setup (6 SpotOptim trials, 5 seasonal periods, weather windows, holiday and polynomial features). Training fit left of the dashed line; recursive forecast through tomorrow 23:00 UTC right of it. The 24 hourly values for y_0 are the rightmost block. The benchmark trace is ENTSO-E's own day-ahead Forecasted Load, an independent baseline the model never trains on (Section 14.5.4).

14.6.11 ENTSO-E's own day-ahead forecast vs. ours

ENTSO-E publishes a day-ahead Forecasted Load alongside the actuals (Section 14.6.3). The model never uses it — it is neither the target nor an exogenous feature — but it is a free, independent baseline: the transmission system operators' own prediction for the same 24 hours. The cell below restricts predictions_entsoe (Section 14.5.4) to y_0's window and overlays it on team_4's forecast. Neither curve can be scored yet (the target day's actuals are not observed), so this is a forecast-vs-forecast comparison; the printed mean

absolute difference quantifies how far the two disagree. The day-ahead forecast reaches the interim file because the download in Section 14.5.2 passes `keep_forecast_future=True` (without it, `download_new_data` drops every future row). It is published around midday `D-1`, so a render before that window — or the last UTC hours of the target day, which fall into the next local publication day — may leave a few target hours uncovered; the cell handles that gracefully and draws only the hours ENTSO-E has actually published.

```
import matplotlib.pyplot as plt

# Reuse predictions_entsoe (@sec-team4-entsoe-predictions) --- already on the
# hourly grid --- restricted to y_0's 24-hour window.
entsoe_fc = predictions_entsoe.reindex(y0.index)
overlap = entsoe_fc.dropna().index
print(f"ENTSO-E day-ahead hours available on target day: {len(overlap)} / {le
if len(overlap) > 0:
    mad = float((y0.loc[overlap] - entsoe_fc.loc[overlap]).abs().mean())
    print(f"mean |team_4 - ENTSO-E| over the overlap: {mad:,.0f} MW")
else:
    print("ENTSO-E has not published a day-ahead forecast for the target day
          "in the local cache yet; showing team_4 only.")

fig, ax = plt.subplots(figsize=(8.5, 3.5))
ax.plot(y0.index, y0.values / 1000, marker="o", ms=3, color="#1F4E79",
        label="team_4 forecast (y_0)")
if len(overlap) > 0:
    ax.plot(entsoe_fc.index, entsoe_fc.values / 1000, marker="x", ms=4,
            color="#E07B00", label="ENTSO-E day-ahead forecast")
ax.set_xlabel("Time (UTC)")
ax.set_ylabel("Load [GW]")
ax.set_title("team_4 vs. ENTSO-E day-ahead forecast --- target day")
ax.grid(True, color="#E5E5E5", linewidth=0.5)
ax.legend(fontsize=8)
for s in ("top", "right"):
```

```
    ax.spines[s].set_visible(False)
fig.autofmt_xdate()
fig.tight_layout()
plt.show()
```

```
ENTSO-E day-ahead hours available on target day: 22 / 24
mean |team_4 - ENTSO-E| over the overlap: 4,988 MW
```

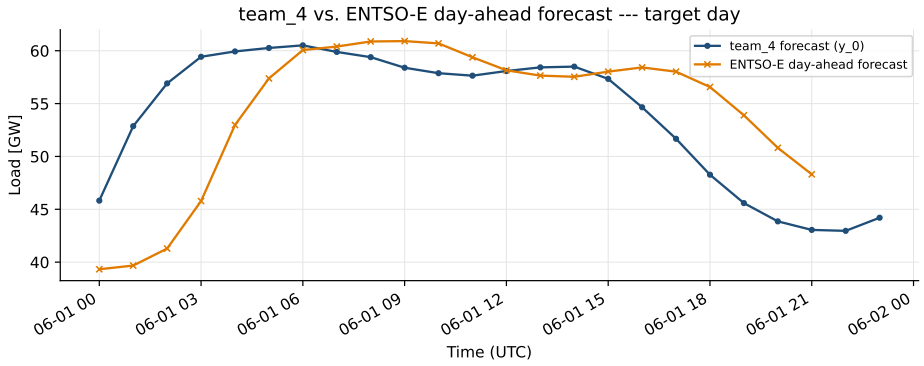


Figure 14.16: ENTSO-E's published day-ahead Forecasted Load for the target day (orange) vs. team_4's forecast `y_0` (blue), on the same 24 hourly steps. Neither is scored — the target day's actuals are not yet observed — so this contrasts two forecasts, not forecast against truth.

14.6.12 Validate the schema and write the submission CSV

Same four contract assertions as Section 13.5: 24 rows, first stamp `TOMORROW 00:00 UTC`, every value strictly positive, no NaN. The write target is `submissions/team_4/<tomorrow>.csv` inside the leaderboard fork.

```
assert len(y0) == 24,           f"expected 24 hourly steps for y_0, got {len
assert y0.index.min() == TOMORROW_UTC, (
    f"first step {y0.index.min()} != TOMORROW {TOMORROW_UTC}"
)
assert (y0 > 0).all(),          "non-positive forecast value --- spec require
assert y0.notna().all(),        "NaN in forecast --- spec forbids"

TEAM_ID = "team_4"
SUBMISSION_DIR = Path.home() / "workspace" / "challenge-leaderboard" / "submi
SUBMISSION_DIR.mkdir(parents=True, exist_ok=True)
submission_path = SUBMISSION_DIR / f"{TOMORROW_UTC.date().isoformat()}.csv"

submission_df = pd.DataFrame({
    "timestamp_utc": y0.index.strftime("%Y-%m-%dT%H:%M:%SZ"),
    "forecast_mw":   y0.round(2).values,
})
submission_df.to_csv(submission_path, index=False)
print(f"wrote {submission_path}")
print(f"  rows: {len(submission_df)}")
print(f"  range: {submission_df.forecast_mw.min():.1f}-{submission_df.forecas
submission_df.head(3)
```

```
wrote /Users/bartz/workspace/challenge-leaderboard/submissions/team_4/2026-
06-01.csv
  rows: 24
  range: 42959.2-60509.2 MW
```

	timestamp_utc	forecast_mw
0	2026-06-01T00:00:00Z	45817.90
1	2026-06-01T01:00:00Z	52863.44
2	2026-06-01T02:00:00Z	56910.18

14.6.13 Local validation with the leaderboard's own script

```
import subprocess
lb_repo = Path.home() / "workspace" / "challenge-leaderboard"
result = subprocess.run(
    [
        "uv", "run", "python", "scripts/validate_submission.py",
        "--path", str(submission_path.relative_to(lb_repo)),
        "--skip-deadline",
    ],
    cwd=lb_repo,
    capture_output=True,
    text=True,
)
print("exit code:", result.returncode)
print("stdout   :", result.stdout.strip())
if result.stderr.strip():
    print("stderr   :", result.stderr.strip())
```

```
exit code: 0
stdout   : OK: team=team_4 target_date=2026-06-01 file=submissions/team_4/2026-
06-01.csv
```

14.7 Commit and push the submission PR

The cells above only *write* the CSV — they deliberately do not commit, push, or open the PR (a careless render must not be able to spam the leaderboard). Run the block below from a shell once the validation cell above is green:

```
cd ~/workspace/challenge-leaderboard
# TARGET=$(date -u -d 'tomorrow' +%F)                # Linux equivalent
TARGET=$(date -u -v+1d +%F)
# Unique branch per attempt: re-submitting for the same target day must not
# collide with an earlier (already-merged) branch of the same name.
BRANCH="submission/team_4-${TARGET}-$(date -u +%H%M%S)"
git switch -c "$BRANCH" main
git add "submissions/team_4/${TARGET}.csv"
git commit -m "submission(team_4): forecast for ${TARGET}"
git push -u origin "$BRANCH"

gh pr create --base main --repo bartzbeielstein/challenge-leaderboard \
    --head "$BRANCH" \
    --title "submission(team_4): forecast for ${TARGET}" \
    --body  "Generated by ddmo-sose-26 chapter 14 (team_4 advanced submissio
```

If validation on the PR passes, auto-merge squashes the submission onto `main`. The next morning at 07:00 UTC the leaderboard's `score-daily.yml` cron (see Section 13.7) scores it against ENTSO-E final actuals.

> **Deadline**
>
> Submissions for target day `D` must be merged before `D-1 23:59 UTC` — the validator enforces this strictly. The advanced configuration in this chapter renders in 5–10 minutes on a typical laptop; give yourself at least 30 minutes of head-room before the deadline.

14.8 What changed vs. chapter 13 — summary

Knob	Chapter 13	This chapter
`periods`	`daily` only	`daily, weekly, monthly, quarterly, yearly`
`lags_consider`	`[1, 2, 24]`	data-selected via PACF (1)
`warm_start_lags`	`False` (default)	`True`
`include_weather_windows`	(default)	`True`
`include_holiday_features`	(default)	`True`
`poly_features_degree`	1 (off)	2
`max_poly_features`	10	10
`n_trials_spotoptim`	10	6
`n_initial_spotoptim`	5	5
`number_folds`	10	10
Team id	`<team_id>` placeholder	`team_4` (hardcoded)

Exercises (with Claude)

1. *Did the bigger budget pay off?* Compare `metrics_future_one_day` from this chapter against the same metric in Section 13.5 for the same target day. Discuss with Claude whether the gain (or loss) justifies the extra ~5 minutes of compute.
2. *Ablate a feature.* Set `poly_features_degree=1` (interactions off) and re-render only the train + write cells. Does the MAE move noticeably? If not, the interactions are not load-bearing for this dataset.
3. *Inspect the tuned hyperparameters.* Print `team4_mt.results["spotoptim"]["Actual Load"]` keys and locate the best-trial summary. Which lag configuration did SpotOptim pick — did the warm-started `key_lags` win, or did a hardcoded candidate beat it?
4. *How much does the CV block matter?* This chapter fixes the CV block at `cv_block_size=24`, decoupled from the live

`predict_size=LIVE_N_STEPS` (Section 14.6.2). Re-run the train cell with `cv_block_size=LIVE_N_STEPS` instead — re-coupling the two — and compare the tuned hyperparameters and `metrics_future_one_day`, and watch the progress-bar total change from `70` to `1680/LIVE_N_STEPS`. Discuss with Claude why a recursive forecaster's CV score depends on the block length, and whether the fixed 24-hour block or the longer live horizon better matches the leaderboard's 24-hour metric.

15 ENTSO-E day-ahead baseline submission

Learning objective

You can submit the *transmission system operators' own* 24-hour DE load forecast — ENTSO-E's published day-ahead `Forecasted Load` — as a zero-model baseline on the live leaderboard, under the registered team `entsoe`. The chapter shows why this baseline is the honest yardstick every tuned forecaster in Chapter 13 and Chapter 14 must beat, and writes the validated CSV into the leaderboard fork.

15.1 Overview

Chapter 13 tunes an Optuna LightGBM forecaster; Chapter 14 tunes a richer SpotOptim variant. Both pull ENTSO-E's own day-ahead `Forecasted Load` purely as a comparison overlay (Section 14.6.11) and never let the model train on it. This chapter inverts that relationship: it submits ENTSO-E's day-ahead forecast *directly*, with no model at all.

That is deliberate. ENTSO-E aggregates the forecasts the European transmission system operators already produce for grid scheduling, so it is a strong, free, professionally-maintained baseline. A tuned model that cannot beat it over the rolling 7-day window is not earning its compute. Submitting it as its own team `entsoe` puts that yardstick on the public leaderboard next to the student models, where the comparison is unavoidable.

Every safety contract from Chapter 13 carries over unchanged (24-row schema, strictly-positive forecasts, no NaN). What disappears is the entire modelling stack: no `MultiTask`, no `ConfigEntsoe`, no tuning, no exogenous features. The forecast is a slice of the downloaded ENTSO-E frame. Two wrinkles specific to a *forward* baseline are handled along the way — the safe downloader drops future rows unless asked to keep them (Section 15.5.2), and a UTC target day straddles two local publication days (Section 15.5.3).

15.2 Submission contract

Unchanged — see Section 13.2 for the CSV schema, deadline, and scoring rules. The leaderboard validator that runs against this chapter's output is byte-identical to the one used by Chapter 13. The only difference is the path: `submissions/entsoe/<YYYY-MM-DD>.csv`.

15.3 Team identity

`entsoe` is hardcoded throughout this chapter — there is no placeholder to edit. The cell below confirms the team is registered in `challenge-leaderboard/teams.yml`.

```
from pathlib import Path
import yaml

teams_yml = Path.home() / "workspace" / "challenge-leaderboard" / "teams.yml"
teams = yaml.safe_load(teams_yml.read_text())["teams"]
team = next((t for t in teams if t["id"] == "entsoe"), None)
assert team is not None, "entsoe is not in challenge-leaderboard/teams.yml"
print(f"entsoe display_name : {team['display_name']}")
print(f"entsoe github       : {', '.join(team['github_handles'])}")
```

```
entsoe display_name : Entso-E
entsoe github       : bartzbeielstein
```

`validate_authorship()` in `scripts/validate_submission.py:89` rejects PRs whose author is not in this `github_handles` list, so the `gh auth login` account used in Section 15.6 must match.

15.4 Prerequisites and walkthrough

Identical to Section 13.4: Python 3.13 + uv, a personal ENTSO-E API key exported as `ENTSOE_API_KEY`, `gh auth login` as a registered `entsoe` handle, and a sibling clone of `challenge-leaderboard` at `~/workspace/challenge-leaderboard/`. Because there is no model to train, this chapter renders in seconds — it issues a single API call and slices it.

15.5 Live pipeline

```
import os
from pathlib import Path
import pandas as pd

# Isolate this chapter's storage from chapters 13 and 14 (each pins its own
# download home so the three chapters never read or write the same interim
# file). The package reads the env var at call time via get_data_home().
os.environ["SPOTFORECAST2_DATA"]  = str(Path.home() / "spotforecast2_data"  / "ddmo_ch15_ent
os.environ["SPOTFORECAST2_CACHE"] = str(Path.home() / ".spotforecast2_cache" / "ddmo_ch15_er

from spotforecast2_safe.downloader.entsoe import download_new_data
from spotforecast2_safe.data.fetch_data import get_data_home
```

```
api_key = os.environ.get("ENTSOE_API_KEY")
assert api_key, "ENTSOE_API_KEY must be set in the environment (see @sec-live
```

15.5.1 Dynamic date constants

Every date is derived from `pd.Timestamp.now(tz="UTC").normalize()` so the chapter stays correct on every render — same convention as Section 13.5. `y_0` is tomorrow's UTC day (00:00–23:00 UTC), the 24 hourly stamps the leaderboard scores.

```
NOW_UTC = pd.Timestamp.now(tz="UTC")
TODAY_UTC = NOW_UTC.normalize()
TOMORROW_UTC = TODAY_UTC + pd.Timedelta(days=1)
LAST_TARGET = TOMORROW_UTC + pd.Timedelta(hours=23)

# A day-ahead baseline needs no training history --- only a short recent wind
# enough to carry the last week of `Actual Load` for the context plot plus a
# previous-day fallback (@sec-entsoe-extract). We download the last 10 days a
# reach the end through the day *after* tomorrow so the full target day is
# covered as soon as ENTSO-E publishes it. download_new_data takes the window
# 'YYYYMMDDHH00' strings.
START_DOWNLOAD = (TODAY_UTC - pd.Timedelta(days=10)).strftime("%Y%m%d%H%M")
END_DOWNLOAD = (TOMORROW_UTC + pd.Timedelta(days=1)).strftime("%Y%m%d%H%M")

print(f"NOW_UTC        = {NOW_UTC}")
print(f"TODAY_UTC      = {TODAY_UTC}")
print(f"TOMORROW_UTC   = {TOMORROW_UTC}  (target day = y_0)")
print(f"LAST_TARGET    = {LAST_TARGET}")
print(f"START_DOWNLOAD = {START_DOWNLOAD}  (10 days back --- no training hist
print(f"END_DOWNLOAD   = {END_DOWNLOAD}  (covers tomorrow's day-ahead forecas
```

```
NOW_UTC        = 2026-05-31 20:20:24.505450+00:00
TODAY_UTC      = 2026-05-31 00:00:00+00:00
TOMORROW_UTC   = 2026-06-01 00:00:00+00:00  (target day = y_0)
LAST_TARGET    = 2026-06-01 23:00:00+00:00
START_DOWNLOAD = 202605210000  (10 days back --- no training history)
END_DOWNLOAD   = 202606020000  (covers tomorrow's day-
ahead forecast)
```

15.5.2 Download ENTSO-E's day-ahead `Forecasted Load`

We fetch through the same safe wrapper the training chapters use, `spotforecast2_safe.downloader.entsoe.downloa` with one argument that matters here: `keep_forecast_future=True`. By default the wrapper drops every row after `now` when it merges the download into the interim file —

```
# spotforecast2_safe/downloader/entsoe.py
if not keep_forecast_future:
    merged_df = merged_df.loc[merged_df.index <= pd.Timestamp.now(tz="UTC")]
```

That truncation is correct for the training chapters (a model must never see the future), but it would erase the very thing this baseline submits: ENTSO-E's day-ahead `Forecasted Load` for tomorrow. The flag keeps those future rows; future `Actual Load` is still blank, so nothing leaks — there is no model here in any case. ENTSO-E's `Forecasted Load` is published per *local* (Europe/Berlin) day; the wrapper normalises the merged interim index to UTC, the zone the leaderboard scores in.

```
download_new_data(
    api_key=api_key,
    country_code="DE",
    start=START_DOWNLOAD,
    end=END_DOWNLOAD,
```

```
    force=True,
    keep_forecast_future=True,   # keep tomorrow's day-ahead forecast
)

interim_csv = get_data_home() / "interim" / "energy_load.csv"
interim = pd.read_csv(interim_csv, index_col=0, parse_dates=True)
interim.index = pd.to_datetime(interim.index, utc=True)

print(f"interim rows       : {len(interim)} (15-min, {interim.index.min()} →
print(f"columns            : {list(interim.columns)}")
print(f"Forecasted Load to : {interim['Forecasted Load'].dropna().index.max()
print(f"Actual Load to     : {interim['Actual Load'].dropna().index.max()}")
```

```
interim rows       : 1144 (15-min, 2026-05-21 00:00:00+00:00 → 2026-
06-01 21:45:00+00:00)
columns            : ['Forecasted Load', 'Actual Load']
Forecasted Load to : 2026-06-01 21:45:00+00:00
Actual Load to     : 2026-05-31 19:15:00+00:00
```

We aggregate the 15-minute series onto the hourly grid the leaderboard scores on — the mean of each hour's four quarter-hour values, matching how the tuned chapters aggregate the actuals.

```
forecast_h = interim["Forecasted Load"].resample("h").mean().dropna()
actual_h = interim["Actual Load"].resample("h").mean().dropna()
forecast_h.name = "Forecasted Load"
actual_h.name = "Actual Load"

print(f"hourly forecast : {len(forecast_h)} h | {forecast_h.index.min()} → {
print(f"hourly actual   : {len(actual_h)} h | {actual_h.index.min()} → {actua
```

```
hourly forecast : 286 h | 2026-05-21 00:00:00+00:00 → 2026-
06-01 21:00:00+00:00
```

```
hourly actual   : 260 h | 2026-05-21 00:00:00+00:00 → 2026-
05-31 19:00:00+00:00
```

15.5.3 Assemble the 24-hour target slice

The target is the 24 UTC hours of tomorrow, `00:00` … `23:00`. ENTSO-E publishes its day-ahead forecast per *local* (Europe/Berlin) day, and Germany runs one or two hours ahead of UTC. So at submission time — the afternoon of `D-1`, when only the local `D` day-ahead is out — the last one or two UTC hours of the target (`22:00`, `23:00` UTC) fall into the *next* local day, which ENTSO-E does not publish until `D`, after the `D-1 23:59 UTC` deadline.

Those trailing hours are therefore filled with ENTSO-E's own forecast for the same clock hour 24 hours earlier — a transparent, fully reproducible persistence of the operators' published forecast (today's day-ahead is already complete, so the fallback source always exists). The fill is logged, and a fail-loud guard (CR-3) aborts if even the fallback is missing rather than writing a short or NaN-padded CSV.

```
target_index = pd.date_range(TOMORROW_UTC, LAST_TARGET, freq="h", tz="UTC")  # 24 stamps
y0 = forecast_h.reindex(target_index)

published = int(y0.notna().sum())
missing = y0.index[y0.isna()]
print(f"day-ahead hours published for {TOMORROW_UTC.date()}: {published} / 24")

for ts in missing:
    fallback = forecast_h.reindex([ts - pd.Timedelta(days=1)]).iloc[0]
    if pd.isna(fallback):
        raise RuntimeError(
            f"No day-ahead value for {ts} and no 24 h-earlier fallback either; "
            "ENTSO-E coverage is too thin to assemble 24 hours. Re-render once "
            "the day-ahead forecast for the target day is published."
```

```
        )
    y0.loc[ts] = fallback
    print(f"  filled {ts} ← {ts - pd.Timedelta(days=1)} (24 h-earlier ENTSO-E

y0.name = "Forecasted Load"
print(f"\ny_0 assembled: {len(y0)} hourly steps | {y0.index.min()} → {y0.inde
print(y0)
```

```
day-ahead hours published for 2026-06-01: 22 / 24
  filled 2026-06-01 22:00:00+00:00 ← 2026-05-31 22:00:00+00:00 (24 h-
earlier ENTSO-E forecast = 41,048 MW)
  filled 2026-06-01 23:00:00+00:00 ← 2026-05-31 23:00:00+00:00 (24 h-
earlier ENTSO-E forecast = 39,752 MW)

y_0 assembled: 24 hourly steps | 2026-06-01 00:00:00+00:00 → 2026-
06-01 23:00:00+00:00
2026-06-01 00:00:00+00:00    39327.047789
2026-06-01 01:00:00+00:00    39675.580085
2026-06-01 02:00:00+00:00    41285.199476
2026-06-01 03:00:00+00:00    45776.233649
2026-06-01 04:00:00+00:00    52975.273053
2026-06-01 05:00:00+00:00    57379.896787
2026-06-01 06:00:00+00:00    60062.325011
2026-06-01 07:00:00+00:00    60396.462782
2026-06-01 08:00:00+00:00    60874.850613
2026-06-01 09:00:00+00:00    60911.805267
2026-06-01 10:00:00+00:00    60694.146027
2026-06-01 11:00:00+00:00    59381.019278
2026-06-01 12:00:00+00:00    58150.097945
2026-06-01 13:00:00+00:00    57647.152979
2026-06-01 14:00:00+00:00    57535.920035
2026-06-01 15:00:00+00:00    58022.758576
2026-06-01 16:00:00+00:00    58422.032589
```

```
2026-06-01 17:00:00+00:00    58017.811127
2026-06-01 18:00:00+00:00    56567.745495
2026-06-01 19:00:00+00:00    53892.870363
2026-06-01 20:00:00+00:00    50815.118195
2026-06-01 21:00:00+00:00    48307.254943
2026-06-01 22:00:00+00:00    41047.617584
2026-06-01 23:00:00+00:00    39752.114576
Freq: h, Name: Forecasted Load, dtype: float64
```

15.5.4 Visual inspection

The plot overlays the last week of published `Actual Load` (blue) with ENTSO-E's day-ahead `Forecasted Load` running into the target day (orange). The 24 hourly values we submit are the shaded block; any fallback-filled trailing hours are marked.

```
import matplotlib.pyplot as plt

LAST_WEEK_START = TODAY_UTC - pd.Timedelta(days=7)
actual_view = actual_h.loc[LAST_WEEK_START:]
fc_view = forecast_h.loc[LAST_WEEK_START:]
filled = y0.index[~y0.index.isin(forecast_h.index)]

fig, ax = plt.subplots(figsize=(8.5, 3.5))
ax.plot(actual_view.index, actual_view.values / 1000, color="#1F4E79",
        lw=1.2, label="Actual Load (published)")
ax.plot(fc_view.index, fc_view.values / 1000, color="#E07B00",
        lw=1.2, label="ENTSO-E day-ahead forecast")
ax.plot(y0.index, y0.values / 1000, color="#E07B00", lw=0, marker="o", ms=3)
if len(filled):
    ax.plot(filled, y0.loc[filled].values / 1000, color="#B22222", lw=0,
            marker="x", ms=7, label="fallback (24 h-earlier forecast)")
ax.axvspan(TOMORROW_UTC, LAST_TARGET, color="#E07B00", alpha=0.10,
```

```
            label="target day (y_0)")
ax.set_xlabel("Time (UTC)")
ax.set_ylabel("Load [GW]")
ax.set_title("ENTSO-E DE day-ahead baseline --- last week + target day")
ax.grid(True, color="#E5E5E5", linewidth=0.5)
ax.legend(fontsize=8)
for s in ("top", "right"):
    ax.spines[s].set_visible(False)
fig.autofmt_xdate()
fig.tight_layout()
plt.show()
```

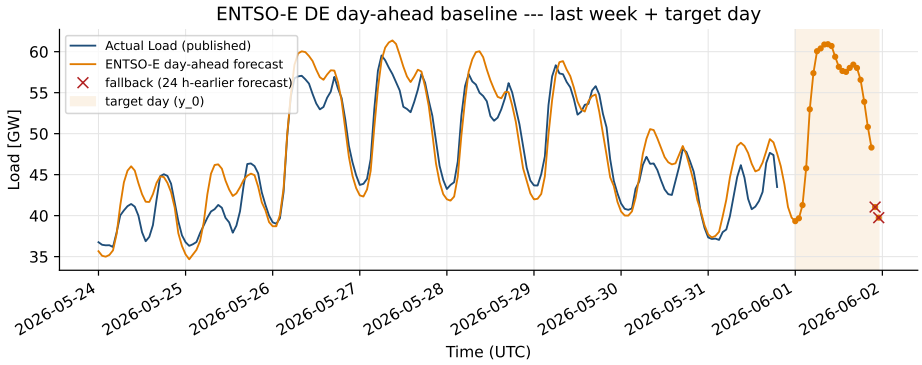


Figure 15.1: ENTSO-E DE day-ahead baseline. Blue: the last week of published `Actual Load`. Orange: ENTSO-E's day-ahead `Forecasted Load`. The 24 hourly values for `y_0` (shaded) are exactly the CSV we submit; red crosses mark any trailing UTC hour filled from the 24 h-earlier forecast (Section 15.5.3).

15.5.5 Validate the schema and write the submission CSV

The same four contract assertions as Section 13.5 guard the write: 24 rows, the first stamp must be `TOMORROW 00:00 UTC`, every value strictly positive, no NaN. Any violation aborts before the file is opened, so a partially-written CSV can never reach the leaderboard.

```
assert len(y0) == 24,           f"expected 24 hourly steps for y_0, got {len(y0)}"
assert y0.index.min() == TOMORROW_UTC, (
    f"first step {y0.index.min()} != TOMORROW {TOMORROW_UTC}"
)
assert (y0 > 0).all(),          "non-positive forecast value --- spec requires > 0"
assert y0.notna().all(),        "NaN in forecast --- spec forbids"

TEAM_ID = "entsoe"
SUBMISSION_DIR = Path.home() / "workspace" / "challenge-leaderboard" / "submissions" / TEAM_
SUBMISSION_DIR.mkdir(parents=True, exist_ok=True)
submission_path = SUBMISSION_DIR / f"{TOMORROW_UTC.date().isoformat()}.csv"

submission_df = pd.DataFrame({
    "timestamp_utc": y0.index.strftime("%Y-%m-%dT%H:%M:%SZ"),
    "forecast_mw":   y0.round(2).values,
})
submission_df.to_csv(submission_path, index=False)
print(f"wrote {submission_path}")
print(f"  rows: {len(submission_df)}")
print(f"  range: {submission_df.forecast_mw.min():.1f}-{submission_df.forecast_mw.max():.1f}
submission_df.head(3)
```

```
wrote /Users/bartz/workspace/challenge-leaderboard/submissions/entsoe/2026-
06-01.csv
  rows: 24
  range: 39327.1-60911.8 MW
```

	timestamp_utc	forecast_mw
0	2026-06-01T00:00:00Z	39327.05
1	2026-06-01T01:00:00Z	39675.58
2	2026-06-01T02:00:00Z	41285.20

15.5.6 Local validation with the leaderboard's own script

A green exit code from `validate_submission.py` is the same check that runs on the PR in CI. We pass `--skip-deadline` because the chapter may be rendered inside the deadline window (the live CI uses the strict form without the flag).

```
import subprocess
lb_repo = Path.home() / "workspace" / "challenge-leaderboard"
result = subprocess.run(
    [
        "uv", "run", "python", "scripts/validate_submission.py",
        "--path", str(submission_path.relative_to(lb_repo)),
        "--skip-deadline",
    ],
    cwd=lb_repo,
    capture_output=True,
    text=True,
)
print("exit code:", result.returncode)
print("stdout   :", result.stdout.strip())
if result.stderr.strip():
    print("stderr   :", result.stderr.strip())
```

```
exit code: 0
stdout   : OK: team=entsoe target_date=2026-06-01 file=submissions/entsoe/202
06-01.csv
```

15.6 Commit and push the submission PR

The cells above only *write* the CSV — they deliberately do not commit, push, or open the PR (a careless render must not be able to spam the leaderboard). Run the block below from a shell once the validation cell above is green:

```
cd ~/workspace/challenge-leaderboard
# TARGET=$(date -u -d 'tomorrow' +%F)                # Linux equivalent
TARGET=$(date -u -v+1d +%F)
# Unique branch per attempt: re-submitting for the same target day must not
# collide with an earlier (already-merged) branch of the same name.
BRANCH="submission/entsoe-${TARGET}-$(date -u +%H%M%S)"
git switch -c "$BRANCH" main
git add "submissions/entsoe/${TARGET}.csv"
git commit -m "submission(entsoe): day-ahead baseline for ${TARGET}"
git push -u origin "$BRANCH"

gh pr create --base main --repo bartzbeielstein/challenge-leaderboard \
    --head "$BRANCH" \
    --title "submission(entsoe): day-ahead baseline for ${TARGET}" \
    --body  "Generated by ddmo-sose-26 chapter 15 (ENTSO-E day-ahead baseline)."
```

If validation on the PR passes, auto-merge squashes the submission onto `main`. The next morning at 07:00 UTC the leaderboard's `score-daily.yml` cron (see Section 13.7) scores it against ENTSO-E final actuals.

> **Deadline**
>
> Submissions for target day `D` must be merged before `D-1 23:59 UTC` — the validator enforces this strictly. This baseline renders in seconds, but the local `D` day-ahead `Forecasted Load` is only published around midday `D-1`; render after that window so the bulk of the 24 target hours come straight from the feed and only the one or two UTC/local boundary hours rely on

the fallback.

15.7 Why a baseline team

The leaderboard ranks on rolling 7-day mean MAE (Section 13.2). Without a reference point, a student team has no way to tell whether an MAE of, say, 1500 MW is good. The `entsoe` row supplies that reference: it is the accuracy the operators' own forecast achieves on the very same scoring rule. Three readings follow directly from its position on the board:

- A team *above* `entsoe` is beating the professional day-ahead forecast — a genuine result worth writing up.
- A team *below* `entsoe` is being out-predicted by a free public series and should revisit features or tuning before adding compute.
- `entsoe`'s own day-to-day MAE variance shows the irreducible difficulty of each target day — a calm Sunday scores tighter than a volatile cold snap, for everyone.

Exercises (with Claude)

1. *Score the baseline yourself.* Once tomorrow becomes a past day with published actuals, re-query `query_load_and_forecast`, align `forecast_h` with the realised `Actual Load`, and compute the 24-hour MAE by hand. Compare it to the leaderboard's scored value.
2. *Beat it on one day.* Render Chapter 14 for the same target day and compare its `y_0` against this baseline's `y_0`. On which hours does the tuned model diverge most from ENTSO-E, and does that help or hurt once the actuals land?
3. *Probe the boundary fill.* How many UTC hours did the fallback in Section 15.5.3 supply today, and why does that count change with the season (CET vs CEST)? Discuss with Claude whether a smarter

fill than 24 h persistence (e.g. the same weekday last week) would be worth the complexity for one or two hours.

References

European Parliament and Council. 2024. *Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 Laying down Harmonised Rules on Artificial Intelligence (AI Act)*. Official Journal of the European Union, L series. https://eur-lex.europa.eu/eli/reg/2024/1689/oj.

Hong, Tao, Pierre Pinson, and Shu Fan. 2014. "Global Energy Forecasting Competition 2012." *International Journal of Forecasting* 30 (2): 357–63. https://doi.org/10.1016/j.ijforecast.2013.07.001.

Hong, Tao, Pierre Pinson, Shu Fan, Hamidreza Zareipour, Alberto Troccoli, and Rob J. Hyndman. 2016. "Probabilistic Energy Forecasting: Global Energy Forecasting Competition 2014 and Beyond." *International Journal of Forecasting* 32 (3): 896–913. https://doi.org/10.1016/j.ijforecast.2016.02.001.

Hong, Tao, Jingrui Xie, and Jonathan Black. 2019. "Global Energy Forecasting Competition 2017: Hierarchical Probabilistic Load Forecasting." *International Journal of Forecasting* 35 (4): 1389–99. https://doi.org/10.1016/j.ijforecast.2019.02.006.

Makridakis, Spyros, Evangelos Spiliotis, and Vassilios Assimakopoulos. 2020. "The M4 Competition: 100,000 Time Series and 61 Forecasting Methods." *International Journal of Forecasting* 36 (1): 54–74. https://doi.org/10.1016/j.ijforecast.2019.04.014.

Makridakis, Spyros, Evangelos Spiliotis, and Vassilios Assimakopoulos. 2022. "M5 Accuracy Competition: Results, Findings, and Conclusions." *Interna-*

tional Journal of Forecasting 38 (4): 1346–64. https://doi.org/10.1016/j.ijforecast.2021.11.013.